

Banking Management System

server/index.js

```
// server/index.js
// Express backend - handles email notifications via NodeMailer.
// Run: cd server && npm install && npm run dev

require('dotenv').config();
const express      = require('express');
const cors         = require('cors');
const { sendKYCEmail } = require('./services/emailService');

const app = express();
const PORT = process.env.PORT || 3001;

// — Middleware


---


app.use(cors({ origin: process.env.ALLOWED_ORIGIN || '*' }));
app.use(express.json());

// — Health check


---


app.get('/health', (req, res) => res.json({ status: 'ok', time: new Date() }));

// — Send KYC email


---


// POST /api/send-email
// Body: { to, customerName, bankName, action, reason }
app.post('/api/send-email', async (req, res) => {
  const { to, customerName, bankName, action, reason } = req.body;

  if (!to || !customerName || !action) {
    return res.status(400).json({ error: 'Missing required fields: to, customerName, action' });
  }

  if (!['approved', 'hold', 'rejected'].includes(action)) {
    return res.status(400).json({ error: 'action must be: approved | hold | rejected' });
  }

  try {
```

```

    const info = await sendKYCEmail({ to, customerName, bankName,
action, reason });
    res.json({ success: true, messageId: info.messageId });
  } catch (err) {
    console.error('[SERVER] Email send failed:', err.message);
    // Return 200 so the KYC action isn't blocked by email failure
    res.json({ success: false, error: err.message });
  }
});

app.listen(PORT, () => {
  console.log(`[SERVER] BMS email server running on
http://localhost:${PORT}`);
});

```

server/services/emailService.js

```

// server/services/emailService.js
// NodeMailer email service - runs on the server only.

const nodemailer = require('nodemailer');

const transporter = nodemailer.createTransport({
  host: process.env.MAIL_HOST,
  port: Number(process.env.MAIL_PORT) || 587,
  secure: false,
  auth: {
    user: process.env.MAIL_USER,
    pass: process.env.MAIL_PASS,
  },
});

// Verify connection on startup
transporter.verify((err) => {
  if (err) console.error('[MAIL] Connection failed:', err.message);
  else console.log('[MAIL] Server ready to send emails');
});

// — Email templates



---



const templates = {

```

```

    approved: (customerName, bankName) => ({
      subject: `Your KYC has been approved - ${bankName}`,
      html: `
        <div style="font-family:Arial,sans-serif;max-width:520px;margin:0
auto;padding:24px">
          <div style="background:#1E3A5F;padding:24px;border-radius:12px
12px 0 0;text-align:center">
            <h1
style="color:#fff;margin:0;font-size:22px">${bankName}</h1>
          </div>
          <div style="background:#fff;padding:32px;border:1px solid
#e2e8f0;border-top:none;border-radius:0 0 12px 12px">
            <h2 style="color:#1E3A5F;margin-top:0">KYC Approved 

```

```

        <p style="color:#4A5568">Dear
<strong>${customerName}</strong>,</p>
        <p style="color:#4A5568">
            Your KYC verification has been put <strong
style="color:#D97706">on hold</strong>.
        </p>
        ${reason ? `
        <div style="background:#FEF3C7;border:1px solid
#F59E0B;border-radius:8px;padding:16px;margin:16px 0">
            <strong style="color:#92400E">Reason:</strong>
            <p style="color:#92400E;margin:8px 0 0">${reason}</p>
        </div>` : ''}
        <p style="color:#4A5568">Our team will contact you soon.
Please ensure your documents are valid and try resubmitting.</p>
        <hr style="border:none;border-top:1px solid
#e2e8f0;margin:24px 0"/>
        <p style="color:#718096;font-size:13px">This is an automated
message from ${bankName}.</p>
    </div>
</div>`,
    }),

    rejected: (customerName, bankName, reason) => ({
        subject: `Your KYC was rejected - ${bankName}`,
        html: `
            <div style="font-family:Arial,sans-serif;max-width:520px;margin:0
auto;padding:24px">
                <div style="background:#1E3A5F;padding:24px;border-radius:12px
12px 0 0;text-align:center">
                    <h1
style="color:#fff;margin:0;font-size:22px">${bankName}</h1>
                </div>
                <div style="background:#fff;padding:32px;border:1px solid
#e2e8f0;border-top:none;border-radius:0 0 12px 12px">
                    <h2 style="color:#1E3A5F;margin-top:0">KYC Rejected ❌</h2>
                    <p style="color:#4A5568">Dear
<strong>${customerName}</strong>,</p>
                    <p style="color:#4A5568">
                        Unfortunately, your KYC verification has been <strong
style="color:#DC2626">rejected</strong>.
                    </p>
                    ${reason ? `

```

```

    <div style="background:#FEF2F2;border:1px solid
#FCA5A5;border-radius:8px;padding:16px;margin:16px 0">
      <strong style="color:#991B1B">Reason:</strong>
      <p style="color:#991B1B;margin:8px 0 0">${reason}</p>
    </div>` : ''}
    <p style="color:#4A5568">
      You may resubmit your KYC with the correct documents by
logging into the app.
    </p>
    <hr style="border:none;border-top:1px solid
#e2e8f0;margin:24px 0"/>
    <p style="color:#718096;font-size:13px">This is an automated
message from ${bankName}.</p>
  </div>
</div>`,
  }),
};

// — Send email

```

```

const sendKYCEmail = async ({ to, customerName, bankName, action,
reason }) => {
  const template = templates[action]?.(customerName, bankName, reason);
  if (!template) throw new Error(`Unknown email action: ${action}`);

  const info = await transporter.sendMail({
    from:    process.env.MAIL_FROM,
    to,
    subject: template.subject,
    html:    template.html,
  });

  console.log(`[MAIL] Sent "${action}" email to ${to} - Message ID:
${info.messageId}`);
  return info;
};

module.exports = { sendKYCEmail };

```

src/components/layout/layout.jsx

```
import { useState } from 'react';
import { useNavigate, useLocation } from 'react-router-dom';
import {
  Box, AppBar, Toolbar, Typography, Button, IconButton,
  Badge, Menu, MenuItem, Divider,
} from '@mui/material';
import { BankLogo } from '../ui';
import NotificationsIcon from '@mui/icons-material/Notifications';
import MenuIcon from '@mui/icons-material/Menu';
import LogoutIcon from '@mui/icons-material/Logout';
import { useAuth } from '../../../context/AuthContext';
import { useNotifications } from '../../../context/NotificationContext';
import { BANK_NAME } from '../../../config/constants';

const staffNav = [
  { to: '/dashboard', label: 'Dashboard' },
  { to: '/send-money', label: 'Send Money' },
  { to: '/transactions', label: 'Transactions' },
  { to: '/accounts', label: 'Accounts' },
  { to: '/notifications', label: 'Notifications', badge: true },
];

const customerNav = [
  { to: '/dashboard', label: 'Dashboard' },
  { to: '/kyc', label: 'KYC' },
  { to: '/portfolio', label: 'Portfolio' },
  { to: '/send-money', label: 'Send Money' },
  { to: '/loans', label: 'Loans' },
  { to: '/transactions', label: 'Transfers' },
  { to: '/notifications', label: 'Alerts', badge: true },
];

const NAV_H = 58;

export const Layout = ({ children }) => {
  const navigate = useNavigate();
  const location = useLocation();
  const { isManager, isStaff, isCustomer, logout, staffData,
customerData } = useAuth();
  const { unreadCount } = useNotifications();
  const [menuAnchor, setMenuAnchor] = useState(null);
```

```

const baseNav = isCustomer ? customerNav : staffNav;
const navItems = [
  ...baseNav,
  ...(isStaff && isManager ? [{ to: '/admin', label: 'Admin' }] :
[]),
];

const userName = staffData?.name ?? customerData?.fullName ?? 'User';

const isActive = (to) =>
  location.pathname === to || location.pathname.startsWith(to + '/');

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', minHeight:
'100vh' }}>
    <AppBar position="sticky" color="inherit" elevation={0}>
      <Toolbar
        sx={{
          px: { xs: 2, md: 3 },
          minHeight: `${NAV_H}px !important`,
          height: NAV_H,
          flexWrap: 'nowrap',
          gap: 0,
        }}
      >
        { /* Logo */ }
        <Box
          onClick={() => navigate('/dashboard')}
          sx={{
            display: 'flex', alignItems: 'center', gap: 1.25,
            cursor: 'pointer', flexShrink: 0, mr: 2,
          }}
        >
          <BankLogo size={28} />
          <Typography fontWeight={700} fontSize={14}
color="text.primary" noWrap>
            {BANK_NAME}
          </Typography>
        </Box>

        { /* Nav links - desktop */ }
        <Box sx={{
          display: { xs: 'none', md: 'flex' },
          alignItems: 'center',

```

```

        flex: 1,
        minWidth: 0,
        height: NAV_H,
        overflow: 'hidden',
    }}>
    {navItems.map(({ to, label, badge }) => {
        const active = isActive(to);
        return (
            <Button
                key={to}
                onClick={() => navigate(to)}
                disableElevation
                sx={{
                    height: NAV_H,
                    px: 1.75,
                    flexShrink: 0,
                    borderRadius: 0,
                    fontSize: 13,
                    fontWeight: active ? 600 : 400,
                    whiteSpace: 'nowrap',
                    color: active ? 'secondary.main' :
'text.secondary',
                    textTransform: 'none',
                    bgcolor: 'transparent',
                    borderBottom: '2px solid',
                    borderBottomColor: active ? 'secondary.main' :
'transparent',
                    position: 'relative',
                    '&:hover': {
                        bgcolor: 'grey.50',
                        color: 'secondary.main',
                        borderBottomColor: active ? 'secondary.main' :
'divider',
                    },
                }}
            >
                {label}
                {badge && unreadCount > 0 && (
                    <Box sx={{
                        position: 'absolute', top: 11, right: 6,
                        width: 6, height: 6,
                        bgcolor: '#E22026', borderRadius: '50%',
                    }} />
                )}
            </Button>
        )
    })}

```



```

    ) }
    </Button>
  );
  }) }
</Box>

  { /* Right - desktop */ }
  <Box sx={{ display: { xs: 'none', md: 'flex' }, alignItems:
'center', gap: 0.5, ml: 'auto', flexShrink: 0 }}>
    <IconButton size="small" onClick={() =>
navigate('/notifications')} sx={{ color: 'text.secondary' }}>
      <Badge badgeContent={unreadCount} color="error" max={9}>
        <NotificationsIcon sx={{ fontSize: 19 }} />
      </Badge>
    </IconButton>
    <Divider orientation="vertical" flexItem sx={{ mx: 1, my:
1.5 }} />
    <Typography fontSize={13} fontWeight={500}
color="text.primary" noWrap sx={{ maxWidth: 140 }}>
      {userName}
    </Typography>
    <IconButton size="small" onClick={logout} sx={{ color:
'text.secondary', ml: 0.5 }} title="Sign out">
      <LogoutIcon sx={{ fontSize: 17 }} />
    </IconButton>
  </Box>

  { /* Right - mobile */ }
  <Box sx={{ display: { xs: 'flex', md: 'none' }, alignItems:
'center', gap: 0.5, ml: 'auto' }}>
    <IconButton size="small" onClick={() =>
navigate('/notifications')} sx={{ color: 'text.secondary' }}>
      <Badge badgeContent={unreadCount} color="error" max={9}>
        <NotificationsIcon sx={{ fontSize: 19 }} />
      </Badge>
    </IconButton>
    <IconButton size="small" onClick={e =>
setMenuAnchor(e.currentTarget)} sx={{ color: 'text.secondary' }}>
      <MenuIcon />
    </IconButton>
  </Box>
</Toolbar>
</AppBar>

```

```

    {/* Mobile dropdown */}
    <Menu
      anchorEl={menuAnchor}
      open={Boolean(menuAnchor)}
      onClose={() => setMenuAnchor(null)}
      PaperProps={{ sx: { minWidth: 200, mt: 0.5 } }}
      transformOrigin={{ horizontal: 'right', vertical: 'top' }}
      anchorOrigin={{ horizontal: 'right', vertical: 'bottom' }}
    >
      {navItems.map(({ to, label, badge }) => (
        <MenuItem
          key={to}
          selected={isActive(to)}
          onClick={() => { navigate(to); setMenuAnchor(null); }}
        >
          {label}
          {badge && unreadCount > 0 && (
            <Box sx={{ ml: 'auto', pl: 2, display: 'flex',
alignItems: 'center' }}>
              <Box sx={{ width: 6, height: 6, bgcolor: '#E22026',
borderRadius: '50%' }} />
            </Box>
          )}
        </MenuItem>
      ))}
      <Divider />
      <MenuItem disabled sx={{ fontSize: 12 }}>{userName}</MenuItem>
      <MenuItem onClick={logout} sx={{ color: 'error.main' }}>Sign
out</MenuItem>
    </Menu>

    {/* Content */}
    <Box component="main" sx={{ flex: 1, overflowY: 'auto', p: { xs:
2, md: 3 } }}>
      <Box sx={{ maxWidth: 1200, mx: 'auto' }}>
        {children}
      </Box>
    </Box>
  </Box>
);
};

```

src/components/shared/ProtectedRoute.jsx

```
// src/components/shared/ProtectedRoute.jsx

import { Navigate, useLocation } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import { LoadingSpinner } from '../../ui';

export const ProtectedRoute = ({ children }) => {
  const { user, loading } = useAuth();
  const location = useLocation();
  if (loading) return <LoadingSpinner message="Authenticating..." />;
  if (!user) return <Navigate to="/login" state={{ from: location }}
replace />;
  return children;
};

export const StaffRoute = ({ children }) => {
  const { isStaff, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  if (!isStaff) return <Navigate to="/dashboard" replace />;
  return children;
};

export const ManagerRoute = ({ children }) => {
  const { isManager, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  if (!isManager) return <Navigate to="/dashboard" replace />;
  return children;
};

export const AdminRoute = ({ children }) => {
  const { isAdmin, loading } = useAuth();
  if (loading) return <LoadingSpinner />;
  if (!isAdmin) return <Navigate to="/dashboard" replace />;
  return children;
};
```

src/components/ui/index.jsx

```
// src/components/ui/index.jsx - HDFC-style MUI components

export const BankLogo = ({ size = 32, color = '#E22026' }) => (
```

```

    <svg width={size} height={size} viewBox="0 0 32 32" fill="none"
xmlns="http://www.w3.org/2000/svg">
    <text
      x="16" y="23"
      textAnchor="middle"
      fontFamily="Arial, Helvetica, sans-serif"
      fontWeight="700"
      fontSize="20"
      letterSpacing="-1"
      fill={color}
    >AB</text>
  </svg>
);

import {
  Card as MuiCard, CardContent, CardHeader,
  CircularProgress, Box, Typography, Chip,
  Button as MuiButton, TextField, MenuItem,
} from '@mui/material';
import ArrowDropUpIcon from '@mui/icons-material/ArrowDropUp';
import ArrowDropDownIcon from '@mui/icons-material/ArrowDropDown';

// Stat card — clean flat white, used across dashboards
export const StatCard = ({ label, value, sub, icon: Icon, trend,
trendLabel, color }) => (
  <MuiCard>
    <CardContent>
      <Box sx={{ display: 'flex', alignItems: 'flex-start',
justifyContent: 'space-between', mb: 1.5 }}>
        <Typography
          variant="caption"
          sx={{ textTransform: 'uppercase', letterSpacing: '0.06em',
fontWeight: 600, color: 'text.secondary', fontSize: 11 }}
        >
          {label}
        </Typography>
        {Icon && (
          <Icon sx={{ fontSize: 18, color: color ?? 'primary.main',
opacity: 0.8 }} />
        )}
      </Box>
      <Typography variant="h5" fontWeight={700} color="text.primary"
sx={{ fontSize: '1.375rem', letterSpacing: '-0.01em' }}>

```

```

        {value}
      </Typography>
      {(sub || trendLabel) && (
        <Box sx={{ display: 'flex', alignItems: 'center', mt: 0.75,
gap: 0.25 }}>
          {trend !== undefined && (
            trend >= 0
              ? <ArrowDropUpIcon sx={{ fontSize: 16, color:
'success.main', mt: '-1px' }} />
              : <ArrowDropDownIcon sx={{ fontSize: 16, color:
'error.main', mt: '-1px' }} />
            )}
          <Typography variant="caption" color="text.secondary">
            {trendLabel ?? sub}
          </Typography>
        </Box>
      )}
    </CardContent>
  </MuiCard>
);

// Generic card - white, minimal border
export const Card = ({ children, title, action, sx = {}, noPadding })
=> (
  <MuiCard sx={sx}>
    {(title || action) && (
      <CardHeader
        title={title}
        action={action}
      />
    )}
    <CardContent sx={noPadding ? { p: 0, '&:last-child': { pb: 0 } } :
{}}>
      {children}
    </CardContent>
  </MuiCard>
);

// Page section heading with optional right element
export const SectionHeader = ({ title, subtitle, action }) => (
  <Box sx={{ display: 'flex', alignItems: 'flex-start', justifyContent:
'space-between', mb: 2, flexWrap: 'wrap', gap: 1 }}>
    <Box>

```

```

        <Typography variant="h6" fontWeight={700}
color="text.primary">{title}</Typography>
        {subtitle && <Typography variant="body2" color="text.secondary"
mt={0.25}>{subtitle}</Typography>}
    </Box>
    {action && <Box>{action}</Box>}
  </Box>
);

export const LoadingSpinner = ({ message = 'Loading...' }) => (
  <Box sx={{ display: 'flex', flexDirection: 'column', alignItems:
'center', justifyContent: 'center', minHeight: 240, gap: 2 }}>
    <CircularProgress size={28} color="primary" />
    <Typography variant="body2"
color="text.secondary">{message}</Typography>
  </Box>
);

export const EmptyState = ({ message = 'No records found.', icon: Icon
}) => (
  <Box sx={{ display: 'flex', flexDirection: 'column', alignItems:
'center', py: 5, gap: 1 }}>
    {Icon && <Icon sx={{ fontSize: 32, color: 'text.disabled' }} />}
    <Typography variant="body2"
color="text.secondary">{message}</Typography>
  </Box>
);

const STATUS_MAP = {
  completed: { color: 'success', label: 'Completed' },
  pending: { color: 'warning', label: 'Pending' },
  failed: { color: 'error', label: 'Failed' },
  active: { color: 'success', label: 'Active' },
  inactive: { color: 'default', label: 'Inactive' },
  frozen: { color: 'info', label: 'Frozen' },
  approved: { color: 'success', label: 'Approved' },
  hold: { color: 'warning', label: 'On Hold' },
  rejected: { color: 'error', label: 'Rejected' },
  rolled_back: { color: 'default', label: 'Rolled Back' },
  cancelled: { color: 'default', label: 'Cancelled' },
};

export const StatusChip = ({ status }) => {

```

```

    const cfg = STATUS_MAP[status] ?? { color: 'default', label: status
    ?? ' ' };
    return <Chip size="small" color={cfg.color} label={cfg.label} />;
};

// Thin horizontal divider used between list items
export const RowDivider = () => (
  <Box sx={{ borderBottom: '1px solid', borderColor: 'divider', mx: 0
  }} />
);

// Key-value detail row used in cards
export const DetailRow = ({ label, value, last }) => (
  <Box>
    <Box sx={{ display: 'flex', justifyContent: 'space-between',
    alignItems: 'flex-start', py: 1.25 }}>
      <Typography variant="body2" color="text.secondary" sx={{
    minWidth: 120 }}>{label}</Typography>
      <Typography variant="body2" fontWeight={500} color="text.primary"
    sx={{ textAlign: 'right', textTransform: 'capitalize'
    }}>{value}</Typography>
    </Box>
    {!!last && <RowDivider />}
  </Box>
);

// Compatibility wrappers
export const Button = ({
  children, type = 'button', variant = 'primary',
  size = 'md', disabled = false, loading = false,
  onClick, ...rest
}) => (
  <MuiButton
    type={type}
    variant={variant === 'secondary' || variant === 'ghost' ?
    'outlined' : 'contained'}
    color={variant === 'danger' ? 'error' : 'primary'}
    size={size === 'sm' ? 'small' : size === 'lg' ? 'large' : 'medium'}
    disabled={disabled || loading}
    onClick={onClick}
    startIcon={loading ? <CircularProgress size={14} color="inherit" />
    : undefined}
    {...rest}
  />

```

```

    >
    {children}
  </MuiButton>
);

export const Input = ({ label, name, error, hint, type = 'text',
register, placeholder, ...rest }) => (
  <TextField
    label={label} type={type} placeholder={placeholder}
    error={!!error} helperText={error || hint}
    {...(register ? register(name) : {})}
    {...rest}
  />
);

export const Select = ({ label, name, error, options = [], register,
...rest }) => (
  <TextField
    select label={label} error={!!error} helperText={error}
    {...(register ? register(name) : {})}
    {...rest}
  >
    {options.map(({ value, label: l }) => (
      <MenuItem key={value} value={value}>{l}</MenuItem>
    ))}
  </TextField>
);

export const Badge = ({ children, color }) => (
  <Chip size="small" label={children}
    color={color === 'red' ? 'error' : color === 'green' ? 'success' :
'default'} />
);

```

src/config/constants.js

```

// src/config/constants.js
export const BANK_ID      = import.meta.env.VITE_BANK_ID;
export const BANK_NAME    = import.meta.env.VITE_BANK_NAME;
export const BANK_IFSC    = import.meta.env.VITE_IFSC_CODE;
export const BANK_COLOR   = import.meta.env.VITE_BANK_COLOR || 'blue';
export const HUB_BANK_EMAIL = import.meta.env.VITE_HUB_BANK_EMAIL;

```



```

export const HUB_BANK_PASSWORD =
import.meta.env.VITE_HUB_BANK_PASSWORD;

export const TRANSFER_MODES = {
  INTERNAL: 'internal',
  IMPS:     'imps',
  NEFT:     'neft',
};

export const ACCOUNT_TYPES = ['savings', 'current', 'salary', 'fd'];

export const TXN_STATUS = {
  PENDING:   'pending',
  COMPLETED: 'completed',
  FAILED:    'failed',
};

```

src/config/firebaseHub.js

```

// src/config/firebaseHub.js
// Initialises the SHARED HUB Firebase app.
// Uses VITE_HUB_* env vars - IDENTICAL across all bank deployments.
// This is the interbank message bus: only routing metadata, never
balances.

import { initializeApp, getApps, getApp } from 'firebase/app';
import { getFirestore } from 'firebase/firestore';
import { getAuth }      from 'firebase/auth';

const hubConfig = {
  apiKey:          import.meta.env.VITE_HUB_API_KEY,
  authDomain:      import.meta.env.VITE_HUB_AUTH_DOMAIN,
  projectId:       import.meta.env.VITE_HUB_PROJECT_ID,
  storageBucket:   import.meta.env.VITE_HUB_STORAGE_BUCKET,
  messagingSenderId: import.meta.env.VITE_HUB_MESSAGING_SENDER_ID,
  appId:           import.meta.env.VITE_HUB_APP_ID,
};

// Named 'hub' to avoid collision with the private app instance
const hubApp = getApps().find(a => a.name === 'hub')
  ?? initializeApp(hubConfig, 'hub');

export const hubDb = getFirestore(hubApp);

```

```
export const hubAuth = getAuth(hubApp);  
export { hubApp };
```

src/config/firebasePrivate.js

```
// src/config/firebasePrivate.js  
// Initialises the PRIVATE Firebase app for this bank.  
// Uses VITE_PRIVATE_* env vars – unique per bank deployment.  
// No other bank's app can access this project.  
  
import { initializeApp, getApps, getApp } from 'firebase/app';  
import { getFirestore } from 'firebase/firestore';  
import { getAuth } from 'firebase/auth';  
  
const privateConfig = {  
  apiKey: import.meta.env.VITE_PRIVATE_API_KEY,  
  authDomain: import.meta.env.VITE_PRIVATE_AUTH_DOMAIN,  
  projectId: import.meta.env.VITE_PRIVATE_PROJECT_ID,  
  storageBucket: import.meta.env.VITE_PRIVATE_STORAGE_BUCKET,  
  messagingSenderId: import.meta.env.VITE_PRIVATE_MESSAGING_SENDER_ID,  
  appId: import.meta.env.VITE_PRIVATE_APP_ID,  
};  
  
// Named 'private' to avoid collision with the hub app instance  
const privateApp = getApps().find(a => a.name === 'private')  
  ?? initializeApp(privateConfig, 'private');  
  
export const privateDb = getFirestore(privateApp);  
export const privateAuth = getAuth(privateApp);  
export { privateApp };
```

src/context/AuthContext.jsx

```
// src/context/AuthContext.jsx  
  
import { createContext, useContext, useEffect, useState, useCallback }  
from 'react';  
  
import {  
  signInWithEmailAndPassword,  
  createUserWithEmailAndPassword,  
  signOut,  
  onAuthStateChanged,  
} from 'firebase/auth';
```

```

import { doc, getDoc, setDoc, serverTimestamp } from
'firebase/firestore';
import { privateAuth, privateDb } from '../config/firebasePrivate';
import { hubAuth } from '../config/firebaseHub';
import { HUB_BANK_EMAIL, HUB_BANK_PASSWORD, BANK_ID } from
'../config/constants';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [userType, setUserType] = useState(null);
  const [staffData, setStaffData] = useState(null);
  const [customerData, setCustomerData] = useState(null);
  const [loading, setLoading] = useState(true);

  const signInToHub = async () => {
    try {
      await signInWithEmailAndPassword(hubAuth, HUB_BANK_EMAIL,
HUB_BANK_PASSWORD);
    } catch (err) {
      console.warn('[HUB AUTH] Hub sign-in failed:', err.message);
    }
  };

  const loadProfile = async (firebaseUser) => {
    const uid = firebaseUser.uid;

    const staffSnap = await getDoc(doc(privateDb, 'staff', uid));
    if (staffSnap.exists()) {
      const staff = staffSnap.data();
      if (!staff.isActive) { await signOut(privateAuth); throw new
Error('Your account has been deactivated.')}
      setStaffData({ uid, ...staff });
      setCustomerData(null);
      setUserType('staff');
      setUser(firebaseUser);
      return;
    }

    const customerSnap = await getDoc(doc(privateDb, 'customers',
uid));
    if (customerSnap.exists()) {

```

```

        setCustomerData({ uid, ...customerSnap.data() });
        setStaffData(null);
        setUserType('customer');
        setUser(firebaseUser);
        return;
    }

    await signOut(privateAuth);
    throw new Error('No account record found.');
```

};

// Refresh customer data (called after KYC submission to update kycStatus)

```

const refreshCustomer = useCallback(async () => {
    if (!user || userType !== 'customer') return;
    const snap = await getDoc(doc(privateDb, 'customers', user.uid));
    if (snap.exists()) setCustomerData({ uid: user.uid, ...snap.data()
});
}, [user, userType]);

const signIn = async (email, password) => {
    const credential = await signInWithEmailAndPassword(privateAuth,
email, password);
    await loadProfile(credential.user);
    await signInToHub();
    return credential.user;
};

const signUp = async ({ fullName, email, password, phone }) => {
    const credential = await
createUserWithEmailAndPassword(privateAuth, email, password);
    const uid = credential.user.uid;

    const customerDoc = {
        uid, fullName, email, phone,
        bankId:    BANK_ID,
        status:    'active',
        kycStatus: 'not_submitted', // KYC starts as not submitted
        createdAt: serverTimestamp(),
        updatedAt: serverTimestamp(),
    };
    await setDoc(doc(privateDb, 'customers', uid), customerDoc);

```

```

    setCustomerData({ uid, ...customerDoc });
    setStaffData(null);
    setUserType('customer');
    setUser(credential.user);
    await signInToHub();
    return credential.user;
  };

  const logout = async () => {
    await Promise.all([signOut(privateAuth), signOut(hubAuth).catch(() => {})]);
    setUser(null);
    setUserType(null);
    setStaffData(null);
    setCustomerData(null);
  };

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(privateAuth, async (firebaseUser) => {
      if (firebaseUser) {
        try {
          await loadProfile(firebaseUser);
          await signInToHub();
        } catch (err) {
          console.error('Profile load failed:', err.message);
          setUser(null); setUserType(null); setStaffData(null); setCustomerData(null);
        }
      } else {
        setUser(null); setUserType(null); setStaffData(null); setCustomerData(null);
      }
      setLoading(false);
    });
    return unsubscribe;
  }, []);

  const value = {
    user, userType, staffData, customerData,
    isStaff: userType === 'staff',
    isCustomer: userType === 'customer',
    role: staffData?.role ?? null,
  };

```

```

    isAdmin:    staffData?.role === 'admin',
    isManager:  ['manager', 'admin'].includes(staffData?.role),
    isTeller:   ['teller', 'manager',
'admin'].includes(staffData?.role),
    displayName: staffData?.name ?? customerData?.fullName ?? 'User',
    kycStatus:  customerData?.kycStatus ?? 'not_submitted',
    loading,
    signIn, signUp, logout, refreshCustomer,
  };

  return <AuthContext.Provider
value={value}>{children}</AuthContext.Provider>;
};

export const useAuth = () => {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used within
AuthProvider');
  return ctx;
};

```

src/context/NotificationContext.jsx

```

// src/context/NotificationContext.jsx
// Writes notifications to Firestore – NO toast popups.
// Notifications appear in the /notifications page only.

import { createContext, useContext, useEffect, useState, useRef } from
'react';
import {
  collection, query, where, onSnapshot,
  updateDoc, doc, addDoc, serverTimestamp,
} from 'firebase/firestore';
import { privateDb } from '../config/firebasePrivate';
import { startIncomingListener, startStatusListener } from
'../services/hubService';
import { paiseToRupees } from '../utils/formatters';
import { useAuth } from './AuthContext';

const NotificationContext = createContext(null);

```

```

// Write to Firestore notifications collection - shows up in
Notifications page
const pushNotification = async (type, message, relatedTxnId = null) =>
{
  try {
    await addDoc(collection(privateDb, 'notifications'), {
      type, message, relatedTxnId,
      isRead: false,
      createdAt: serverTimestamp(),
    });
  } catch (err) {
    console.warn('[NOTIF] Failed to write notification:', err.message);
  }
};

export const NotificationProvider = ({ children }) => {
  const { user } = useAuth();
  const [notifications, setNotifications] = useState([]);
  const [unreadCount, setUnreadCount] = useState(0);
  const unsubRefs = useRef([]);

  const clearListeners = () => {
    unsubRefs.current.forEach(fn => fn?.());
    unsubRefs.current = [];
  };

  useEffect(() => {
    if (!user) {
      clearListeners();
      setNotifications([]);
      setUnreadCount(0);
      return;
    }

    // 1. Listen to unread notifications for badge count
    const notifQ = query(
      collection(privateDb, 'notifications'),
      where('isRead', '==', false),
    );

    const unsubNotif = onSnapshot(notifQ, (snap) => {
      const docs = snap.docs
        .map(d => ({ id: d.id, ...d.data() }));

```

```

        .sort((a, b) => (b.createdAt?.toMillis?.() ?? 0) -
(a.createdAt?.toMillis?.() ?? 0));
        setNotifications(docs);
        setUnreadCount(docs.length);
    });
    unsubRefs.current.push(unsubNotif);

    // 2. Incoming transfer listener - write to notifications, NO toast
    const unsubIncoming = startIncomingListener(
        (transfer) => {
            pushNotification(
                'incoming_transfer',
                `You received ${paiseToRupees(transfer.amount)} from
${transfer.fromBankId.toUpperCase()} via
${transfer.mode.toUpperCase()}`,
                transfer.transferId,
            );
        },
        (transfer, err) => {
            if (transfer) {
                pushNotification(
                    'transfer_failed',
                    `Incoming transfer of ${paiseToRupees(transfer.amount)}
failed: ${err.message}`,
                    transfer.transferId,
                );
            }
        },
    );
    unsubRefs.current.push(unsubIncoming);

    // 3. Outgoing status listener - write to notifications, NO toast
    const unsubStatus = startStatusListener((transfer) => {
        if (transfer.status === 'completed') {
            pushNotification(
                'transfer_completed',
                `Your transfer of ${paiseToRupees(transfer.amount)} was
delivered successfully`,
                transfer.transferId ?? transfer.id,
            );
        } else if (transfer.status === 'failed') {
            pushNotification(
                'transfer_failed',

```



```

        `Your transfer of ${paiseToRupees(transfer.amount)} failed:
        ${transfer.failureReason ?? 'Unknown reason'}`,
        transfer.transferId ?? transfer.id,
    );
    }
  });
  unsubRefs.current.push(unsubStatus);

  return clearListeners;
}, [user]);

const markAllRead = async () => {
  await Promise.all(
    notifications.map(n => updateDoc(doc(privateDb, 'notifications',
n.id), { isRead: true }))
  );
};

return (
  <NotificationContext.Provider value={{ notifications, unreadCount,
markAllRead }}>
    {children}
  </NotificationContext.Provider>
);
};

export const useNotifications = () => {
  const ctx = useContext(NotificationContext);
  if (!ctx) throw new Error('useNotifications must be used within
NotificationProvider');
  return ctx;
};

```

src/pages/accounts/AccountDetails.jsx

```

// src/pages/accounts/AccountDetails.jsx

import { useEffect, useState } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import {
  Box, Typography, Button, Card, CardContent,
  List, ListItem, ListItemText, Divider, Grid, Alert,

```

```

    TextField, Collapse,
  } from '@mui/material';
import ArrowBackIcon      from '@mui/icons-material/ArrowBack';
import SwapHorizIcon      from '@mui/icons-material/SwapHoriz';
import ArrowDownwardIcon  from '@mui/icons-material/ArrowDownward';
import ArrowUpwardIcon    from '@mui/icons-material/ArrowUpward';
import AcUnitIcon         from '@mui/icons-material/AcUnit';
import LockOpenIcon       from '@mui/icons-material/LockOpen';
import AddCircleIcon       from '@mui/icons-material/AddCircle';
import toast from 'react-hot-toast';
import { getAccount, updateAccountStatus, depositToAccount } from
'../../services/accountService';
import { getTransactionsByAccount } from
'../../services/transactionService';
import { paiseToRupees, formatDate, rupeesToPaise } from
'../../utils/formatters';
import { LoadingSpinner, StatusChip } from '../../components/ui';
import { useAuth } from '../../context/AuthContext';

export const AccountDetails = () => {
  const { id } = useParams();
  const navigate = useNavigate();
  const { isManager, isStaff, staffData } = useAuth();
  const [account, setAccount] = useState(null);
  const [transactions, setTransactions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [togglng, setTogglng] = useState(false);
  const [showDeposit, setShowDeposit] = useState(false);
  const [depositAmt, setDepositAmt] = useState('');
  const [depositNote, setDepositNote] = useState('');
  const [depositing, setDepositing] = useState(false);

  useEffect(() => {
    (async () => {
      try {
        const [acc, txns] = await Promise.all([getAccount(id),
getTransactionsByAccount(id)]);
        setAccount(acc);
        setTransactions(txns);
      } catch (err) { toast.error(err.message); }
      finally { setLoading(false); }
    })();
  }, [id]);

```

```

const toggleStatus = async () => {
  if (!account) return;
  const next = account.status === 'active' ? 'frozen' : 'active';
  setToggling(true);
  try {
    await updateAccountStatus(id, next);
    setAccount(a => ({ ...a, status: next }));
    toast.success(`Account ${next === 'active' ? 'unfrozen' :
'frozen'}.`);
  } catch (err) { toast.error(err.message); }
  finally { setToggling(false); }
};

const handleDeposit = async () => {
  const paise = rupeesToPaise(depositAmt);
  if (!paise || paise <= 0) { toast.error('Enter a valid amount.');
```

```
return; }
  setDepositing(true);
  try {
    await depositToAccount(id, paise, staffData?.uid ?? 'staff',
depositNote);
    toast.success(`₹${depositAmt} deposited successfully.`);
    // Refresh account balance and transactions
    const [acc, txns] = await Promise.all([getAccount(id),
getTransactionsByAccount(id)]);
    setAccount(acc); setTransactions(txns);
    setDepositAmt(''); setDepositNote(''); setShowDeposit(false);
  } catch (err) { toast.error(err.message); }
  finally { setDepositing(false); }
};

if (loading) return <LoadingSpinner message="Loading account..." />;
if (!account) return <Alert severity="error">Account not
found.</Alert>;

return (
  <Box sx={{ maxWidth: 720, display: 'flex', flexDirection: 'column',
gap: 2.5 }}>
    {/* Header */}
    <Box sx={{ display: 'flex', alignItems: 'center', gap: 1 }}>
      <Button startIcon={<ArrowBackIcon />} onClick={() =>
navigate('/accounts')} size="small" variant="text" color="secondary">
```

```

        Back
      </Button>
      <Typography variant="h5" sx={{ ml: 0.5 }}>Account
Details</Typography>
    </Box>

    { /* Account hero card */ }
    <Card sx={{ bgcolor: 'secondary.main', color: '#fff', border:
'none' }}>
      <CardContent>
        <Grid container justifyContent="space-between"
alignItems="flex-start">
          <Grid item>
            <Typography variant="caption" sx={{ opacity: 0.65,
fontSize: 11, textTransform: 'uppercase', letterSpacing: '0.06em' }}>
              Account Number
            </Typography>
            <Typography variant="h5" fontFamily="monospace"
fontWeight={700} mt={0.5} letterSpacing="0.04em">
              {account.accountNumber}
            </Typography>
            <Typography variant="caption" sx={{ opacity: 0.65
}}>{account.ifscCode}</Typography>
          </Grid>
          <Grid item sx={{ textAlign: 'right' }}>
            <Typography variant="caption" sx={{ opacity: 0.65,
fontSize: 11, textTransform: 'uppercase', letterSpacing: '0.06em' }}>
              Balance
            </Typography>
            <Typography variant="h4" fontWeight={700}
mt={0.5}>{paiseToRupees(account.balance)}</Typography>
            <StatusChip status={account.status} />
          </Grid>
        </Grid>
      <Divider sx={{ borderColor: 'rgba(255,255,255,0.15)', my: 2
}} />

      <Grid container spacing={3}>
        {[['Type', account.accountType], ['Customer',
account.customerName], ['Opened',
formatDate(account.createdAt)]]}.map(([l, v]) => (
          <Grid item xs={4} key={l}>

```

```

        <Typography variant="caption" sx={{ opacity: 0.6,
fontSize: 11, textTransform: 'uppercase', letterSpacing: '0.04em',
display: 'block' }}>
            {1}
        </Typography>
        <Typography variant="body2" fontWeight={500} sx={{
textTransform: 'capitalize', mt: 0.5, color: '#fff' }}>
            {v}
        </Typography>
    </Grid>
    )})
</Grid>
</CardContent>
</Card>

{/* Action buttons */}
<Box sx={{ display: 'flex', gap: 1.5, flexWrap: 'wrap' }}>
    <Button
        variant="contained"
        color="primary"
        startIcon={<SwapHorizIcon />}
        onClick={() => navigate('/send-money')}
        size="small"
    >
        Send Money
    </Button>

    {isStaff && (
        <Button
            variant="outlined"
            color="secondary"
            startIcon={<AddCircleIcon />}
            onClick={() => setShowDeposit(v => !v)}
            size="small"
        >
            {showDeposit ? 'Cancel Deposit' : 'Deposit Cash'}
        </Button>
    )}

    {isManager && (
        <Button
            variant="outlined"
            color={account.status === 'active' ? 'error' : 'success'}

```

```

        startIcon={account.status === 'active' ? <AcUnitIcon /> :
<LockOpenIcon />}
        disabled={toggling}
        onClick={toggleStatus}
        size="small"
    >
        {toggling ? 'Updating...' : account.status === 'active' ?
'Freeze Account' : 'Unfreeze Account'}
    </Button>
  )}
</Box>

{/* Deposit panel */}
<Collapse in={showDeposit}>
  <Card>
    <Box sx={{ px: 2.5, py: 1.75, borderBottom: '1px solid',
borderColor: 'divider' }}>
      <Typography variant="subtitle2">Cash Deposit</Typography>
    </Box>
    <CardContent>
      <Box sx={{ display: 'flex', flexDirection: 'column', gap:
2, maxWidth: 400 }}>
        <TextField
          label="Amount (₹)"
          type="number"
          value={depositAmt}
          onChange={e => setDepositAmt(e.target.value)}
          inputProps={{ min: 1 }}
          size="small"
        />
        <TextField
          label="Remarks (optional)"
          value={depositNote}
          onChange={e => setDepositNote(e.target.value)}
          size="small"
          placeholder="e.g. Cash deposit at branch"
        />
        <Box sx={{ display: 'flex', gap: 1 }}>
          <Button
            variant="contained"
            color="primary"
            size="small"
            disabled={depositing || !depositAmt}

```

```

        onClick={handleDeposit}
      >
        {depositing ? 'Processing...' : 'Confirm Deposit'}
      </Button>
      <Button
        variant="outlined"
        size="small"
        onClick={() => { setShowDeposit(false);
setDepositAmt(''); setDepositNote(''); }}
      >
        Cancel
      </Button>
    </Box>
  </Box>
</CardContent>
</Card>
</Collapse>

{/* Transactions */}
<Card>
  <Box sx={{ px: 2.5, py: 1.75, borderBottom: '1px solid',
borderColor: 'divider' }}>
    <Typography variant="subtitle2">Transactions
({transactions.length})</Typography>
  </Box>
  <CardContent sx={{ p: 0, '&:last-child': { pb: 0 } }}>
    {transactions.length === 0 ? (
      <Typography variant="body2" color="text.secondary"
textAlign="center" py={4}>
        No transactions on this account.
      </Typography>
    ) : (
      <List dense disablePadding>
        {transactions.map((txn, i) => (
          <Box key={txn.id}>
            <ListItem disablePadding sx={{ px: 2.5, py: 1.25 }}>
              <Box sx={{
                width: 30, height: 30, borderRadius: 1,
                bgcolor: txn.direction === 'credit' ?
'success.light' : 'error.light',
                display: 'flex', alignItems: 'center',
justifyContent: 'center',
                mr: 1.5, flexShrink: 0,

```

```

    }}>
    {txn.direction === 'credit'
      ? <ArrowDownwardIcon sx={{ color:
'success.main', fontSize: 15 }} />
      : <ArrowUpwardIcon sx={{ color: 'error.main',
fontSize: 15 }} />}
  </Box>
  <ListItemText
    primary={
      txn.type === 'deposit' ? `Cash
Deposit${txn.remarks ? ` - ${txn.remarks}` : ''}`
      : txn.type === 'intrabank_transfer' ? 'Internal
Transfer'
      : `${txn.mode?.toUpperCase()} · ${txn.direction
=== 'credit' ? txn.fromBankId : txn.toBankId}`
    }
    secondary={formatDate(txn.createdAt)}
    primaryTypographyProps={{ fontSize: 13,
fontWeight: 500, color: 'text.primary' }}
    secondaryTypographyProps={{ fontSize: 11 }}
  />
  <Box sx={{ textAlign: 'right', ml: 1, flexShrink: 0
}}>
    <Typography variant="body2" fontWeight={700}
sx={{ color: txn.direction === 'credit' ? 'success.main' :
'error.main', fontSize: 13 }}>
      {txn.direction === 'credit' ? '+' :
'-'}{paiseToRupees(txn.amount)}
    </Typography>
    <StatusChip status={txn.status} />
  </Box>
</ListItem>
{i < transactions.length - 1 && <Divider />}
</Box>
  )}
</List>
)}
</CardContent>
</Card>
</Box>
);
};

```


src/pages/accounts/Accounts.jsx

```
// src/pages/accounts/Accounts.jsx - MUI

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  Box, Typography, Button, Table, TableBody, TableCell,
  TableContainer, TableHead, TableRow, Paper,
} from '@mui/material';
import CreditCardIcon from '@mui/icons-material/CreditCard';
import AddIcon from '@mui/icons-material/Add';
import { getAllAccounts } from '../../services/accountService';
import { paiseToRupees, formatDate } from '../../utils/formatters';
import { LoadingSpinner, StatusChip } from '../../components/ui';

export const Accounts = () => {
  const navigate = useNavigate();
  const [accounts, setAccounts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => { getAllAccounts().then(setAccounts).finally(() =>
    setLoading(false)); }, []);

  if (loading) return <LoadingSpinner message="Loading accounts..." />;

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 3 }}>
      <Box sx={{ display: 'flex', alignItems: 'center', justifyContent:
'space-between' }}>
        <Box>
          <Typography variant="h5" sx={{ display: 'flex', alignItems:
'center', gap: 1 }}>
            <CreditCardIcon /> Accounts
          </Typography>
          <Typography variant="body2" color="text.secondary"
mt={0.5}>{accounts.length} accounts</Typography>
        </Box>
        <Button variant="contained" startIcon={<AddIcon />} onClick={()
=> navigate('/accounts/new')}>
          New account
        </Button>
      </Box>
    </Box>
  )
}
```

```

    <TableContainer component={Paper} variant="outlined" sx={{
borderRadius: 2 }}>
      <Table size="small">
        <TableHead sx={{ bgcolor: 'grey.50' }}>
          <TableRow>
            {[ 'Account
no.', 'Customer', 'Type', 'Balance', 'IFSC', 'Status', 'Created', '' ].map(h =>
(
              <TableCell key={h}>{h}</TableCell>
            )) }
          </TableRow>
        </TableHead>
        <TableBody>
          {accounts.length === 0 ? (
            <TableRow><TableCell colspan={8} sx={{
textAlign: 'center', py: 4, color: 'text.secondary' }}>No accounts
yet.</TableCell></TableRow>
          ) : accounts.map(acc => (
            <TableRow key={acc.id} hover sx={{ cursor: 'pointer' }}
onClick={() => navigate(`/accounts/${acc.id}`)}>
              <TableCell><Typography variant="caption"
fontFamily="monospace">....{acc.accountNumber?.slice(-4)}</Typography><
/TableCell>
              <TableCell><Typography
variant="body2">{acc.customerName}</Typography></TableCell>
              <TableCell><Typography variant="caption" sx={{
textTransform: 'capitalize'
}}>{acc.accountType}</Typography></TableCell>
              <TableCell><Typography variant="body2"
fontWeight={600}>{paiseToRupees(acc.balance)}</Typography></TableCell>
              <TableCell><Typography variant="caption"
fontFamily="monospace">{acc.ifscCode}</Typography></TableCell>
              <TableCell><StatusChip status={acc.status}
/></TableCell>
              <TableCell><Typography
variant="caption">{formatDate(acc.createdAt)}</Typography></TableCell>
              <TableCell><Button size="small" onClick={e => {
e.stopPropagation(); navigate(`/accounts/${acc.id}`);
}}>View</Button></TableCell>
            </TableRow>
          )) }
        </TableBody>

```

```

        </Table>
      </TableContainer>
    </Box>
  );
};

```

src/pages/accounts/CreateAccount.jsx

```

// src/pages/accounts/CreateAccount.jsx - MUI

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import {
  Box, Typography, TextField, Button, MenuItem,
  Card, CardContent, List, ListItemButton,
  ListItemText, InputAdornment, Chip, Alert,
} from '@mui/material';
import PersonAddIcon from '@mui/icons-material/PersonAdd';
import SearchIcon from '@mui/icons-material/Search';
import CheckIcon from '@mui/icons-material/Check';
import toast from 'react-hot-toast';
import { getAllCustomers, createAccount } from
'../../services/accountService';
import { registerPublicAccount } from '../../services/hubService';
import { rupeesToPaise, paiseToRupees } from '../../utils/formatters';
import { ACCOUNT_TYPES } from '../../config/constants';
import { LoadingSpinner } from '../../components/ui';

const schema = z.object({
  accountType: z.enum(['savings', 'current', 'salary', 'fd']),
  initialDeposit: z.string().refine(v => !isNaN(parseFloat(v)) &&
parseFloat(v) >= 0, 'Invalid amount'),
});

export const CreateAccount = () => {
  const navigate = useNavigate();
  const [customers, setCustomers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [search, setSearch] = useState('');
  const [selectedCustomer, setSelectedCustomer] = useState(null);

```

```

const [showList, setShowList] = useState(false);

const { control, handleSubmit, formState: { errors, isSubmitting } }
= useForm({
  resolver: zodResolver(schema),
  defaultValues: { accountType: 'savings', initialDeposit: '1000' },
});

useEffect(() => {
  getAllCustomers()
    .then(c => setCustomers(c.filter(x => x.kycStatus ===
'approved')))
    .finally(() => setLoading(false));
}, []);

const filtered = customers.filter(c => {
  const q = search.toLowerCase();
  return c.fullName?.toLowerCase().includes(q) ||
c.email?.toLowerCase().includes(q) || c.phone?.includes(q);
});

const onSubmit = async (data) => {
  if (!selectedCustomer) { toast.error('Select a customer.');
```

```

        <Typography variant="h5" sx={{ display: 'flex', alignItems:
'center', gap: 1 }}>
          <PersonAddIcon /> Open New Account
        </Typography>
        <Typography variant="body2" color="text.secondary" mt={0.5}>
          Only KYC-approved customers are shown.
        </Typography>
      </Box>

      {/* Step 1 – Select customer */}
      <Card sx={{ overflow: 'visible' }}>
        <CardContent sx={{ overflow: 'visible' }}>
          <Typography variant="subtitle1" fontWeight={600} mb={2}>Step
1 – Select customer</Typography>

          {customers.length === 0 ? (
            <Alert severity="info">No KYC-approved customers yet.
Approve KYC from Admin Dashboard.</Alert>
          ) : (
            <Box sx={{ position: 'relative' }}>
              <TextField
                fullWidth
                size="small"
                placeholder="Search by name, email or phone..."
                value={search}
                onChange={e => { setSearch(e.target.value);
setShowList(true); }}
                onFocus={() => setShowList(true)}
                InputProps={{ startAdornment: <InputAdornment
position="start"><SearchIcon fontSize="small" /></InputAdornment> }}
              />

              {/* Selected badge */}
              {selectedCustomer && (
                <Box sx={{ mt: 1.5, p: 1.5, bgcolor: 'primary.light',
borderRadius: 2, display: 'flex', justifyContent: 'space-between',
alignItems: 'center' }}>
                  <Box>
                    <Typography variant="body2" fontWeight={600}
color="primary.dark">{selectedCustomer.fullName}</Typography>
                    <Typography variant="caption"
color="text.secondary">{selectedCustomer.email}</Typography>
                  </Box>

```

```

        <CheckIcon color="primary" />
      </Box>
    )}

    {/* Dropdown */}
    {showList && filtered.length > 0 && (
      <Card sx={{ position: 'absolute', zIndex: 10, width:
'100%', mt: 0.5, maxHeight: 240, overflowY: 'auto' }}>
        <List dense disablePadding>
          {filtered.map(c => (
            <ListItemButton key={c.id}
selected={selectedCustomer?.id === c.id}
onClick={() => { setSelectedCustomer(c);
setSearch(c.fullName); setShowList(false); }}>
              <ListItemText
                primary={c.fullName}
                secondary={` ${c.email} · ${c.phone}`}
                primaryTypographyProps={{ fontSize: 14 }}
                secondaryTypographyProps={{ fontSize: 12 }}
              />
              <Chip size="small" label={c.kycStatus}
color="success" sx={{ ml: 1 }} />
            </ListItemButton>
          ))}
        </List>
      </Card>
    )}

    {showList && <Box sx={{ position: 'fixed', inset: 0,
zIndex: 9 }} onClick={() => setShowList(false)} />}
  </Box>
)}
</CardContent>
</Card>

{/* Step 2 – Account details */}
<Card>
  <CardContent>
    <Typography variant="subtitle1" fontWeight={600} mb={2}>Step
2 – Account details</Typography>
    <Box component="form" onSubmit={handleSubmit(onSubmit)} sx={{
display: 'flex', flexDirection: 'column', gap: 2 }}>
      <Controller name="accountType" control={control} render={({
field }) => (

```

```

        <TextField {...field} select label="Account type"
error={!!errors.accountType} helperText={errors.accountType?.message}>
            {ACCOUNT_TYPES.map(t => <MenuItem key={t} value={t}
sx={{ textTransform: 'capitalize' }}>{t}</MenuItem>)}
        </TextField>
    )} />

    <Controller name="initialDeposit" control={control}
render={({ field }) => (
        <TextField {...field} type="number" label="Initial
deposit (₹)"
            error={!!errors.initialDeposit}
helperText={errors.initialDeposit?.message ?? 'Minimum ₹1,000'}
            InputProps={{ startAdornment: <InputAdornment
position="start">₹</InputAdornment> }} />
        )} />

    {selectedCustomer && (
        <Alert severity="info" sx={{ py: 0.5 }}>
            Opening for:
            <strong>{selectedCustomer.fullName}</strong> ({selectedCustomer.email})
        </Alert>
    )}

    <Button type="submit" variant="contained" size="large"
disabled={!selectedCustomer || isSubmitting}>
        {isSubmitting ? 'Opening...' : 'Open Account'}
    </Button>
</Box>
</CardContent>
</Card>
</Box>
);
};

```

src/pages/admin/AdminDashboard.jsx

```
// src/pages/admin/AdminDashboard.jsx
```

```

import { useEffect, useState } from 'react';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { createUserWithEmailAndPassword } from 'firebase/auth';

```

```

import { doc, setDoc, updateDoc, serverTimestamp, getDocs, collection,
query, where } from 'firebase/firestore';
import {
  Box, Typography, Grid, Card, CardContent, TextField, MenuItem,
  Button, Alert, Chip, Divider, List, ListItem, ListItemText,
  Avatar, Collapse, Tab, Tabs, CircularProgress,
} from '@mui/material';
import SettingsIcon    from '@mui/icons-material/Settings';
import CheckCircleIcon  from '@mui/icons-material/CheckCircle';
import CancelIcon       from '@mui/icons-material/Cancel';
import PauseIcon        from '@mui/icons-material/Pause';
import ExpandMoreIcon   from '@mui/icons-material/ExpandMore';
import ExpandLessIcon   from '@mui/icons-material/ExpandLess';
import AttachMoneyIcon  from '@mui/icons-material/AttachMoney';
import SyncIcon          from '@mui/icons-material/Sync';
import AccountBalanceWalletIcon from
 '@mui/icons-material/AccountBalanceWallet';
import toast from 'react-hot-toast';
import { privateAuth, privateDb } from '../../config/firebasePrivate';
import { addStaffSchema } from '../../utils/validators';
import { BANK_ID } from '../../config/constants';
import { StatCard, LoadingSpinner } from '../../components/ui';
import { getAllAccounts, getAllCustomers } from
 '../../services/accountService';
import { getAllTransactions } from '../../services/transactionService';
import { getPendingKYC, getHoldKYC, getApprovedKYC, approveKYC,
holdKYC, rejectKYC } from '../../services/kycService';
import { getPendingLoans, getHoldLoans, approveLoan, holdLoan,
rejectLoan } from '../../services/loanService';
import { registerBankInHub } from '../../services/hubService';
import { paiseToRupees, rupeesToPaise, formatDate } from
 '../../utils/formatters';
import { useAuth } from '../../context/AuthContext';

// Section wrapper matching HDFC card style
const Section = ({ title, children }) => (
  <Card>
    <Box sx={{ px: 2.5, py: 1.75, borderBottom: '1px solid',
borderColor: 'divider' }}>
      <Typography variant="subtitle1"
fontWeight={600}>{title}</Typography>
    </Box>
    <CardContent>{children}</CardContent>
  </Card>

```



```

    </Card>
  );

  // Expandable item row
  const ItemRow = ({ primary, secondary, badge, expanded, onToggle,
children }) => (
    <Box sx={{ border: '1px solid', borderColor: 'divider', borderRadius:
1, mb: 1, overflow: 'hidden' }}>
      <Box
        sx={{
          display: 'flex', alignItems: 'center', justifyContent:
'space-between',
          px: 2, py: 1.5, cursor: 'pointer', bgcolor: '#fff',
          '&:hover': { bgcolor: 'grey.50' },
        }}
        onClick={onToggle}
      >
        <Box>
          <Typography variant="body2"
fontWeight={500}>{primary}</Typography>
          <Typography variant="caption"
color="text.secondary">{secondary}</Typography>
        </Box>
        <Box sx={{ display: 'flex', alignItems: 'center', gap: 1 }}>
          {badge}
          {expanded ? <ExpandLessIcon fontSize="small" sx={{ color:
'text.secondary' }} /> : <ExpandMoreIcon fontSize="small" sx={{ color:
'text.secondary' }} />}
        </Box>
      </Box>
      <Collapse in={expanded}>
        <Divider />
        <Box sx={{ px: 2, py: 2, bgcolor: 'grey.50' }}>{children}</Box>
      </Collapse>
    </Box>
  );

  // Reason input + confirm / cancel buttons
  const ReasonForm = ({ label, confirmColor, onConfirm, onCancel }) => {
    const [reason, setReason] = useState('');
    return (
      <Box sx={{ display: 'flex', flexDirection: 'column', gap: 1, mt: 1
}}>

```

```

        <TextField
          size="small" fullWidth
          placeholder={label}
          value={reason}
          onChange={e => setReason(e.target.value)}
        />
        <Box sx={{ display: 'flex', gap: 1 }}>
          <Button size="small" variant="contained" color={confirmColor}
            onClick={() => onConfirm(reason)}>
            Confirm
          </Button>
          <Button size="small" variant="outlined"
            onClick={onCancel}>Cancel</Button>
        </Box>
      </Box>
    );
  };

export const AdminDashboard = () => {
  const { staffData } = useAuth();
  const [staff, setStaff] = useState([]);
  const [kycData, setKycData] = useState({ pending: [], hold: [], approved: [] });
  const [loanData, setLoanData] = useState({ pending: [], hold: [], approved: [] });
  const [customers, setCustomers] = useState([]);
  const [stats, setStats] = useState({ accounts: 0, txns: 0 });
  const [loading, setLoading] = useState(true);
  const [kycTab, setKycTab] = useState(0);
  const [loanTab, setLoanTab] = useState(0);
  const [expandedKyc, setExpandedKyc] = useState(null);
  const [expandedLoan, setExpandedLoan] = useState(null);
  const [kycAction, setKycAction] = useState(null); // { id, type: 'hold'|'reject' }
  const [loanAction, setLoanAction] = useState(null); // { id, type: 'hold'|'reject' }
  const [limitTarget, setLimitTarget] = useState(null);
  const [limitValue, setLimitValue] = useState('');

  const { control, handleSubmit, reset, formState: { errors, isSubmitting } } = useForm({
    resolver: zodResolver(addStaffSchema),
  });

```

```

    defaultValues: { name: '', email: '', password: '', role: 'teller'
  },
});

const load = async () => {
  const [staffSnap, accs, txns, kPending, kHold, kApproved, lPending,
lHold, custs] = await Promise.all([
    getDocs(query(collection(privateDb, 'staff'), where('bankId',
'==', BANK_ID))),
    getAllAccounts(), getAllTransactions(),
    getPendingKYC(), getHoldKYC(), getApprovedKYC(),
    getPendingLoans(), getHoldLoans(), getAllCustomers(),
  ]);
  setStaff(staffSnap.docs.map(d => ({ id: d.id, ...d.data() })));
  setStats({ accounts: accs.length, txns: txns.length });
  setKycData({ pending: kPending, hold: kHold, approved: kApproved
});
  setLoanData({ pending: lPending, hold: lHold });
  setCustomers(custs);
};

useEffect(() => { load().finally(() => setLoading(false)); }, []);

const getEmail = (cid) => customers.find(c => c.id === cid)?.email ??
'';

// KYC handlers
const handleKycApprove = async (kyc) => {
  try {
    await approveKYC(kyc.customerId, staffData.uid,
getEmail(kyc.customerId), kyc.fullName);
    toast.success(`KYC approved for ${kyc.fullName}.`);
    await load();
  } catch (err) { toast.error(err.message); }
};

const handleKycHold = async (kyc, reason) => {
  if (!reason?.trim()) { toast.error('Enter a reason.');
```

```

};

const handleKycReject = async (kyc, reason) => {
  if (!reason?.trim()) { toast.error('Enter a reason.');
```

 return; }
 try {
 await rejectKYC(kyc.customerId, staffData.uid, reason,
getEmail(kyc.customerId), kyc.fullName);
 toast.success(`KYC rejected for \${kyc.fullName}.`);
 setKycAction(null); await load();
 } catch (err) { toast.error(err.message); }
};

// Loan handlers
const handleLoanApprove = async (loan) => {
 try {
 await approveLoan(loan.id, staffData.uid);
 toast.success('Loan approved.');
 await load();
 } catch (err) { toast.error(err.message); }
};

const handleLoanHold = async (loan, reason) => {
 if (!reason?.trim()) { toast.error('Enter a reason.');
 return; }
 try {
 await holdLoan(loan.id, staffData.uid, reason);
 toast.success('Loan put on hold.');
 setLoanAction(null); await load();
 } catch (err) { toast.error(err.message); }
};

const handleLoanReject = async (loan, reason) => {
 if (!reason?.trim()) { toast.error('Enter a reason.');
 return; }
 try {
 await rejectLoan(loan.id, staffData.uid, reason);
 toast.success('Loan rejected.');
 setLoanAction(null); await load();
 } catch (err) { toast.error(err.message); }
};

const setLimit = async (cid) => {
 const p = rupeesToPaise(limitValue);
 if (!p || p <= 0) { toast.error('Enter a valid amount.');
 return; }
 try {
 await updateDoc(doc(privateDb, 'customers', cid), {
transferLimitPaise: p, updatedAt: serverTimestamp() });
 toast.success(`Limit set to \${paiseToRupees(p)}`);
 }
};

```

        setLimitTarget(null); setLimitValue('');
        setCustomers(prev => prev.map(c => c.id === cid ? { ...c,
transferLimitPaise: p } : c));
    } catch (err) { toast.error(err.message); }
  };

  const addStaff = async (data) => {
    try {
      const cred = await createUserWithEmailAndPassword(privateAuth,
data.email, data.password);
      await setDoc(doc(privateDb, 'staff', cred.user.uid), {
        uid: cred.user.uid, name: data.name, email: data.email, role:
data.role,
        bankId: BANK_ID, isActive: true, createdAt: serverTimestamp(),
      });
      toast.success(`${data.name} added as ${data.role}.`);
      setStaff(p => [...p, { id: cred.user.uid, ...data, isActive: true
}]);
      reset();
    } catch (err) { toast.error(err.message); }
  };

  if (loading) return <LoadingSpinner />;

  const KYC_TABS = ['pending', 'hold', 'approved'];
  const LOAN_TABS = ['pending', 'hold'];
  const currentKYC = kycData[KYC_TABS[kycTab]] ?? [];
  const currentLoan = loanData[LOAN_TABS[loanTab]] ?? [];

  const statusColor = { pending: 'warning', hold: 'warning', approved:
'success', rejected: 'error' };

  return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2.5 }}>
      <Typography variant="h5">Admin Dashboard</Typography>

      { /* Stats */ }
      <Grid container spacing={2}>
        <Grid item xs={6} md={3}><StatCard label="Accounts"
value={stats.accounts} icon={SettingsIcon} /></Grid>
        <Grid item xs={6} md={3}><StatCard label="Transactions"
value={stats.txns} icon={SettingsIcon} /></Grid>

```

```

        <Grid item xs={6} md={3}><StatCard label="Pending KYC"
value={kycData.pending.length} icon={SettingsIcon} /></Grid>
        <Grid item xs={6} md={3}><StatCard label="Pending Loans"
value={loanData.pending.length} icon={SettingsIcon} /></Grid>
    </Grid>

    {/* KYC Management */}
    <Section title="KYC Management">
        <Tabs value={kycTab} onChange={(_, v) => setKycTab(v)} sx={{
mb: 2, mt: -1 }}>
            {KYC_TABS.map(t => (
                <Tab key={t} label={`${t.charAt(0).toUpperCase() +
t.slice(1)} (${kycData[t]?.length ?? 0})`} />
            ))}
        </Tabs>

        {currentKYC.length === 0 ? (
            <Typography variant="body2" color="text.secondary"
textAlign="center" py={2}>
                No {KYC_TABS[kycTab]} KYC requests.
            </Typography>
        ) : currentKYC.map(kyc => (
            <ItemRow
                key={kyc.id}
                primary={kyc.fullName}
                secondary={`Submitted: ${formatDate(kyc.submittedAt)} `}
                badge={<Chip size="small" color={statusColor[kyc.status]} ??
'default'>{kyc.status}</Chip> sx={{ textTransform: 'capitalize' }} />
                expanded={expandedKyc === kyc.id}
                onToggle={() => setExpandedKyc(expandedKyc === kyc.id ?
null : kyc.id)}
            >
                <Grid container spacing={1} mb={1.5}>
                    {[['PAN', kyc.pan], ['Aadhaar',
`.....${kyc.aadhaarLast4}`], ['DOB', kyc.dob], ['Doc type',
kyc.docType?.replace('_', ' ')], ['Address', kyc.address]].map(([l, v])
=> (
                        <Grid item xs={6} key={l}>
                            <Typography variant="caption" color="text.secondary"
display="block">{l}</Typography>
                            <Typography variant="body2" fontWeight={500} sx={{
textTransform: 'capitalize' }}>{v}</Typography>
                        </Grid>

```

```

    ))}
    {kyc.holdReason && (
      <Grid item xs={12}>
        <Alert severity="warning" sx={{ py: 0.5 }}>Hold
reason: {kyc.holdReason}</Alert>
      </Grid>
    )}
  </Grid>

  {kycAction?.id === kyc.id ? (
    <ReasonForm
      label={kycAction.type === 'hold' ? 'Reason for hold...' :
'Reason for rejection...'}
      confirmColor={kycAction.type === 'hold' ? 'warning' :
'error'}
      onConfirm={reason => kycAction.type === 'hold' ?
handleKycHold(kyc, reason) : handleKycReject(kyc, reason)}
      onCancel={() => setKycAction(null)}
    />
  ) : kycTab !== 2 && (
    <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
      <Button size="small" variant="contained"
color="success" startIcon={<CheckCircleIcon />} onClick={() =>
handleKycApprove(kyc)}>Approve</Button>
      <Button size="small" variant="outlined"
color="warning" startIcon={<PauseIcon />} onClick={() =>
setKycAction({ id: kyc.id, type: 'hold' })}>Hold</Button>
      <Button size="small" variant="outlined" color="error"
startIcon={<CancelIcon />} onClick={() => setKycAction({ id:
kyc.id, type: 'reject' })}>Reject</Button>
    </Box>
  )}
  {kycTab === 2 && (
    <Button size="small" variant="outlined" color="secondary"
href="/accounts/new">Open account for customer</Button>
  )}
</ItemRow>
))}
</Section>

{/* Loan Management */}
<Section title="Loan Management">

```

```

        <Tabs value={loanTab} onChange={({_, v}) => setLoanTab(v)} sx={{
mb: 2, mt: -1 }}>
        {LOAN_TABS.map(t => (
            <Tab key={t} label={` ${t.charAt(0).toUpperCase() +
t.slice(1)} (${loanData[t]?.length ?? 0)} `} />
        ))}
    </Tabs>

    {currentLoan.length === 0 ? (
        <Typography variant="body2" color="text.secondary"
textAlign="center" py={2}>
            No {LOAN_TABS[loanTab]} loans.
        </Typography>
    ) : currentLoan.map(loan => (
        <ItemRow
            key={loan.id}
            primary={` ${loan.loanType?.replace('_', ' ')} -
${paiseToRupees(loan.amountPaise)} `}
            secondary={` ${loan.durationMonths} months · EMI
${paiseToRupees(loan.emiPaise)} · Applied:
${formatDate(loan.appliedAt)} `}
            badge={<Chip size="small" color={statusColor[loan.status]
?? 'default'} label={loan.status} sx={{ textTransform: 'capitalize' }}
/>}
            expanded={expandedLoan === loan.id}
            onToggle={() => setExpandedLoan(expandedLoan === loan.id ?
null : loan.id)}
        >
            <Box sx={{ mb: 1.5 }}>
                <Typography variant="body2"><strong>Purpose:</strong>
{loan.purpose}</Typography>
                <Typography variant="body2"><strong>Interest:</strong>
{loan.interestRate}% p.a.</Typography>
                {loan.holdReason && (
                    <Alert severity="warning" sx={{ py: 0.5, mt: 1 }}>Hold
reason: {loan.holdReason}</Alert>
                )}
            </Box>

            {loanAction?.id === loan.id ? (
                <ReasonForm
                    label={loanAction.type === 'hold' ? 'Reason for hold...'
: 'Reason for rejection...'}

```



```

        confirmColor={loanAction.type === 'hold' ? 'warning' :
'error'}

        onConfirm={reason => loanAction.type === 'hold' ?
handleLoanHold(loan, reason) : handleLoanReject(loan, reason)}
        onCancel={() => setLoanAction(null)}
      />
    ) : (
      <Box sx={{ display: 'flex', gap: 1, flexWrap: 'wrap' }}>
        <Button size="small" variant="contained"
color="success" startIcon={<CheckCircleIcon />} onClick={() =>
handleLoanApprove(loan)}>Approve</Button>
        <Button size="small" variant="outlined"
color="warning" startIcon={<PauseIcon />}          onClick={() =>
setLoanAction({ id: loan.id, type: 'hold' })}>Hold</Button>
        <Button size="small" variant="outlined" color="error"
startIcon={<CancelIcon />}          onClick={() => setLoanAction({ id:
loan.id, type: 'reject' })}>Reject</Button>
      </Box>
    )}
  </ItemRow>
))}
</Section>

{/* Transfer Limits */}
<Section title="Customer Transfer Limits">
  {customers.length === 0 ? (
    <Typography variant="body2" color="text.secondary">No
customers yet.</Typography>
  ) : (
    <List dense disablePadding>
      {customers.map((c, i) => (
        <Box key={c.id}>
          <ListItem disablePadding sx={{ py: 1 }}>
            <Avatar sx={{ width: 30, height: 30, mr: 1.5,
fontSize: 13 }}>
              {c.fullName?.charAt(0)}
            </Avatar>
            <ListItemText
              primary={c.fullName}
              secondary={`Limit: ${c.transferLimitPaise ?
paiseToRupees(c.transferLimitPaise) : 'Not set'}`}
              primaryTypographyProps={{ fontSize: 13, fontWeight:
500 }}

```

```

        secondaryTypographyProps={{ fontSize: 12 }}
      />
      {limitTarget === c.id ? (
        <Box sx={{ display: 'flex', gap: 1, alignItems:
'center' }}>
          <TextField size="small" type="number"
placeholder="₹ amount" value={limitValue}
          onChange={e => setLimitValue(e.target.value)}
          sx={{ width: 110 }} />
          <Button size="small" variant="contained"
onClick={() => setLimit(c.id)}>Set</Button>
          <Button size="small" variant="outlined"
onClick={() => setLimitTarget(null)}>X</Button>
        </Box>
      ) : (
        <Button size="small" variant="outlined"
color="secondary" startIcon={<AttachMoneyIcon />}
        onClick={() => { setLimitTarget(c.id);
setLimitValue(''); }}>
          Set limit
        </Button>
      )}
    </ListItem>
    {i < customers.length - 1 && <Divider />}
  </Box>
  )}
</List>
)}
</Section>

{/* Hub sync */}
<Section title="Hub Registration">
  <Typography variant="body2" color="text.secondary" mb={2}>
    Register or refresh this bank in the shared inter-bank hub.
  </Typography>
  <Button variant="outlined" color="secondary"
startIcon={<SyncIcon />}
  onClick={async () => {
    try { await registerBankInHub(); toast.success('Synced!');
}

    catch { toast.error('Sync failed.'); }
  }}>
    Sync bank to hub

```

```

        </Button>
    </Section>

    <Grid container spacing={2.5}>
        {/* Add staff */}
        <Grid item xs={12} md={6}>
            <Section title="Add Staff Member">
                <Box component="form" onSubmit={handleSubmit(addStaff)}
sx={{ display: 'flex', flexDirection: 'column', gap: 2 }}>
                    <Controller name="name" control={control} render={({
field }) => (<TextField {...field} label="Full name"
error={!!errors.name} helperText={errors.name?.message} />)} />
                    <Controller name="email" control={control} render={({
field }) => (<TextField {...field} label="Email" type="email"
error={!!errors.email} helperText={errors.email?.message} />)} />
                    <Controller name="password" control={control} render={({
field }) => (<TextField {...field} label="Password" type="password"
error={!!errors.password} helperText={errors.password?.message} />)} />
                    <Controller name="role" control={control} render={({
field }) => (
                        <TextField {...field} select label="Role"
error={!!errors.role} helperText={errors.role?.message}>
                            {[ 'teller', 'manager', 'admin' ].map(r => <MenuItem
key={r} value={r} sx={{ textTransform: 'capitalize' }}>{r}</MenuItem>)}
                        </TextField>
                    )} />
                    <Button type="submit" variant="contained"
disabled={isSubmitting}
startIcon={isSubmitting ? <CircularProgress size={14}
color="inherit" /> : null}>
                        {isSubmitting ? 'Adding...' : 'Add staff member'}
                    </Button>
                </Box>
            </Section>
        </Grid>

        {/* Staff list */}
        <Grid item xs={12} md={6}>
            <Section title="Current Staff">
                {staff.length === 0 ? (
                    <Typography variant="body2" color="text.secondary">No
staff yet.</Typography>
                ) : (

```

```

        <List dense disablePadding>
          {staff.map((s, i) => (
            <Box key={s.id}>
              <ListItem disablePadding sx={{ py: 1 }}>
                <Avatar sx={{ width: 30, height: 30, mr: 1.5,
fontSize: 13 }}>{s.name?.charAt(0)}</Avatar>
                <ListItemText
                  primary={s.name}
                  secondary={s.email}
                  primaryTypographyProps={{ fontSize: 13,
fontWeight: 500 }}
                  secondaryTypographyProps={{ fontSize: 12 }}
                />
                <Box sx={{ display: 'flex', gap: 0.5 }}>
                  <Chip size="small" label={s.role}
color="secondary" sx={{ textTransform: 'capitalize' }} />
                  <Chip size="small" label={s.isActive ? 'active'
: 'inactive'} color={s.isActive ? 'success' : 'default'} />
                </Box>
              </ListItem>
              {i < staff.length - 1 && <Divider />}
            </Box>
          )})}
        </List>
      )}
    </Section>
  </Grid>
</Grid>
</Box>
);
};

```

src/pages/auth/Login.jsx

```

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import {
  Box, Card, CardContent, TextField, Button,
  Typography, Divider,
} from '@mui/material';
import { BankLogo } from '../../components/ui';
import toast from 'react-hot-toast';

```

```

import { useAuth } from '../..context/AuthContext';
import { loginSchema } from '../..utils/validators';
import { BANK_NAME } from '../..config/constants';

const Skyline = () => (
  <svg viewBox="0 0 480 160" xmlns="http://www.w3.org/2000/svg"
width="100%" style={{ display: 'block', marginTop: 'auto' }}>
    <defs>
      <radialGradient id="glow" cx="50%" cy="100%" r="60%">
        <stop offset="0%" stopColor="rgba(0,85,165,0.4)" />
        <stop offset="100%" stopColor="rgba(0,85,165,0)" />
      </radialGradient>
      <linearGradient id="fade" x1="0" y1="0" x2="0" y2="1">
        <stop offset="0%" stopColor="rgba(0,21,64,0)" />
        <stop offset="100%" stopColor="rgba(0,21,64,0.7)" />
      </linearGradient>
    </defs>

    {/* Glow on horizon */}
    <ellipse cx="240" cy="155" rx="240" ry="60" fill="url(#glow)" />

    {/* — Far layer (faintest) — */}
    <g fill="rgba(255,255,255,0.10)">
      <rect x="0" y="95" width="28" height="65" />
      <rect x="22" y="75" width="16" height="85" />
      <rect x="35" y="88" width="32" height="72" />
      <rect x="62" y="68" width="14" height="92" />
      <rect x="73" y="82" width="38" height="78" />
      <rect x="107" y="60" width="22" height="100"/>
      <rect x="125" y="72" width="28" height="88" />
      <rect x="149" y="50" width="16" height="110"/>
      <rect x="162" y="78" width="36" height="82" />
      <rect x="195" y="65" width="20" height="95" />
      <rect x="212" y="55" width="26" height="105"/>
      <rect x="234" y="80" width="34" height="80" />
      <rect x="265" y="70" width="18" height="90" />
      <rect x="280" y="58" width="30" height="102"/>
      <rect x="307" y="75" width="24" height="85" />
      <rect x="328" y="62" width="28" height="98" />
      <rect x="353" y="85" width="38" height="75" />
      <rect x="388" y="72" width="22" height="88" />
      <rect x="408" y="80" width="36" height="80" />
      <rect x="440" y="65" width="20" height="95" />
    </g>
  </svg>
)

```

```

    <rect x="458" y="78" width="22" height="82" />
</g>

{/* — Mid layer — */}
<g fill="rgba(255,255,255,0.18)">
    <rect x="0" y="105" width="42" height="55" />
    <rect x="38" y="90" width="22" height="70" />
    <rect x="56" y="100" width="48" height="60" />
    <rect x="99" y="78" width="20" height="82" />
    <rect x="116" y="92" width="38" height="68" />
    <rect x="150" y="68" width="16" height="92" />
    <rect x="163" y="85" width="42" height="75" />
    <rect x="200" y="72" width="28" height="88" />
    <rect x="225" y="88" width="36" height="72" />
    <rect x="257" y="78" width="22" height="82" />
    <rect x="276" y="68" width="38" height="92" />
    <rect x="311" y="82" width="32" height="78" />
    <rect x="340" y="74" width="28" height="86" />
    <rect x="366" y="88" width="44" height="72" />
    <rect x="407" y="78" width="24" height="82" />
    <rect x="429" y="85" width="51" height="75" />
</g>

{/* — Near layer (clearest) — */}
<g fill="rgba(255,255,255,0.28)">
    <rect x="0" y="112" width="50" height="48" />
    <rect x="46" y="98" width="28" height="62" />
    <rect x="70" y="108" width="55" height="52" />
    {/* Tall tower */}
    <rect x="120" y="70" width="18" height="90" />
    <rect x="134" y="82" width="4" height="6" /> {/* antenna */}
    <rect x="136" y="78" width="2" height="4" />
    <rect x="137" y="68" width="1" height="10" />
    <rect x="148" y="102" width="42" height="58" />
    <rect x="186" y="85" width="32" height="75" />
    {/* Central tall building */}
    <rect x="214" y="62" width="22" height="98" />
    <rect x="225" y="55" width="4" height="8" />
    <rect x="226" y="45" width="2" height="10" />
    <rect x="232" y="95" width="48" height="65" />
    <rect x="276" y="88" width="26" height="72" />
    <rect x="298" y="75" width="38" height="85" />
    <rect x="332" y="90" width="34" height="70" />

```

```
<rect x="363" y="80" width="28" height="80" />
<rect x="388" y="95" width="50" height="65" />
<rect x="435" y="88" width="45" height="72" />
</g>
```

```
{/* — Lit windows — */}
```

```
<g fill="rgba(255,220,130,0.55)">
  <rect x="26" y="93" width="3" height="2" />
  <rect x="31" y="98" width="3" height="2" />
  <rect x="112" y="62" width="4" height="3" />
  <rect x="117" y="70" width="4" height="3" />
  <rect x="157" y="71" width="4" height="3" />
  <rect x="218" y="68" width="4" height="3" />
  <rect x="218" y="78" width="4" height="3" />
  <rect x="222" y="74" width="4" height="3" />
  <rect x="284" y="70" width="4" height="3" />
  <rect x="302" y="78" width="4" height="3" />
  <rect x="308" y="65" width="4" height="3" />
  <rect x="335" y="80" width="4" height="3" />
  <rect x="369" y="82" width="3" height="2" />
  <rect x="394" y="98" width="3" height="2" />
  <rect x="442" y="90" width="3" height="2" />
</g>
```

```
{/* — Stars — */}
```

```
<g fill="rgba(255,255,255,0.65)">
  <circle cx="18" cy="12" r="1.0" />
  <circle cx="55" cy="8" r="1.2" />
  <circle cx="88" cy="20" r="0.8" />
  <circle cx="130" cy="10" r="1.0" />
  <circle cx="170" cy="18" r="0.7" />
  <circle cx="205" cy="6" r="1.1" />
  <circle cx="258" cy="14" r="0.9" />
  <circle cx="310" cy="8" r="1.0" />
  <circle cx="350" cy="20" r="0.8" />
  <circle cx="398" cy="10" r="1.2" />
  <circle cx="440" cy="16" r="0.9" />
  <circle cx="470" cy="7" r="0.8" />
</g>
```

```
{/* — Crescent moon — */}
```

```
<circle cx="448" cy="28" r="14" fill="rgba(255,255,255,0.18)" />
<circle cx="441" cy="24" r="11" fill="rgba(0,30,80,0.95)" />
```

```

    { /* — Bottom ground line — */}
    <rect x="0" y="158" width="480" height="2"
fill="rgba(255,255,255,0.12)" />

    { /* Depth fade overlay */}
    <rect x="0" y="0" width="480" height="160" fill="url(#fade)" />
  </svg>
);

export const Login = () => {
  const { signIn } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const from = location.state?.from?.pathname ?? '/dashboard';

  const { register, handleSubmit, formState: { errors, isSubmitting } }
= useForm({
  resolver: zodResolver(loginSchema),
  defaultValues: { email: '', password: '' },
});

  const onSubmit = async ({ email, password }) => {
    try {
      await signIn(email, password);
      navigate(from, { replace: true });
    } catch (err) {
      toast.error(err.message ?? 'Login failed.');
```



```

    { /* Ambient glow blobs */ }
    <Box sx={{
      position: 'absolute', width: 600, height: 600, borderRadius:
'50%',
      background: 'radial-gradient(circle, rgba(0,85,165,0.25) 0%,
transparent 70%)',
      top: '-200px', right: '-100px', pointerEvents: 'none',
    }} />
    <Box sx={{
      position: 'absolute', width: 400, height: 400, borderRadius:
'50%',
      background: 'radial-gradient(circle, rgba(226,32,38,0.10) 0%,
transparent 70%)',
      bottom: '-100px', left: '-80px', pointerEvents: 'none',
    }} />

    { /* Card shell */ }
    <Box sx={{
      display: 'flex',
      width: '100%',
      maxWidth: 880,
      borderRadius: '12px',
      overflow: 'hidden',
      boxShadow: '0 24px 64px rgba(0,0,0,0.45)',
    }}>
      { /* — Left panel (desktop) — */ }
      <Box sx={{
        display: { xs: 'none', md: 'flex' },
        flexDirection: 'column',
        flex: 1,
        background: 'linear-gradient(160deg, rgba(0,30,80,0.6) 0%,
rgba(0,55,130,0.4) 100%)',
        backdropFilter: 'blur(2px)',
        borderRight: '1px solid rgba(255,255,255,0.10)',
        p: 4,
        pb: 0,
        overflow: 'hidden',
      }}>
        { /* Brand */ }
        <Box sx={{ display: 'flex', alignItems: 'center', gap: 1.5,
mb: 3 }}>
          <BankLogo size={42} color="white" />

```

```

        <Typography fontWeight={700} fontSize={18} color="#fff"
lineHeight={1.2}>
            {BANK_NAME}
        </Typography>
    </Box>

    <Typography fontSize={22} fontWeight={700} color="#fff"
lineHeight={1.35} mb={1.5}>
        Banking at your<br />fingertips.
    </Typography>
    <Typography fontSize={13} sx={{ color:
'rgba(255,255,255,0.55)', lineHeight: 1.7 }}>
        Manage accounts, send money, track transactions, and more —
all in one secure place.
    </Typography>

    {/* Feature list */}
    <Box sx={{ mt: 3, display: 'flex', flexDirection: 'column',
gap: 1 }}>
        {['Instant fund transfers', 'Real-time notifications', 'KYC
& loan management'].map(f => (
            <Box key={f} sx={{ display: 'flex', alignItems: 'center',
gap: 1 }}>
                <Box sx={{ width: 5, height: 5, borderRadius: '50%',
bgcolor: '#E22026', flexShrink: 0 }} />
                <Typography fontSize={12} sx={{ color:
'rgba(255,255,255,0.60)' }}>{f}</Typography>
            </Box>
        ))}
    </Box>

    {/* Skyline fills remaining vertical space */}
    <Box sx={{ flex: 1, display: 'flex', flexDirection: 'column',
justifyContent: 'flex-end', mt: 3 }}>
        <Skyline />
    </Box>
</Box>

{/* — Right panel — form — */}
<Box sx={{
    width: { xs: '100%', md: 380 },
    bgcolor: '#fff',
    display: 'flex',

```

```

        flexDirection: 'column',
        justifyContent: 'center',
        p: { xs: 3, md: 4 },
      }}>
      { /* Mobile logo */ }
      <Box sx={{ display: { md: 'none' }, mb: 3, textAlign:
'center' }}>
        <Box sx={{ display: 'inline-flex', alignItems: 'center',
gap: 1.25 }}>
          <BankLogo size={34} />
          <Typography fontWeight={700} fontSize={15}
color="text.primary">{BANK_NAME}</Typography>
        </Box>
      </Box>

      <Typography variant="h6" fontWeight={700}
color="text.primary" mb={0.5}>
        Sign In
      </Typography>

      <Box component="form" onSubmit={handleSubmit(onSubmit)} sx={{
display: 'flex', flexDirection: 'column', gap: 2 }}>
        <TextField
          label="Customer ID / Email"
          type="email"
          autoComplete="email"
          autoFocus
          {...register('email')}
          error={!!errors.email}
          helperText={errors.email?.message}
        />
        <TextField
          label="Password / IPIN"
          type="password"
          autoComplete="current-password"
          {...register('password')}
          error={!!errors.password}
          helperText={errors.password?.message}
        />
        <Button
          type="submit"
          variant="contained"
          color="primary"

```

```

        size="large"
        disabled={isSubmitting}
        sx={{ mt: 0.5 }}
      >
        {isSubmitting ? 'Signing in...' : 'Login'}
      </Button>
    </Box>

    <Divider sx={{ my: 2.5 }} />

    <Typography variant="body2" textAlign="center"
color="text.secondary">
      New customer?{' '}
      <Link to="/register" style={{ color: '#003E8A', fontWeight:
600 }}>
        Register now
      </Link>
    </Typography>

  </Box>
</Box>
</Box>
);
};

```

src/pages/auth/Register.jsx

```

// src/pages/auth/Register.jsx

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { Link, useNavigate } from 'react-router-dom';
import {
  Box, Card, CardContent, TextField, Button,
  Typography, Stepper, Step, StepLabel, Divider,
} from '@mui/material';
import { BankLogo } from '../../../components/ui';
import toast from 'react-hot-toast';
import { useAuth } from '../../../context/AuthContext';
import { registerSchema } from '../../../utils/validators';
import { BANK_NAME, BANK_IFSC } from '../../../config/constants';

const steps = ['Create Account', 'Submit KYC', 'Account Active'];

```

```

export const Register = () => {
  const { signUp } = useAuth();
  const navigate = useNavigate();

  const { register, handleSubmit, formState: { errors, isSubmitting } } = useForm({
    resolver: zodResolver(registerSchema),
  });

  const onSubmit = async (data) => {
    try {
      await signUp(data);
      toast.success('Account created! Please submit your KYC.');
```

```

      navigate('/kyc');
    } catch (err) {
      toast.error(err.message ?? 'Registration failed.');
```

```

    }
  };

  return (
    <Box sx={{
      minHeight: '100vh',
      bgcolor: '#F4F5F8',
      display: 'flex',
      flexDirection: 'column',
      alignItems: 'center',
      justifyContent: 'center',
      p: 2,
    }}>
      { /* Brand */ }
      <Box sx={{ mb: 3, textAlign: 'center' }}>
        <Box sx={{ display: 'inline-flex', alignItems: 'center', gap:
1.25, mb: 1 }}>
          <BankLogo size={38} />
          <Typography variant="h6" fontWeight={700}
color="#1A1A1A">{BANK_NAME}</Typography>
        </Box>
        <Typography variant="caption" color="text.secondary"
display="block">IFSC: {BANK_IFSC}</Typography>
      </Box>

      { /* Stepper */ }

```

```

<Box sx={{ width: '100%', maxWidth: 460, mb: 2.5 }}>
  <Stepper activeStep={0} alternativeLabel>
    {steps.map(label => (
      <Step key={label}>
        <StepLabel>{label}</StepLabel>
      </Step>
    ))}
  </Stepper>
</Box>

{/* Form card */}
<Card sx={{ width: '100%', maxWidth: 460 }}>
  <Box sx={{ px: 2.5, pt: 2.5, pb: 0, borderBottom: '1px solid',
borderColor: 'divider' }}>
    <Typography variant="subtitle1" fontWeight={600}
pb={1.5}>Step 1 – Create Account</Typography>
  </Box>
  <CardContent sx={{ px: 2.5, pt: 2.5, pb: '20px !important' }}>
    <Box component="form" onSubmit={handleSubmit(onSubmit)} sx={{
display: 'flex', flexDirection: 'column', gap: 2 }}>
      <TextField
        label="Full name"
        autoComplete="name"
        {...register('fullName')}
        error={!!errors.fullName}
        helperText={errors.fullName?.message}
      />
      <TextField
        label="Email address"
        type="email"
        autoComplete="email"
        {...register('email')}
        error={!!errors.email}
        helperText={errors.email?.message}
      />
      <TextField
        label="Phone number"
        autoComplete="tel"
        {...register('phone')}
        error={!!errors.phone}
        helperText={errors.phone?.message}
      />
      <TextField

```

```

        label="Password"
        type="password"
        {...register('password')}
        error={!!errors.password}
        helperText={errors.password?.message}
    />
    <TextField
        label="Confirm password"
        type="password"
        {...register('confirmPassword')}
        error={!!errors.confirmPassword}
        helperText={errors.confirmPassword?.message}
    />
    <Button
        type="submit"
        variant="contained"
        color="primary"
        size="large"
        disabled={isSubmitting}
        sx={{ mt: 0.5 }}
    >
        {isSubmitting ? 'Creating account...' : 'Continue to KYC'}
    </Button>
</Box>

<Divider sx={{ my: 2.5 }} />
<Typography variant="body2" textAlign="center"
color="text.secondary">
    Already registered?{' '}
    <Link to="/login" style={{ color: '#003E8A', fontWeight:
600 }}>Sign in</Link>
</Typography>
</CardContent>
</Card>

<Typography variant="caption" color="text.disabled" mt={3}
textAlign="center" display="block">
    &copy; {new Date().getFullYear()} {BANK_NAME}. All rights
reserved.
</Typography>
</Box>
);
};

```

src/pages/kyc/KYCUUpload.jsx

```
// src/pages/kyc/KYCUUpload.jsx - MUI, no lucide/tailwind

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { Box, Typography, Card, CardContent, TextField, MenuItem,
Button, Alert, Stepper, Step, StepLabel, CircularProgress, Grid,
Divider } from '@mui/material';
import VerifiedUserIcon from '@mui/icons-material/VerifiedUser';
import CheckCircleIcon from '@mui/icons-material/CheckCircle';
import CancelIcon from '@mui/icons-material/Cancel';
import HourglassIcon from '@mui/icons-material/HourglassEmpty';
import RefreshIcon from '@mui/icons-material/Refresh';
import toast from 'react-hot-toast';

import { useAuth } from '../../../context/AuthContext';
import { submitKYC, getKYC } from '../../../services/kycService';
import { LoadingSpinner } from '../../../components/ui';

const schema = z.object({
  fullName: z.string().min(2, 'Full name required'),
  pan: z.string().regex(/^[A-Z]{5}[0-9]{4}[A-Z]{1}$/, 'Invalid
PAN (e.g. ABCDE1234F)'),
  aadhaarLast4: z.string().regex(/^\d{4}$/, 'Enter last 4 digits of
Aadhaar'),
  dob: z.string().min(1, 'Date of birth required'),
  address: z.string().min(10, 'Enter your full address'),
  docType: z.enum(['aadhaar', 'pan', 'passport', 'driving_license']),
});

const STATUS_CFG = {
  pending: { Icon: HourglassIcon, severity: 'warning', title: 'KYC under
review', message: 'Your documents are being reviewed. We will notify
you within 1-2 business days.' },
  hold: { Icon: HourglassIcon, severity: 'warning', title: 'KYC on
hold', message: null },
  approved: { Icon: CheckCircleIcon, severity: 'success', title: 'KYC
approved!', message: 'Your identity has been verified. Your account
will be set up shortly.' },
};
```



```

    rejected: { Icon:CancelIcon,      severity:'error',    title:'KYC
rejected',      message:null },
  });

const steps = ['Submit KYC','Under review','Account active'];
const docTypes = [
  {value:'aadhaar',label:'Aadhaar Card'}, {value:'pan',label:'PAN
Card'},

{value:'passport',label:'Passport'}, {value:'driving_license',label:'Dri
ving License'},
];

export const KYCUpload = () => {
  const { user, customerData, refreshCustomer } = useAuth();
  const navigate = useNavigate();
  const [kyc, setKyc] = useState(null);
  const [loading, setLoading] = useState(true);

  const { control, handleSubmit, formState:{ errors, isSubmitting } } =
useForm({
  resolver: zodResolver(schema),
  defaultValues: { fullName:customerData?.fullName??'', pan:'',
aadhaarLast4:'', dob:'', address:'', docType:'aadhaar' },
  });

  useEffect(()=>{ if(user)
getKYC(user.uid).then(setKyc).finally(()=>setLoading(false));
}, [user]);

const onSubmit = async(data)=>{
  try{
    await submitKYC(user.uid,data);
    toast.success('KYC submitted! We will review it shortly.');
```

```

    const updated = await getKYC(user.uid);
    setKyc(updated); refreshCustomer?.();
  }catch(err){ toast.error(err.message??'Submission failed.');
```

```

  };

  if(loading) return <LoadingSpinner message="Loading KYC status..."/>;

  if(kyc){
    const cfg = STATUS_CFG[kyc.status]??STATUS_CFG.pending;
```

```

return (
  <Box sx={{ maxWidth:560, display:'flex', flexDirection:'column',
gap:3 }}>
    <Typography variant="h5" sx={{ display:'flex',
alignItems:'center', gap:1 }}>
      <VerifiedUserIcon/> KYC Verification
    </Typography>
    <Stepper activeStep={kyc.status=== 'approved'?2:1}
alternativeLabel>
      {steps.map(l=><Step
key={l}><StepLabel>{l}</StepLabel></Step>)}
    </Stepper>
    <Alert severity={cfg.severity}>
      <Typography fontWeight={600}>{cfg.title}</Typography>
      <Typography variant="body2"
mt={0.5}>{cfg.message??kyc.rejectionReason??kyc.holdReason??''}</Typogr
aphy>
    </Alert>
    <Card>
      <CardContent>
        <Typography variant="subtitle1" fontWeight={600}
mb={2}>Submitted details</Typography>
        {[['Name',kyc.fullName],['PAN',kyc.pan],['Aadhaar',`.....${kyc.aadha
arLast4}`],['DOB',kyc.dob],['Doc type',kyc.docType?.replace('_',''
')],['Address',kyc.address]].map(( [l,v], i, arr) => (
          <Box key={l}>
            <Box sx={{ display:'flex',
justifyContent:'space-between', py:1 }}>
              <Typography variant="body2"
color="text.secondary">{l}</Typography>
              <Typography variant="body2" fontWeight={500} sx={{
textTransform:'capitalize' }}>{v}</Typography>
            </Box>
            {i<arr.length-1&&<Divider/>}
          </Box>
        ))}
      </CardContent>
    </Card>
    {(kyc.status=== 'rejected' || kyc.status=== 'hold') && (
      <Button variant="outlined" startIcon={<RefreshIcon/>}
onClick={ () => setKyc(null) }>Resubmit documents</Button>
    )}
  )}

```

```

        {kyc.status==='approved'&&(
          <Button variant="contained"
onClick={ ()=>navigate('/dashboard')}>Go to dashboard</Button>
        )}
      </Box>
    );
  }

  return (
    <Box sx={{ maxWidth:560, display:'flex', flexDirection:'column',
gap:3 }}>
      <Box>
        <Typography variant="h5" sx={{ display:'flex',
alignItems:'center', gap:1 }}>
          <VerifiedUserIcon/> KYC Verification
        </Typography>
        <Typography variant="body2" color="text.secondary" mt={0.5}>
          Submit your identity documents for verification.
        </Typography>
      </Box>
      <Stepper activeStep={0} alternativeLabel>
        {steps.map(l=><Step key={l}><StepLabel>{l}</StepLabel></Step>)}
      </Stepper>
      <Alert severity="info">We only store the last 4 digits of your
Aadhaar. All information is stored securely.</Alert>
      <Card>
        <CardContent>
          <Typography variant="subtitle1" fontWeight={600}
mb={2}>Personal details</Typography>
          <Box component="form" onSubmit={handleSubmit(onSubmit)} sx={{
display:'flex', flexDirection:'column', gap:2 }}>
            <Controller name="fullName" control={control}
render={({field})=>(
              <TextField {...field} label="Full name (as per
documents)" size="small" fullWidth error={!!errors.fullName}
helperText={errors.fullName?.message}/>
            )}/>
            <Controller name="pan" control={control}
render={({field})=>(
              <TextField {...field} label="PAN number" size="small"
fullWidth placeholder="ABCDE1234F"
              inputProps={{ style:{textTransform:'uppercase'}} }}
              error={!!errors.pan} helperText={errors.pan?.message}/>
            )}/>
          </Box>
        </CardContent>
      </Card>
    </Box>
  );
}

```

```

    ) } />
    <Controller name="aadhaarLast4" control={control}
render={({field})=>(
    <TextField {...field} label="Aadhaar – last 4 digits
only" size="small" fullWidth placeholder="5678"
    inputProps={{ maxLength:4 }}
error={!!errors.aadhaarLast4}
helperText={errors.aadhaarLast4?.message??'We never store your full
Aadhaar number.'} />
    ) } />
    <Controller name="dob" control={control}
render={({field})=>(
    <TextField {...field} label="Date of birth" type="date"
size="small" fullWidth InputLabelProps={{ shrink:true }}
error={!!errors.dob} helperText={errors.dob?.message} />
    ) } />
    <Controller name="address" control={control}
render={({field})=>(
    <TextField {...field} label="Residential address"
size="small" fullWidth multiline rows={2}
    placeholder="Flat 4B, 12 Main Street, Chennai 600001"
error={!!errors.address} helperText={errors.address?.message} />
    ) } />
    <Controller name="docType" control={control}
render={({field})=>(
    <TextField {...field} select label="Primary ID document
type" size="small" fullWidth error={!!errors.docType}
helperText={errors.docType?.message}>
        {docTypes.map(d=><MenuItem key={d.value}
value={d.value}>{d.label}</MenuItem>)}
    </TextField>
    ) } />
    <Button type="submit" variant="contained" size="large"
disabled={isSubmitting}
    startIcon={isSubmitting?<CircularProgress size={18}
color="inherit"/>:<VerifiedUserIcon/>}>
        {isSubmitting?'Submitting...':'Submit KYC'}
    </Button>
</Box>
</CardContent>
</Card>
</Box>
);

```

```
};
```

src/pages/loans/Loans.jsx

```
// src/pages/loans/Loans.jsx - MUI, no lucide/tailwind

import { useEffect, useState } from 'react';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { Box, Typography, Button, Card, CardContent, TextField,
MenuItem, Grid, Alert, Divider, List, ListItem, ListItemText, Collapse,
CircularProgress } from '@mui/material';
import CreditCardIcon from '@mui/icons-material/CreditCard';
import AddIcon from '@mui/icons-material/Add';
import CloseIcon from '@mui/icons-material/Close';
import toast from 'react-hot-toast';
import { useAuth } from '../../../context/AuthContext';
import { applyLoan, getMyLoans, calculateEMI, LOAN_TYPES } from
 '../../../services/loanService';
import { rupeesToPaise, paiseToRupees, formatDate } from
 '../../../utils/formatters';
import { LoadingSpinner, StatusChip } from '../../../components/ui';

const loanSchema = z.object({
  loanType:
z.enum(['personal', 'home', 'education', 'vehicle', 'business']),
  amount:
z.string().refine(v=>parseFloat(v)>=1000, 'Minimum
₹1,000'),
  durationMonths: z.string().refine(v=>parseInt(v)>=1, 'Select
duration'),
  purpose:
z.string().min(10, 'Describe the purpose (min 10
chars)'),
});

export const Loans = () => {
  const { customerData } = useAuth();
  const [loans, setLoans] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showForm, setShowForm] = useState(false);
  const [previewEMI, setPreviewEMI] = useState(null);
```

```

    const { control, handleSubmit, watch, reset, formState: { errors,
isSubmitting } } = useForm({
    resolver: zodResolver(loanSchema),
    defaultValues: { loanType: 'personal', amount: '',
durationMonths: '12', purpose: '' },
    });

    const wType=watch('loanType'), wAmt=watch('amount'),
wDur=watch('durationMonths');

    useEffect(()=>{
        const amt=parseFloat(wAmt), dur=parseInt(wDur),
rate=LOAN_TYPES[wType]?.interestRate??12;
        setPreviewEMI(amt>=1000&&dur>0 ?
calculateEMI(rupeesToPaise(wAmt),rate,dur) : null);
    }, [wAmt,wDur,wType]);

    useEffect(()=>{
        if(!customerData?.uid) return;

getMyLoans(customerData.uid).then(setLoans).finally(()=>setLoading(false));
    }, [customerData]);

    const onSubmit = async(data)=>{
        try{
            await applyLoan(customerData.uid,{ loanType:data.loanType,
amountPaise:rupeesToPaise(data.amount),
durationMonths:parseInt(data.durationMonths), purpose:data.purpose });
            toast.success('Loan application submitted!');
            setLoans(await getMyLoans(customerData.uid));
            setShowForm(false); reset();
        }catch(err){ toast.error(err.message??'Application failed.')}
    };

    if(loading) return <LoadingSpinner message="Loading loans..."/>;

    const loanTypeOptions=Object.entries(LOAN_TYPES).map(([k,v])=>({
value:k, label:`${v.label} (${v.interestRate}% p.a.)` }));
    const durationOptions=[6,12,24,36,48,60,84,120].map(m=>({
value:String(m), label:`${m} months${m>=12?` (${m/12}
yr${m>12?'s':''})`:''}` }));

```

```

    return (
      <Box sx={{ maxWidth:600, display:'flex', flexDirection:'column',
gap:3 }}>
        <Box sx={{ display:'flex', alignItems:'center',
justifyContent:'space-between' }}>
          <Box>
            <Typography variant="h5" sx={{ display:'flex',
alignItems:'center', gap:1 }}>
              <CreditCardIcon/> Loans
            </Typography>
            <Typography variant="body2" color="text.secondary"
mt={0.5}>{loans.length} loan{loans.length!==1?'s':''}</Typography>
          </Box>
          {!showForm&&<Button variant="contained" startIcon={<AddIcon/>}
onClick={()=>setShowForm(true)}>Apply for loan</Button>}
        </Box>

        <Collapse in={showForm}>
          <Card>
            <CardContent>
              <Box sx={{ display:'flex', justifyContent:'space-between',
alignItems:'center', mb:2 }}>
                <Typography variant="subtitle1" fontWeight={600}>Loan
application</Typography>
                <Button size="small" color="inherit" onClick={()=>{
setShowForm(false); reset(); }}><CloseIcon fontSize="small"/></Button>
              </Box>
              <Box component="form" onSubmit={handleSubmit(onSubmit)}
sx={{ display:'flex', flexDirection:'column', gap:2 }}>
                <Controller name="loanType" control={control}
render={({field})=>(
                  <TextField {...field} select label="Loan type"
size="small" fullWidth error={!!errors.loanType}
helperText={errors.loanType?.message}>
                    {loanTypeOptions.map(o=><MenuItem key={o.value}
value={o.value}>{o.label}</MenuItem>)}
                  </TextField>
                </>
              </Box>
              <Grid container spacing={2}>
                <Grid item xs={6}>
                  <Controller name="amount" control={control}
render={({field})=>(

```

```

        <TextField {...field} type="number" label="Amount
(₹)" size="small" fullWidth error={!errors.amount}
helperText={errors.amount?.message}/>
    ) }/>
</Grid>
<Grid item xs={6}>
    <Controller name="durationMonths" control={control}
render={({field})=>(
        <TextField {...field} select label="Duration"
size="small" fullWidth error={!errors.durationMonths}
helperText={errors.durationMonths?.message}>
            {durationOptions.map(o=><MenuItem key={o.value}
value={o.value}><o.label></MenuItem>)}
        </TextField>
    ) }/>
</Grid>
</Grid>
<Controller name="purpose" control={control}
render={({field})=>(
        <TextField {...field} label="Purpose" size="small"
fullWidth multiline rows={2} error={!errors.purpose}
helperText={errors.purpose?.message}/>
    ) }/>
    {previewEMI&&(
        <Box sx={{ bgcolor:'primary.light', borderRadius:2,
p:2, display:'flex', justifyContent:'space-between',
alignItems:'center' }}>
            <Box>
                <Typography variant="caption"
color="text.secondary">Estimated monthly EMI</Typography>
                <Typography variant="h6" fontWeight={700}
color="primary.dark">{paiseToRupees(previewEMI)}</Typography>
            </Box>
            <Box sx={{ textAlign:'right' }}>
                <Typography variant="caption"
color="text.secondary" display="block">Rate:
{LOAN_TYPES[wType]?.interestRate}% p.a.</Typography>
                <Typography variant="caption"
color="text.secondary">Total:
{paiseToRupees(previewEMI*parseInt(wDur||1))}</Typography>
            </Box>
        </Box>
    ) }

```



```

        <Button type="submit" variant="contained" size="large"
disabled={isSubmitting}
        startIcon={isSubmitting?<CircularProgress size={18}
color="inherit"/>:null}>
        {isSubmitting?'Submitting...':'Submit application'}
    </Button>
  </Box>
</CardContent>
</Card>
</Collapse>

{loans.length===0 ? (
  <Card><CardContent><Typography variant="body2"
color="text.secondary" textAlign="center" py={4}>No loans yet. Apply
above to get started.</Typography></CardContent></Card>
) : (
  <Box sx={{ display:'flex', flexDirection:'column', gap:2 }}>
    {loans.map(loan=>(
      <Card key={loan.id}>
        <CardContent>
          <Box sx={{ display:'flex',
justifyContent:'space-between', alignItems:'flex-start', mb:2 }}>
            <Box>
              <Typography variant="subtitle1" fontWeight={600}
sx={{ textTransform:'capitalize'
}}>{LOAN_TYPES[loan.loanType]?.label??loan.loanType}</Typography>
              <Typography variant="body2"
color="text.secondary">{loan.purpose}</Typography>
              <Typography variant="caption"
color="text.disabled">Applied:
{formatDate(loan.appliedAt)}</Typography>
            </Box>
            <Box sx={{ textAlign:'right' }}>
              <Typography variant="h6"
fontWeight={700}>{paiseToRupees(loan.amountPaise)}</Typography>
              <StatusChip status={loan.status}/>
            </Box>
          </Box>
          <Grid container spacing={1}>
            {[['Duration',`${loan.durationMonths}
months`], ['Rate',`${loan.interestRate}%
p.a.`], ['EMI',paiseToRupees(loan.emiPaise)]]}.map(([l,v])=>(
              <Grid item xs={4} key={l}>

```

```

        <Box sx={{ bgcolor:'grey.50', borderRadius:1,
p:1, textAlign:'center' }}>
            <Typography variant="caption"
color="text.secondary" display="block">{l}</Typography>
            <Typography variant="body2"
fontWeight={600}>{v}</Typography>
        </Box>
    </Grid>
    )}}
</Grid>
    {loan.status==='rejected'&&loan.rejectionReason&&(
        <Alert severity="error" sx={{ mt:2 }}>Rejection
reason: {loan.rejectionReason}</Alert>
    )}
    </CardContent>
</Card>
    )}}
</Box>
    )}
</Box>
);
};

```

src/pages/notifications/notifications.jsx

```

// src/pages/loans/Loans.jsx - MUI, no lucide/tailwind

import { useEffect, useState } from 'react';
import { useForm, Controller } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { Box, Typography, Button, Card, CardContent, TextField,
MenuItem, Grid, Alert, Divider, List, ListItem, ListItemText, Collapse,
CircularProgress } from '@mui/material';
import CreditCardIcon from '@mui/icons-material/CreditCard';
import AddIcon from '@mui/icons-material/Add';
import CloseIcon from '@mui/icons-material/Close';
import toast from 'react-hot-toast';
import { useAuth } from '../../context/AuthContext';
import { applyLoan, getMyLoans, calculateEMI, LOAN_TYPES } from
'../../services/loanService';
import { rupeesToPaise, paiseToRupees, formatDate } from
'../../utils/formatters';

```

```

import { LoadingSpinner, StatusChip } from '../components/ui';

const loanSchema = z.object({
  loanType:
z.enum(['personal','home','education','vehicle','business']),
  amount:      z.string().refine(v=>parseFloat(v)>=1000, 'Minimum
₹1,000'),
  durationMonths: z.string().refine(v=>parseInt(v)>=1, 'Select
duration'),
  purpose:      z.string().min(10, 'Describe the purpose (min 10
chars)'),
});

export const Loans = () => {
  const { customerData } = useAuth();
  const [loans,      setLoans]      = useState([]);
  const [loading,    setLoading]    = useState(true);
  const [showForm,   setShowForm]   = useState(false);
  const [previewEMI, setPreviewEMI] = useState(null);

  const { control, handleSubmit, watch, reset, formState:{ errors,
isSubmitting } } = useForm({
    resolver: zodResolver(loanSchema),
    defaultValues: { loanType:'personal', amount:'',
durationMonths:'12', purpose:'' },
  });

  const wType=watch('loanType'), wAmt=watch('amount'),
wDur=watch('durationMonths');

  useEffect(()=>{
    const amt=parseFloat(wAmt), dur=parseInt(wDur),
rate=LOAN_TYPES[wType]?.interestRate??12;
    setPreviewEMI(amt>=1000&&dur>0 ?
calculateEMI(rupeesToPaise(wAmt),rate,dur) : null);
  }, [wAmt,wDur,wType]);

  useEffect(()=>{
    if(!customerData?.uid) return;

    getMyLoans(customerData.uid).then(setLoans).finally(()=>setLoading(false));
  }, [customerData]);

```

```

const onSubmit = async(data)=>{
  try{
    await applyLoan(customerData.uid,{ loanType:data.loanType,
amountPaise:rupeesToPaise(data.amount),
durationMonths:parseInt(data.durationMonths), purpose:data.purpose });
    toast.success('Loan application submitted!');
    setLoans(await getMyLoans(customerData.uid));
    setShowForm(false); reset();
  }catch(err){ toast.error(err.message??'Application failed. '); }
};

if(loading) return <LoadingSpinner message="Loading loans..."/>;

const loanTypeOptions=Object.entries(LOAN_TYPES).map(([k,v])=>({
value:k, label:`${v.label} (${v.interestRate}% p.a.)` }));
const durationOptions=[6,12,24,36,48,60,84,120].map(m=>({
value:String(m), label:`${m} months${m>=12?` (${m/12}
yr${m>12?'s':''})`:'` }));

return (
  <Box sx={{ maxWidth:600, display:'flex', flexDirection:'column',
gap:3 }}>
    <Box sx={{ display:'flex', alignItems:'center',
justifyContent:'space-between' }}>
      <Box>
        <Typography variant="h5" sx={{ display:'flex',
alignItems:'center', gap:1 }}>
          <CreditCardIcon/> Loans
        </Typography>
        <Typography variant="body2" color="text.secondary"
mt={0.5}>{loans.length} loan{loans.length!==1?'s':''}</Typography>
      </Box>
      {!showForm&&<Button variant="contained" startIcon={<AddIcon/>}
onClick={ ()=>setShowForm(true) }>Apply for loan</Button>}
    </Box>

    <Collapse in={showForm}>
      <Card>
        <CardContent>
          <Box sx={{ display:'flex', justifyContent:'space-between',
alignItems:'center', mb:2 }}>

```

```

        <Typography variant="subtitle1" fontWeight={600}>Loan
application</Typography>
        <Button size="small" color="inherit" onClick={ () =>{
setShowForm(false); reset(); }}><CloseIcon fontSize="small"/></Button>
    </Box>
    <Box component="form" onSubmit={handleSubmit(onSubmit)}
sx={{ display:'flex', flexDirection:'column', gap:2 }}>
        <Controller name="loanType" control={control}
render={({field})=> (
            <TextField {...field} select label="Loan type"
size="small" fullWidth error={!!errors.loanType}
helperText={errors.loanType?.message}>
                {loanTypeOptions.map(o=><MenuItem key={o.value}
value={o.value}>{o.label}</MenuItem>)}
            </TextField>
        ) }/>
        <Grid container spacing={2}>
            <Grid item xs={6}>
                <Controller name="amount" control={control}
render={({field})=> (
                    <TextField {...field} type="number" label="Amount
(₹)" size="small" fullWidth error={!!errors.amount}
helperText={errors.amount?.message}/>
                ) }/>
            </Grid>
            <Grid item xs={6}>
                <Controller name="durationMonths" control={control}
render={({field})=> (
                    <TextField {...field} select label="Duration"
size="small" fullWidth error={!!errors.durationMonths}
helperText={errors.durationMonths?.message}>
                        {durationOptions.map(o=><MenuItem key={o.value}
value={o.value}>{o.label}</MenuItem>)}
                    </TextField>
                ) }/>
            </Grid>
        </Grid>
        <Controller name="purpose" control={control}
render={({field})=> (
            <TextField {...field} label="Purpose" size="small"
fullWidth multiline rows={2} error={!!errors.purpose}
helperText={errors.purpose?.message}/>
        ) }/>

```

```

        {previewEMI&&(
            <Box sx={{ bgcolor:'primary.light', borderRadius:2,
p:2, display:'flex', justifyContent:'space-between',
alignItems:'center' }}>
                <Box>
                    <Typography variant="caption"
color="text.secondary">Estimated monthly EMI</Typography>
                    <Typography variant="h6" fontWeight={700}
color="primary.dark">{paiseToRupees (previewEMI) }</Typography>
                </Box>
                <Box sx={{ textAlign:'right' }}>
                    <Typography variant="caption"
color="text.secondary" display="block">Rate:
{LOAN_TYPES[wType]?.interestRate}% p.a.</Typography>
                    <Typography variant="caption"
color="text.secondary">Total:
{paiseToRupees (previewEMI*parseInt (wDur||1) )}</Typography>
                </Box>
            </Box>
        ) }
        <Button type="submit" variant="contained" size="large"
disabled={isSubmitting}
startIcon={isSubmitting?<CircularProgress size={18}
color="inherit"/>:null}>
            {isSubmitting?'Submitting...':'Submit application'}
        </Button>
    </Box>
</CardContent>
</Card>
</Collapse>

{loans.length===0 ? (
    <Card><CardContent><Typography variant="body2"
color="text.secondary" textAlign="center" py={4}>No loans yet. Apply
above to get started.</Typography></CardContent></Card>
) : (
    <Box sx={{ display:'flex', flexDirection:'column', gap:2 }}>
        {loans.map(loan=>(
            <Card key={loan.id}>
                <CardContent>
                    <Box sx={{ display:'flex',
justifyContent:'space-between', alignItems:'flex-start', mb:2 }}>
                        <Box>

```

```

        <Typography variant="subtitle1" fontWeight={600}
sx={{ textTransform:'capitalize'
}}>{LOAN_TYPES[loan.loanType]?.label??loan.loanType}</Typography>
        <Typography variant="body2"
color="text.secondary">{loan.purpose}</Typography>
        <Typography variant="caption"
color="text.disabled">Applied:
{formatDate(loan.appliedAt)}</Typography>
    </Box>
    <Box sx={{ textAlign:'right' }}>
        <Typography variant="h6"
fontWeight={700}>{paiseToRupees(loan.amountPaise)}</Typography>
        <StatusChip status={loan.status}/>
    </Box>
</Box>
    <Grid container spacing={1}>
        [[['Duration',`${loan.durationMonths}
months`],['Rate',`${loan.interestRate}%
p.a.`],['EMI',paiseToRupees(loan.emiPaise)]]].map(([l,v])=>(
        <Grid item xs={4} key={l}>
            <Box sx={{ bgcolor:'grey.50', borderRadius:1,
p:1, textAlign:'center' }}>
                <Typography variant="caption"
color="text.secondary" display="block">{l}</Typography>
                <Typography variant="body2"
fontWeight={600}>{v}</Typography>
            </Box>
        </Grid>
    ))}
</Grid>
    {loan.status==='rejected'&&loan.rejectionReason&&(
        <Alert severity="error" sx={{ mt:2 }}>Rejection
reason: {loan.rejectionReason}</Alert>
    )}
    </CardContent>
</Card>
    )}
</Box>
    )}
</Box>
);
};

```

src/pages/portfolio/Portfolio.jsx

```
// src/pages/portfolio/Portfolio.jsx - MUI, no lucide/tailwind

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Box, Typography, Grid, Card, CardContent, List, ListItem,
ListItemText, Divider, Button } from '@mui/material';
import TrendingUpIcon from '@mui/icons-material/TrendingUp';
import TrendingDownIcon from '@mui/icons-material/TrendingDown';
import CreditCardIcon from '@mui/icons-material/CreditCard';
import PieChartIcon from '@mui/icons-material/PieChart';
import { useAuth } from '../../../context/AuthContext';
import { getMyAccounts } from '../../../services/accountService';
import { getTransactionsByAccount } from
'../../../services/transactionService';
import { getMyLoans } from '../../../services/loanService';
import { paiseToRupees, formatDate } from '../../../utils/formatters';
import { StatCard, LoadingSpinner, StatusChip } from
'../../../components/ui';

export const Portfolio = () => {
  const { isCustomer, customerData } = useAuth();
  const navigate = useNavigate();
  const [accounts, setAccounts] = useState([]);
  const [transactions, setTransactions] = useState([]);
  const [loans, setLoans] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      try {
        const accs = isCustomer ? await getMyAccounts(customerData.uid)
: [];
        setAccounts(accs);
        if (accs.length > 0) {
          const map = new Map();
          (await Promise.all(accs.map(a =>
getTransactionsByAccount(a.id)))).flat().forEach(t => map.set(t.id,
t));

          setTransactions([...map.values()].sort((a,b)=>(b.createdAt?.toMillis?.()??0)-(a.createdAt?.toMillis?.()??0)));
        }
      } catch {
        //
      }
    })();
  }, [isCustomer, customerData]);
}
```



```

        if (isCustomer) setLoans(await getMyLoans(customerData.uid));
      } finally { setLoading(false); }
    })();
  }, [isCustomer, customerData]);

  if (loading) return <LoadingSpinner message="Loading portfolio..." />;

  const totalBalance = accounts.reduce((s,a)=>s+(a.balance??0),0);
  const totalCredit =
transactions.filter(t=>t.direction==='credit').reduce((s,t)=>s+t.amount
,0);
  const totalDebit =
transactions.filter(t=>t.direction==='debit').reduce((s,t)=>s+t.amount,
0);

  const activeLoans = loans.filter(l=>l.status==='approved');
  const totalDebt = activeLoans.reduce((s,l)=>s+l.amountPaise,0);
  const monthlyEMI = activeLoans.reduce((s,l)=>s+l.emiPaise,0);

  return (
    <Box sx={{ display:'flex', flexDirection:'column', gap:3 }}>
      <Box>
        <Typography variant="h5" sx={{ display:'flex',
alignItems:'center', gap:1 }}>
          <PieChartIcon /> My Portfolio
        </Typography>
        <Typography variant="body2" color="text.secondary"
mt={0.5}>Complete overview of your accounts, transfers and
loans</Typography>
      </Box>

      <Grid container spacing={2}>
        <Grid item xs={6} md={3}><StatCard label="Total balance"
value={paiseToRupees(totalBalance)} icon={TrendingUpIcon}/></Grid>
        <Grid item xs={6} md={3}><StatCard label="Total received"
value={paiseToRupees(totalCredit)} icon={TrendingUpIcon}/></Grid>
        <Grid item xs={6} md={3}><StatCard label="Total sent"
value={paiseToRupees(totalDebit)} icon={TrendingDownIcon}/></Grid>
        <Grid item xs={6} md={3}><StatCard label="Active loans"
value={activeLoans.length} icon={CreditCardIcon}/></Grid>
      </Grid>

      <Grid container spacing={3}>
        <Grid item xs={12} md={6}>

```

```

<Card>
  <CardContent>
    <Typography variant="subtitle1" fontWeight={600}
mb={2}>My accounts</Typography>
    {accounts.length===0 ? (
      <Typography variant="body2" color="text.secondary"
textAlign="center" py={3}>No accounts yet.</Typography>
    ) : accounts.map((acc,i)=>{
      const
cr=transactions.filter(t=>t.toAccountId===acc.id&&t.direction==='credit
').reduce((s,t)=>s+t.amount,0);
      const
dr=transactions.filter(t=>t.fromAccountId===acc.id&&t.direction==='debi
t').reduce((s,t)=>s+t.amount,0);
      return (
        <Box key={acc.id}>
          <Box sx={{ py:1.5 }}>
            <Box sx={{ display:'flex',
justifyContent:'space-between', mb:1 }}>
              <Box>
                <Typography variant="body2" fontWeight={600}
fontFamily="monospace">{acc.accountNumber}</Typography>
                <Typography variant="caption"
color="text.secondary" sx={{ textTransform:'capitalize'
}}>{acc.accountType} · {acc.ifscCode}</Typography>
              </Box>
              <Box sx={{ textAlign:'right' }}>
                <Typography variant="h6"
fontWeight={700}>{paiseToRupees(acc.balance)}</Typography>
                <StatusChip status={acc.status}/>
              </Box>
            </Box>
            <Grid container spacing={1}>
              <Grid item xs={6}>
                <Box sx={{ bgcolor:'success.light',
borderRadius:1, p:1, textAlign:'center' }}>
                  <Typography variant="caption"
color="text.secondary">Received</Typography>
                  <Typography variant="body2"
fontWeight={600} color="success.dark">{paiseToRupees(cr)}</Typography>
                </Box>
              </Grid>
              <Grid item xs={6}>

```

```

                <Box sx={{ bgcolor:'error.light',
borderRadius:1, p:1, textAlign:'center' }}>
                    <Typography variant="caption"
color="text.secondary">Sent</Typography>
                    <Typography variant="body2"
fontWeight={600} color="error.dark">{paiseToRupees(dr)}</Typography>
                </Box>
            </Grid>
        </Grid>
    </Box>
    {i<accounts.length-1&&<Divider/>}
</Box>
    );
    })}
</CardContent>
</Card>
</Grid>

<Grid item xs={12} md={6}>
    <Card>
        <CardContent>
            <Typography variant="subtitle1" fontWeight={600}
mb={2}>Transfer summary</Typography>
            <List dense disablePadding>
                {[
                    {label:'Total transactions',
value:transactions.length,
color:'text.primary'},
                    {label:'Internal transfers',
value:transactions.filter(t=>t.type==='intrabank_transfer').length,
color:'primary.main'},
                    {label:'Interbank transfers',
value:transactions.filter(t=>t.type==='interbank_transfer').length,
color:'secondary.main'},
                    {label:'Pending transfers',
value:transactions.filter(t=>t.status==='pending').length,
color:'warning.main'},
                    {label:'Failed transfers',
value:transactions.filter(t=>t.status==='failed').length,
color:'error.main'},
                ]}.map(({label,value,color},i,arr)=>(
                    <Box key={label}>
                        <ListItem disablePadding sx={{ py:1 }}>

```

```

        <ListItemText primary={label}
primaryTypographyProps={{variant:'body2',color:'text.secondary'}}/>
        <Typography variant="h6" fontWeight={700}
color={color}>{value}</Typography>
      </ListItem>
      {i<arr.length-1&&<Divider/>}
    </Box>
  )})
</List>
</CardContent>
</Card>
</Grid>

<Grid item xs={12}>
  <Card>
    <CardContent>
      <Typography variant="subtitle1" fontWeight={600}
mb={2}>Loan portfolio</Typography>
      {loans.length===0 ? (
        <Box sx={{ textAlign:'center', py:3 }}>
          <Typography variant="body2" color="text.secondary"
mb={1}>No loans yet.</Typography>
          <Button variant="text"
onClick={()=>navigate('/loans')}>Apply for a loan →</Button>
        </Box>
      ) : (
        <>
          {activeLoans.length>0&&(
            <Grid container spacing={2} mb={2}>
              <Grid item xs={6}>
                <Box sx={{ bgcolor:'warning.light',
borderRadius:2, p:2, textAlign:'center' }}>
                  <Typography variant="caption"
color="text.secondary">Total outstanding</Typography>
                  <Typography variant="h6" fontWeight={700}
color="warning.dark">{paiseToRupees(totalDebt)}</Typography>
                </Box>
              </Grid>
              <Grid item xs={6}>
                <Box sx={{ bgcolor:'primary.light',
borderRadius:2, p:2, textAlign:'center' }}>
                  <Typography variant="caption"
color="text.secondary">Monthly EMI</Typography>

```

```

        <Typography variant="h6" fontWeight={700}
color="primary.dark">{paiseToRupees (monthlyEMI) }</Typography>
      </Box>
    </Grid>
  </Grid>
)}
<List dense disablePadding>
  {loans.map((loan,i)=>(
    <Box key={loan.id}>
      <ListItem disablePadding sx={{ py:1 }}>
        <ListItemText
          primary={`$${loan.loanType?.replace('_', '
')}} loan`}
          secondary={`$${loan.durationMonths} months ·
$${loan.interestRate}% p.a. · EMI $${paiseToRupees(loan.emiPaise)}}`
primaryTypographyProps={{fontWeight:600,textTransform:'capitalize'}}
        />
        <Box sx={{ textAlign:'right' }}>
          <Typography variant="body2"
fontWeight={600}>{paiseToRupees(loan.amountPaise) }</Typography>
          <StatusChip status={loan.status}/>
        </Box>
      </ListItem>
      {i<loans.length-1&&<Divider/>}
    </Box>
  ) ) }
</List>
</>
)}
</CardContent>
</Card>
</Grid>
</Grid>
</Box>
);
};

```

src/pages/transactions/TransactionHistory.jsx

```
// src/pages/transactions/TransactionHistory.jsx
```

```
import { useEffect, useState, useCallback } from 'react';
```

```

import {
  Box, Typography, Table, TableBody, TableCell,
  TableContainer, TableHead, TableRow, Paper,
  ToggleButtonGroup, ToggleButton, Button,
  Dialog, DialogTitle, DialogContent, DialogContentText,
  DialogActions, Alert, Tooltip, Chip,
} from '@mui/material';
import HistoryIcon      from '@mui/icons-material/History';
import ArrowDownwardIcon from '@mui/icons-material/ArrowDownward';
import ArrowUpwardIcon  from '@mui/icons-material/ArrowUpward';
import UndoIcon          from '@mui/icons-material/Undo';
import toast from 'react-hot-toast';
import { useAuth } from '../../context/AuthContext';
import {
  getAllTransactions, getTransactionsByAccount,
  rollbackTransaction,
} from '../../services/transactionService';
import { getMyAccounts } from '../../services/accountService';
import { getAllBanks }   from '../../services/hubService';
import { paiseToRupees, formatDate, modeLabel } from
  '../../utils/formatters';
import { LoadingSpinner, StatusChip } from '../../components/ui';
import { BANK_ID, BANK_NAME } from '../../config/constants';

const FILTERS = ['all', 'credit', 'debit', 'pending', 'failed',
  'interbank'];

// A transaction is eligible for rollback if it is an interbank debit
// that is still pending or failed (money left sender but never reached
// recipient).
const isRollbackEligible = (txn) =>
  txn.type === 'interbank_transfer' &&
  txn.direction === 'debit' &&
  (txn.status === 'pending' || txn.status === 'failed');

export const TransactionHistory = () => {
  const { isCustomer, isStaff, customerData, staffData } = useAuth();
  const [transactions, setTransactions] = useState([]);
  const [bankMap,      setBankMap]      = useState({});
  const [filter,       setFilter]       = useState('all');
  const [loading,      setLoading]      = useState(true);
  // Single rollback
  const [rollbackTarget, setRollbackTarget] = useState(null);

```

```

    const [rolling, setRolling] = useState(false);
    // Bulk rollback
    const [rollbackAllOpen, setRollbackAllOpen] =
useState(false);
    const [rollingAll, setRollingAll] =
useState(false);
    const [rollbackAllProgress, setRollbackAllProgress] = useState(null);
    // {done, total}

    const loadTransactions = useCallback(async () => {
        const banks = await getAllBanks();
        const map = { [BANK_ID]: BANK_NAME };
        banks.forEach(b => { map[b.bankId] = b.bankName ?? b.bankId; });
        setBankMap(map);

        if (isCustomer) {
            const accs = await getMyAccounts(customerData.uid);
            if (accs.length > 0) {
                const seen = new Map();
                (await Promise.all(accs.map(a =>
getTransactionsByAccount(a.id))))
                    .flat()
                    .forEach(t => seen.set(t.id, t));
                setTransactions(
                    [...seen.values()].sort((a, b) =>
                        (b.createdAt?.toMillis?.() ?? 0) -
(a.createdAt?.toMillis?.() ?? 0)
                    )
                );
            }
        } else {
            setTransactions(await getAllTransactions());
        }
    }, [isCustomer, customerData]);

    useEffect(() => {
        loadTransactions().finally(() => setLoading(false));
    }, [loadTransactions]);

    const bankName = (id) => bankMap[id] ?? id ?? '-';

    const filtered = transactions.filter(t => {
        if (filter === 'all') return true;

```

```

    if (filter === 'credit')    return t.direction === 'credit';
    if (filter === 'debit')    return t.direction === 'debit';
    if (filter === 'pending')  return t.status    === 'pending';
    if (filter === 'failed')   return t.status    === 'failed';
    if (filter === 'interbank') return t.type      ===
'interbank_transfer';
    return true;
  });

  const rollbackCount = transactions.filter(isRollbackEligible).length;

  const handleRollback = async () => {
    if (!rollbackTarget) return;
    setRolling(true);
    try {
      await rollbackTransaction(rollbackTarget.id, staffData?.uid ??
'staff');
      toast.success(`₹${paiseToRupees(rollbackTarget.amount)} refunded
to sender.`);
      setRollbackTarget(null);
      await loadTransactions();
    } catch (err) {
      toast.error(err.message ?? 'Rollback failed.');
```

```

    } finally {
      setRolling(false);
    }
  };

  const handleRollbackAll = async () => {
    const eligible = transactions.filter(isRollbackEligible);
    if (eligible.length === 0) return;
    setRollingAll(true);
    setRollbackAllProgress({ done: 0, total: eligible.length });
    let succeeded = 0;
    let failed = 0;
    for (let i = 0; i < eligible.length; i++) {
      try {
        await rollbackTransaction(eligible[i].id, staffData?.uid ??
'staff');
```

```

        succeeded++;
      } catch {
        failed++;
      }
    }
  };

```



```

        setRollbackAllProgress({ done: i + 1, total: eligible.length });
    }
    setRollingAll(false);
    setRollbackAllProgress(null);
    setRollbackAllOpen(false);
    if (succeeded > 0) toast.success(`${succeeded} transfer${succeeded
> 1 ? 's' : ''} rolled back.`);
    if (failed > 0) toast.error(`${failed} rollback${failed > 1 ?
's' : ''} failed.`);
    await loadTransactions();
};

if (loading) return <LoadingSpinner message="Loading transactions..."
/>;

return (
    <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2.5 }}>
        { /* Header */ }
        <Box sx={{ display: 'flex', alignItems: 'flex-start',
justifyContent: 'space-between', flexWrap: 'wrap', gap: 2 }}>
            <Box>
                <Typography variant="h5">
                    {isCustomer ? 'My Transfers' : 'Transaction History'}
                </Typography>
                <Typography variant="body2" color="text.secondary" mt={0.5}>
                    {transactions.length} records
                    {isStaff && rollbackCount > 0 && (
                        <Chip
                            label={` ${rollbackCount} need attention`}
                            size="small"
                            color="warning"
                            sx={{ ml: 1.5, height: 20, fontSize: 11 }}
                        />
                    )}
                </Typography>
            </Box>
        </Box>

        { /* Rollback banner - staff only */ }
        {isStaff && rollbackCount > 0 && (
            <Alert
                severity="warning"
                icon={<UndoIcon fontSize="small" />}
            </Alert>
        )}
    </Box>
);

```

```

        action={
          <Button
            size="small"
            variant="outlined"
            color="warning"
            startIcon={<UndoIcon sx={{ fontSize: 14 }} />}
            onClick={() => setRollbackAllOpen(true)}
            sx={{ fontSize: 11, whiteSpace: 'nowrap' }}
          >
            Rollback All ({rollbackCount})
          </Button>
        }
      >
        <strong>{rollbackCount} interbank transfer{rollbackCount > 1
? 's are' : ' is'} pending or failed</strong>
        { ' ' }and may not have reached the recipient.
      </Alert>
    )}

    { /* Filter bar */}
    <ToggleButtonGroup
      value={filter}
      exclusive
      onChange={(_, v) => v && setFilter(v)}
      size="small"
      sx={{ flexWrap: 'wrap' }}
    >
      {FILTERS.map(f => (
        <ToggleButton
          key={f}
          value={f}
          sx={{ textTransform: 'capitalize', px: 2, fontSize: 12 }}
        >
          {f}
        </ToggleButton>
      ))}
    </ToggleButtonGroup>

    { /* Table */}
    <TableContainer component={Paper} elevation={0} sx={{ border:
'1px solid', borderColor: 'divider' }}>
      <Table size="small">
        <TableHead>

```



```

        : txn.type === 'deposit' ? 'Deposit'
        : 'Interbank'}
      </Typography>
    </TableCell>

    { /* Direction */ }
    <TableCell>
      <Box sx={{ display: 'flex', alignItems: 'center',
gap: 0.5 }}>
        {txn.direction === 'credit'
          ? <ArrowDownwardIcon sx={{ color:
'success.main', fontSize: 15 }} />
          : <ArrowUpwardIcon sx={{ color: 'error.main',
fontSize: 15 }} />}
        <Typography
          variant="caption"
          fontWeight={600}
          sx={{ color: txn.direction === 'credit' ?
'success.main' : 'error.main', textTransform: 'capitalize' }}
        >
          {txn.direction}
        </Typography>
      </Box>
    </TableCell>

    { /* Amount */ }
    <TableCell>
      <Typography
        variant="body2"
        fontWeight={600}
        sx={{ color: txn.direction === 'credit' ?
'success.main' : 'error.main' }}
      >
        {txn.direction === 'credit' ? '+' :
'-'} {paiseToRupees(txn.amount)}
      </Typography>
    </TableCell>

    { /* Route */ }
    <TableCell>
      <Typography variant="caption"
fontFamily="monospace">

```

```

                {bankName(txn.fromBankId)} →
{bankName(txn.toBankId)}
            </Typography>
        </TableCell>

        { /* Mode */ }
        <TableCell>
            <Typography
variant="caption">{modeLabel(txn.mode)}</Typography>
        </TableCell>

        { /* Status */ }
        <TableCell>
            <StatusChip status={txn.status} />
        </TableCell>

        { /* Date */ }
        <TableCell>
            <Typography variant="caption"
noWrap>{formatDate(txn.createdAt)}</Typography>
        </TableCell>

        { /* Action — staff only */ }
        {isStaff && (
            <TableCell>
                {eligible ? (
                    <Tooltip
                        title={
                            txn.status === 'failed'
                                ? 'Hub write failed — refund sender'
                                : 'Cancel hub transfer and refund sender'
                        }
                        arrow
                    >
                        <Button
                            size="small"
                            variant="outlined"
                            color="warning"
                            startIcon={<UndoIcon sx={{ fontSize: 14 }}
/>}

                            onClick={() => setRollbackTarget(txn)}
                            sx={{ fontSize: 11, py: 0.25, px: 1,
whiteSpace: 'nowrap' }}

```

```

        >
        Rollback
      </Button>
    </Tooltip>
  ) : txn.status === 'rolled_back' ? (
    <Typography variant="caption"
color="text.disabled">Refunded</Typography>
  ) : null}
</TableCell>
  )}
</TableRow>
);
  })}
</TableBody>
</Table>
</TableContainer>

{/* Rollback All confirmation dialog */}
<Dialog
  open={rollbackAllOpen}
  onClose={() => !rollingAll && setRollbackAllOpen(false)}
  maxWidth="xs"
  fullWidth
>
  <DialogTitle sx={{ pb: 1, fontWeight: 700, fontSize: 16 }}>
    Rollback All Pending Transfers
  </DialogTitle>
  <DialogContent>
    <DialogContentText sx={{ fontSize: 13, mb: 2 }}>
      This will rollback <strong>{rollbackCount}
transfer{rollbackCount > 1 ? 's' : ''}</strong> totalling{' '}
      <strong>
{paiseToRupees(transactions.filter(isRollbackEligible).reduce((s, t) =>
s + t.amount, 0))}
      </strong>.
      Each sender will receive a full refund.
    </DialogContentText>
    {rollbackAllProgress && (
      <Box sx={{ mt: 1 }}>
        <Typography variant="caption" color="text.secondary">
          Processing {rollbackAllProgress.done} of
{rollbackAllProgress.total}...

```

```

        </Typography>
        <Box sx={{
          mt: 0.75, height: 4, borderRadius: 2,
          bgcolor: 'grey.200', overflow: 'hidden',
        }}>
          <Box sx={{
            height: '100%', borderRadius: 2, bgcolor:
'warning.main',
            width: `${(rollbackAllProgress.done /
rollbackAllProgress.total) * 100}%`,
            transition: 'width 0.2s',
          }} />
        </Box>
      </Box>
    )}
  </DialogContent>
  <DialogActions sx={{ px: 3, pb: 2, gap: 1 }}>
    <Button
      variant="outlined"
      size="small"
      onClick={() => setRollbackAllOpen(false)}
      disabled={rollingAll}
    >
      Cancel
    </Button>
    <Button
      variant="contained"
      color="warning"
      size="small"
      startIcon={rollingAll ? null : <UndoIcon />}
      onClick={handleRollbackAll}
      disabled={rollingAll}
    >
      {rollingAll ? `Processing...` : `Rollback All
(${rollbackCount})`}
    </Button>
  </DialogActions>
</Dialog>

{/* Single rollback confirmation dialog */}
<Dialog
  open={!rollbackTarget}
  onClose={() => !rolling && setRollbackTarget(null)}

```

```

        maxWidth="xs"
        fullWidth
    >
    <DialogTitle sx={{ pb: 1, fontWeight: 700, fontSize: 16 }}>
        Confirm Rollback
    </DialogTitle>
    <DialogContent>
        {rollbackTarget && (
            <>
                <DialogContentText sx={{ fontSize: 13, mb: 2 }}>
                    This will cancel the transfer and refund{' '}
                    <strong>{paiseToRupees(rollbackTarget.amount)}</strong>
to the sender's account.
                </DialogContentText>

                { /* Summary box */ }
                <Box sx={{
                    border: '1px solid',
                    borderColor: 'divider',
                    borderRadius: 1,
                    overflow: 'hidden',
                    fontSize: 13,
                }}>
                    { [
                        ['Transaction ID', rollbackTarget.id],
                        ['Amount',
paiseToRupees(rollbackTarget.amount)],
                        ['Route',
` ${bankName(rollbackTarget.fromBankId)} →
${bankName(rollbackTarget.toBankId)} `],
                        ['Current status', rollbackTarget.status],
                        ['Date',
formatDate(rollbackTarget.createdAt)],
                    ].map(([label, value], i, arr) => (
                        <Box
                            key={label}
                            sx={{
                                display: 'flex',
                                justify-content: 'space-between',
                                px: 2, py: 1,
                                bgcolor: i % 2 === 0 ? 'grey.50' :
'background.paper',

```



```

borderBottom: i < arr.length - 1 ? '1px solid' :
'none',
borderColor: 'divider',
}}
>
<Typography variant="caption"
color="text.secondary">{label}</Typography>
<Typography
variant="caption"
fontWeight={500}
fontFamily={label === 'Transaction ID' ?
'monospace' : 'inherit'}
sx={{ textTransform: 'capitalize', maxWidth: 200,
textAlign: 'right', wordBreak: 'break-all' }}
>
{value}
</Typography>
</Box>
)}}
</Box>

{rollbackTarget.status === 'pending' && (
<Alert severity="warning" sx={{ mt: 2, fontSize: 12 }}>
The hub transfer will be cancelled before the refund
is processed.
If the receiving bank has already credited the
recipient, contact them to reverse it manually.
</Alert>
)}}
</>
)}}
</DialogContent>
<DialogActions sx={{ px: 3, pb: 2, gap: 1 }}>
<Button
variant="outlined"
size="small"
onClick={() => setRollbackTarget(null)}
disabled={rolling}
>
Cancel
</Button>
<Button
variant="contained"

```

```

        color="warning"
        size="small"
        startIcon={rolling ? null : <UndoIcon />}
        onClick={handleRollback}
        disabled={rolling}
      >
        {rolling ? 'Processing...' : 'Confirm Rollback'}
      </Button>
    </DialogActions>
  </Dialog>
</Box>
);
};

```

src/pages/transfer/Dashboard.jsx

```

// src/pages/Dashboard.jsx

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import {
  Grid, Box, Typography, Button, Alert,
  List, ListItem, ListItemText, Divider,
} from '@mui/material';
import SwapHorizIcon      from '@mui/icons-material/SwapHoriz';
import CreditCardIcon     from '@mui/icons-material/CreditCard';
import HistoryIcon        from '@mui/icons-material/History';
import TrendingUpIcon     from '@mui/icons-material/TrendingUp';
import AccountBalanceIcon from '@mui/icons-material/AccountBalance';
import ArrowDownwardIcon  from '@mui/icons-material/ArrowDownward';
import ArrowUpwardIcon    from '@mui/icons-material/ArrowUpward';
import AddIcon            from '@mui/icons-material/Add';
import { useAuth }        from '../context/AuthContext';
import { getAllAccounts, getMyAccounts } from
'../services/accountService';
import { getAllTransactions, getTransactionsByAccount } from
'../services/transactionService';
import { paiseToRupees, formatDate } from '../utils/formatters';
import { BANK_NAME } from '../config/constants';
import { StatCard, Card, LoadingSpinner, StatusChip } from
'../components/ui';

const KYC_ALERTS = {

```

```

    not_submitted: { severity: 'info',    title: 'Complete your KYC',
body: 'Submit your identity documents to activate your account.', cta:
'Submit KYC', to: '/kyc' },
    pending:      { severity: 'warning', title: 'KYC under review',
body: 'Your documents are being reviewed. We will notify you within 1-2
business days.', cta: 'View status', to: '/kyc' },
    hold:         { severity: 'warning', title: 'KYC on hold',
body: 'Additional information required. Please check your KYC page.',
cta: 'View status', to: '/kyc' },
    rejected:     { severity: 'error',   title: 'KYC rejected',
body: 'Your KYC was rejected. Please resubmit with correct documents.',
cta: 'Resubmit', to: '/kyc' },
  };

export const Dashboard = () => {
  const { isCustomer, isStaff, customerData, displayName, kycStatus } =
useAuth();

  const navigate = useNavigate();
  const [accounts, setAccounts] = useState([]);
  const [transactions, setTransactions] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    (async () => {
      try {
        if (isCustomer && kycStatus === 'approved') {
          const accs = await getMyAccounts(customerData.uid);
          setAccounts(accs);
          if (accs.length > 0) {
            const map = new Map();
            (await Promise.all(accs.map(a =>
getTransactionsByAccount(a.id)))).flat()
              .forEach(t => map.set(t.id, t));
            setTransactions(
              [...map.values()]
                .sort((a, b) => (b.createdAt?.toMillis?.() ?? 0) -
(a.createdAt?.toMillis?.() ?? 0))
                .slice(0, 10)
            );
          }
        } else if (isStaff) {
          const [accs, txns] = await Promise.all([getAllAccounts(),
getAllTransactions()]);

```

```

        setAccounts(accs);
        setTransactions(txns.slice(0, 10));
    }
    } finally { setLoading(false); }
  })();
}, [isCustomer, isStaff, customerData, kycStatus]);

if (loading) return <LoadingSpinner message="Loading dashboard..." />;

const totalBalance = accounts.reduce((s, a) => s + (a.balance ??
0), 0);
const activeAccounts = accounts.filter(a => a.status ===
'active').length;
const pendingTxns = transactions.filter(t => t.status ===
'pending').length;
const alert = isCustomer ? KYC_ALERTS[kycStatus] : null;
const hour = new Date().getHours();
const greeting = hour < 12 ? 'Good Morning' : hour < 17 ? 'Good
Afternoon' : 'Good Evening';

return (
  <Box sx={{ display: 'flex', flexDirection: 'column', gap: 2.5 }}>
    </* Page header */>
    <Box sx={{ display: 'flex', alignItems: 'center', justifyContent:
'space-between', flexWrap: 'wrap', gap: 2 }}>
      <Box>
        <Typography variant="h5">{greeting}, {displayName.split('
')[0]}</Typography>
        <Typography variant="body2" color="text.secondary" mt={0.5}>
          {BANK_NAME}
        </Typography>
      </Box>

      {(isStaff || kycStatus === 'approved') && (
        <Box sx={{ display: 'flex', gap: 1 }}>
          <Button variant="contained" color="primary" size="small"
startIcon={<SwapHorizIcon />} onClick={() => navigate('/send-money')}>
            Send Money
          </Button>
          <Button variant="outlined" color="secondary" size="small"
startIcon={<HistoryIcon />} onClick={() => navigate('/transactions')}>
            History
          </Button>

```

```

        {isStaff && (
            <Button variant="outlined" color="secondary" size="small"
startIcon={<AddIcon />} onClick={() => navigate('/accounts/new')}>
                Open Account
            </Button>
        )}
    </Box>
)}
</Box>

{/* KYC alert */}
{alert && (
    <Alert
        severity={alert.severity}
        action={
            <Button size="small" color="inherit" onClick={() =>
navigate(alert.to)} variant="text">
                {alert.cta}
            </Button>
        }
    >
        <strong>{alert.title}</strong> &mdash; {alert.body}
    </Alert>
)}

{/* Stats row */}
{(isStaff || kycStatus === 'approved') && (
    <>
        <Grid container spacing={2}>
            <Grid item xs={6} sm={3}>
                <StatCard
                    label="Total Balance"
                    value={paiseToRupees(totalBalance)}
                    icon={TrendingUpIcon}
                    color="#E22026"
                    trendLabel="Across all accounts"
                />
            </Grid>
            <Grid item xs={6} sm={3}>
                <StatCard
                    label="Active Accounts"
                    value={activeAccounts}
                    icon={CreditCardIcon}

```

```

        color="#003E8A"
        trendLabel={` ${accounts.length} total`}
      />
    </Grid>
    <Grid item xs={6} sm={3}>
      <StatCard
        label="Transactions"
        value={transactions.length}
        icon={HistoryIcon}
        color="#1B7E47"
        trendLabel="Recent activity"
      />
    </Grid>
    <Grid item xs={6} sm={3}>
      <StatCard
        label="Pending"
        value={pendingTxns}
        icon={AccountBalanceIcon}
        color={pendingTxns > 0 ? '#B25E00' : '#1B7E47'}
        trend={pendingTxns === 0 ? 1 : -1}
        trendLabel={pendingTxns > 0 ? 'Needs review' : 'All
clear'}
      />
    </Grid>
  </Grid>

  <Grid container spacing={2.5}>
    { /* Recent Transactions */ }
    <Grid item xs={12} md={7}>
      <Card
        title="Recent Transfers"
        action={
          <Button size="small" variant="text" color="secondary"
onClick={() => navigate('/transactions')}>
            View all
          </Button>
        }
      >
        {transactions.length === 0 ? (
          <Box sx={{ py: 4, textAlign: 'center' }}>
            <Typography variant="body2"
color="text.secondary">No transfers yet.</Typography>
          </Box>

```

```

    ) : (
      <List dense disablePadding>
        {transactions.slice(0, 8).map((txn, i) => {
          const isCredit = txn.direction === 'credit';
          return (
            <Box key={txn.id}>
              <ListItem disablePadding sx={{ py: 1.25 }}>
                <Box sx={{
                  width: 32, height: 32,
                  bgcolor: isCredit ? 'success.light' :
'error.light',
                  borderRadius: 1,
                  display: 'flex', alignItems: 'center',
justifyContent: 'center',
                  mr: 1.5, flexShrink: 0,
                }}>
                  {isCredit
                    ? <ArrowDownwardIcon sx={{ fontSize:
15, color: 'success.main' }} />
                    : <ArrowUpwardIcon sx={{ fontSize:
15, color: 'error.main' }} />}
                </Box>
                <ListItemText
                  primary={
                    txn.type === 'intrabank_transfer'
                      ? 'Internal Transfer'
                      : `${txn.mode?.toUpperCase()} ·
${isCredit ? txn.fromBankId : txn.toBankId}`
                  }
                  secondary={formatDate(txn.createdAt)}
                  primaryTypographyProps={{ fontSize: 13,
fontWeight: 500, color: 'text.primary' }}
                  secondaryTypographyProps={{ fontSize: 11
                }}
                />
                <Box sx={{ textAlign: 'right', ml: 1,
flexShrink: 0 }}>
                  <Typography
                    variant="body2"
                    fontWeight={700}
                    sx={{ color: isCredit ? 'success.main'
: 'error.main', fontSize: 13 }}
                  >

```

```

                                {isCredit ? '+' :
'-'} {paiseToRupees(txn.amount) }
                                </Typography>
                                <StatusChip status={txn.status} />
                                </Box>
                                </ListItem>
                                {i < Math.min(transactions.length, 8) - 1 &&
<Divider />}

                                </Box>
                                );
                                }}}
                                </List>
                                )}
                                </Card>
                                </Grid>

                                { /* Accounts */
                                <Grid item xs={12} md={5}>
                                <Card
                                title={isCustomer ? 'My Accounts' : 'Accounts'}
                                action={
                                isStaff
                                ? <Button size="small" variant="text"
color="secondary" onClick={() => navigate('/accounts')}>Manage</Button>
                                : null
                                }
                                >
                                {accounts.length === 0 ? (
                                <Box sx={{ py: 4, textAlign: 'center' }}>
                                <Typography variant="body2" color="text.secondary">
                                {isCustomer
                                ? 'Your account will appear here once opened by
the bank.'
                                : 'No accounts yet.'}
                                </Typography>
                                </Box>
                                ) : (
                                <List dense disablePadding>
                                {accounts.slice(0, 5).map((acc, i) => (
                                <Box key={acc.id}>
                                <ListItem
                                disablePadding
                                sx={{

```



```

        py: 1.25,
        cursor: isStaff ? 'pointer' : 'default',
        '&:hover': isStaff ? { bgcolor: 'grey.50' }
: {},

        borderRadius: 1,
    }}
    onClick={() => isStaff &&
navigate(`/accounts/${acc.id}`)}
    >
    <Box sx={{
        width: 32, height: 32,
        bgcolor: 'secondary.light',
        borderRadius: 1,
        display: 'flex', alignItems: 'center',
justifyContent: 'center',
        mr: 1.5, flexShrink: 0,
    }}>
        <CreditCardIcon sx={{ fontSize: 15, color:
'secondary.main' }} />
    </Box>
    <ListItemText
        primary={
            <Typography variant="body2"
fontWeight={600} fontFamily="monospace" fontSize={12}
color="text.primary">
                {isCustomer ? acc.accountNumber :
`.....${acc.accountNumber?.slice(-4)} `}
            </Typography>
        }
        secondary={
            <Typography variant="caption"
color="text.secondary">
                {acc.accountType} &middot;
{acc.ifscCode}
                {isStaff && ` · ${acc.customerName}`}
            </Typography>
        }
    />
    <Box sx={{ textAlign: 'right', ml: 1,
flexShrink: 0 }}>
        <Typography variant="body2"
fontWeight={700} fontSize={13} color="text.primary">
            {paiseToRupees(acc.balance)}

```

```

        </Typography>
        <StatusChip status={acc.status} />
      </Box>
    </ListItem>
    {i < accounts.length - 1 && <Divider />}
  </Box>
  )}}
</List>
  )}
</Card>
</Grid>
</Grid>
</>
  )}
</Box>
);
};

```

src/services/accountService.js

```

// src/services/accountService.js
// No orderBy() alongside where() - sorts in JavaScript to avoid
composite indexes.

import {
  collection, doc, getDoc, getDocs, addDoc, updateDoc,
  query, where, serverTimestamp, runTransaction,
} from 'firebase/firestore';
import { privateDb } from '../config/firebasePrivate';
import { BANK_ID, BANK_IFSC } from '../config/constants';
import { generateAccountNumber } from '../utils/formatters';

const byCreatedAtDesc = (a, b) =>
  (b.createdAt?.toMillis?.() ?? 0) - (a.createdAt?.toMillis?.() ?? 0);

// — Customers

export const createCustomer = async (data) => {
  const ref = await addDoc(collection(privateDb, 'customers'), {
    ...data,
    bankId: BANK_ID,
    status: 'active',
  });
};

```

```

        createdAt: serverTimestamp(),
        updatedAt: serverTimestamp(),
    });
    return ref.id;
};

export const getCustomer = async (customerId) => {
    const snap = await getDoc(doc(privateDb, 'customers', customerId));
    if (!snap.exists()) throw new Error('Customer not found');
    return { id: snap.id, ...snap.data() };
};

export const getAllCustomers = async () => {
    const q = query(collection(privateDb, 'customers'),
    where('bankId', '==', BANK_ID));
    const snap = await getDocs(q);
    return snap.docs.map(d => ({ id: d.id, ...d.data()
    })).sort(byCreatedAtDesc);
};

// — Accounts

```

```

export const createAccount = async ({ customerId, accountType,
initialDepositPaise = 0 }) => {
    const customer = await getCustomer(customerId);
    const accountNumber = generateAccountNumber();

    const ref = await addDoc(collection(privateDb, 'accounts'), {
        customerId,
        customerName: customer.fullName,
        accountNumber,
        accountType,
        bankId: BANK_ID,
        ifscCode: BANK_IFSC,
        balance: initialDepositPaise,
        status: 'active',
        createdAt: serverTimestamp(),
        updatedAt: serverTimestamp(),
    });

    return { id: ref.id, accountNumber };
};

```

```

export const getAccount = async (accountId) => {
  const snap = await getDoc(doc(privateDb, 'accounts', accountId));
  if (!snap.exists()) throw new Error('Account not found');
  return { id: snap.id, ...snap.data() };
};

export const getAccountByNumber = async (accountNumber) => {
  const q = query(
    collection(privateDb, 'accounts'),
    where('accountNumber', '==', accountNumber),
    where('bankId', '==', BANK_ID),
  );
  const snap = await getDocs(q);
  if (snap.empty) return null;
  const d = snap.docs[0];
  return { id: d.id, ...d.data() };
};

export const getAccountsByCustomer = async (customerId) => {
  const q = query(collection(privateDb, 'accounts'),
    where('customerId', '==', customerId));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data()
})).sort(byCreatedAtDesc);
};

// Staff: all accounts in this bank
export const getAllAccounts = async () => {
  const q = query(collection(privateDb, 'accounts'), where('bankId',
'==', BANK_ID));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data()
})).sort(byCreatedAtDesc);
};

// Customer: only their own accounts (by uid = customerId)
export const getMyAccounts = async (uid) => {
  return getAccountsByCustomer(uid);
};

export const updateAccountStatus = async (accountId, status) => {
  await updateDoc(doc(privateDb, 'accounts', accountId), {

```

```

        status,
        updatedAt: serverTimestamp(),
    });
};

// Staff deposits cash into an account and records a credit transaction
export const depositToAccount = async (accountId, amountPaise,
staffUid, remarks = '') => {
    const accountRef = doc(privateDb, 'accounts', accountId);
    const txnRef      = doc(collection(privateDb, 'transactions'));

    await runTransaction(privateDb, async (txn) => {
        const snap = await txn.get(accountRef);
        if (!snap.exists()) throw new Error('Account not found. ');
        if (snap.data().status !== 'active') throw new Error('Account is
not active. ');

        const newBalance = (snap.data().balance ?? 0) + amountPaise;
        txn.update(accountRef, { balance: newBalance, updatedAt:
serverTimestamp() });
        txn.set(txnRef, {
            accountId,
            bankId:    BANK_ID,
            type:      'deposit',
            mode:      'cash',
            direction: 'credit',
            amount:    amountPaise,
            status:    'completed',
            remarks,
            performedBy: staffUid,
            createdAt:  serverTimestamp(),
        });
    });
};
};
};

```

src/services/emailService.js

```

// src/services/emailService.js
// Calls the Express backend which uses NodeMailer to send emails.

import { BANK_NAME } from '../config/constants';

```

```

const API_URL = import.meta.env.VITE_API_URL ||
'http://localhost:3001';

export const sendKYCEmail = async (toEmail, customerName, action,
reason = '') => {
  try {
    const res = await fetch(`${API_URL}/api/send-email`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        to: toEmail,
        customerName,
        bankName: BANK_NAME,
        action,
        reason,
      }),
    });
    const data = await res.json();
    if (!data.success) {
      console.warn('[EMAIL] Server reported failure:', data.error);
    } else {
      console.log('[EMAIL] Sent successfully:', data.messageId);
    }
  } catch (err) {
    // Non-fatal - email failure should never block KYC actions
    console.warn('[EMAIL] Could not reach email server:', err.message);
    console.warn('Make sure the backend is running: cd server && npm
run dev');
  }
};

```

src/services/hubService.js

```

// src/services/hubService.js
// Manages all interactions with the shared Firebase hub.
// Bank B uses startIncomingListener() to detect and process incoming
transfers.
// Bank A uses startStatusListener() to track the completion of sent
transfers.

import {
  collection, doc, query, where, onSnapshot, getDocs,
  setDoc, serverTimestamp,

```

```

} from 'firebase/firestore';
import { hubDb } from '../config/firebaseHub';
import { BANK_ID, BANK_NAME, BANK_IFSC } from '../config/constants';
import { processIncomingTransfer, syncTransactionStatus } from
'./transactionService';

// — Incoming transfer listener (runs on Bank B's instance)


---


// Listens for pending transfers addressed to THIS bank in the hub.
// When a new doc appears: validate → credit → update hub status.
//
// Returns an unsubscribe function — call it on component unmount or
logout.

export const startIncomingListener = (onProcessed, onError) => {
  console.log(`[HUB LISTENER] Starting incoming listener for bankId:
${BANK_ID}`);
  const q = query(
    collection(hubDb, 'interbank_transfers'),
    where('toBankId', '==', BANK_ID),
    where('status', '==', 'pending'),
  );

  const unsubscribe = onSnapshot(q, async (snapshot) => {
    console.log(`[HUB LISTENER] Snapshot fired —
${snapshot.docChanges().length} change(s)`);
    for (const change of snapshot.docChanges()) {
      if (change.type === 'added') {
        const transfer = { id: change.doc.id, ...change.doc.data() };
        console.log(`[HUB LISTENER] New transfer detected:`,
transfer.transferId, 'amount:', transfer.amount);
        try {
          await processIncomingTransfer(transfer);
          console.log(`[HUB LISTENER] Processed successfully:`,
transfer.transferId);
          onProcessed?.(transfer);
        } catch (err) {
          console.error(`[HUB LISTENER] Processing failed:`,
transfer.transferId, err.message);
          onError?.(transfer, err);
        }
      }
    }
  })
}

```

```

    }, (err) => {
      console.error('[HUB LISTENER] onSnapshot error:', err.code,
err.message);
      onError?.(null, err);
    });

    return unsubscribe;
  };

// — Outgoing transfer status listener (runs on Bank A's instance)


---


// Watches hub for status changes on transfers sent BY this bank.
// When status becomes completed/failed: sync local private DB
transaction.

export const startStatusListener = (onStatusChange) => {
  const q = query(
    collection(hubDb, 'interbank_transfers'),
    where('fromBankId', '==', BANK_ID),
    where('status', 'in', ['completed', 'failed']),
  );

  const unsubscribe = onSnapshot(q, async (snapshot) => {
    for (const change of snapshot.docChanges()) {
      if (change.type === 'added' || change.type === 'modified') {
        const transfer = { id: change.doc.id, ...change.doc.data() };
        if (transfer.status === 'pending') continue; // Skip pending

        console.log(`[HUB] Transfer ${transfer.transferId} status →
${transfer.status}`);
        try {
          await syncTransactionStatus(transfer.transferId,
transfer.status, transfer.failureReason);
          onStatusChange?.(transfer);
        } catch (err) {
          console.error('[HUB] Status sync error:', err);
        }
      }
    }
  });

  return unsubscribe;
};

```



```
// — Register this bank in the hub registry


---


// Called once during setup. Idempotent — safe to call on every app
start.

export const registerBankInHub = async () => {
  try {
    await setDoc(doc(hubDb, 'banks', BANK_ID), {
      bankId:    BANK_ID,
      bankName:  BANK_NAME,
      ifscPrefix: BANK_IFSC.slice(0, 4),
      isActive:  true,
      updatedAt: serverTimestamp(),
    }, { merge: true });
  } catch (err) {
    // Non-fatal — hub rules may restrict this write; admin does it
manually
    console.warn('[HUB] Could not register bank in hub:', err.message);
  }
};

// — Register a public account routing entry


---


// Called when a new account is created, so other banks can route to
it.
// Stores ONLY masked account number — no balance, no customer PII.

export const registerPublicAccount = async (accountId, accountNumber)
=> {
  const masked = '....' + String(accountNumber).slice(-4);
  try {
    await setDoc(doc(hubDb, 'public_accounts', accountId), {
      accountId,
      bankId:    BANK_ID,
      maskedAccountNumber: masked,
      ifscCode:  BANK_IFSC,
      isActive:  true,
      registeredAt: serverTimestamp(),
    }, { merge: true });
  } catch (err) {
    console.warn('[HUB] Could not register public account:',
err.message);
  }
};
```

```

    }
  };

  // — Fetch all registered banks from hub
  _____

  // Used to populate the "Destination Bank" dropdown in Send Money form.

  export const getAllBanks = async () => {
    try {
      const snap = await getDocs(collection(hubDb, 'banks'));
      return snap.docs.map(d => ({ id: d.id, ...d.data() }));
    } catch {
      return [];
    }
  };

```

src/services/kycServices.js

```

// src/services/kycService.js

import {
  doc, setDoc, updateDoc, getDoc, getDocs,
  collection, query, where, serverTimestamp,
} from 'firebase/firestore';
import { privateDb } from '../config/firebasePrivate';
import { sendKYCEmail } from './emailService';

export const submitKYC = async (uid, data) => {
  await setDoc(doc(privateDb, 'kyc', uid), {
    customerId: uid,
    fullName: data.fullName,
    pan: data.pan,
    aadhaarLast4: data.aadhaarLast4,
    dob: data.dob,
    address: data.address,
    docType: data.docType,
    status: 'pending',
    submittedAt: serverTimestamp(),
    reviewedAt: null,
    reviewedBy: null,
    rejectionReason: null,
    holdReason: null,
  });

```

```

    await updateDoc(doc(privateDb, 'customers', uid), {
      kycStatus: 'pending',
      updatedAt: serverTimestamp(),
    });
  });
};

export const getKYC = async (uid) => {
  const snap = await getDoc(doc(privateDb, 'kyc', uid));
  if (!snap.exists()) return null;
  return { id: snap.id, ...snap.data() };
};

export const getPendingKYC = async () => {
  const q = query(collection(privateDb, 'kyc'), where('status',
'==', 'pending'));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }));
};

export const getHoldKYC = async () => {
  const q = query(collection(privateDb, 'kyc'), where('status',
'==', 'hold'));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }));
};

export const getApprovedKYC = async () => {
  const q = query(collection(privateDb, 'kyc'), where('status',
'==', 'approved'));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }));
};

export const approveKYC = async (customerId, staffUid, customerEmail,
customerName) => {
  await updateDoc(doc(privateDb, 'kyc', customerId), {
    status: 'approved', reviewedAt: serverTimestamp(), reviewedBy:
staffUid,
  });
  await updateDoc(doc(privateDb, 'customers', customerId), {
    kycStatus: 'approved', updatedAt: serverTimestamp(),
  });
  // Send email notification

```

```

    await sendKYCEmail(customerEmail, customerName, 'approved');
  };

export const holdKYC = async (customerId, staffUid, reason,
customerEmail, customerName) => {
  await updateDoc(doc(privateDb, 'kyc', customerId), {
    status: 'hold', reviewedAt: serverTimestamp(), reviewedBy:
staffUid, holdReason: reason,
  });
  await updateDoc(doc(privateDb, 'customers', customerId), {
    kycStatus: 'hold', updatedAt: serverTimestamp(),
  });
  await sendKYCEmail(customerEmail, customerName, 'hold', reason);
};

export const rejectKYC = async (customerId, staffUid, reason,
customerEmail, customerName) => {
  await updateDoc(doc(privateDb, 'kyc', customerId), {
    status: 'rejected', reviewedAt: serverTimestamp(), reviewedBy:
staffUid, rejectionReason: reason,
  });
  await updateDoc(doc(privateDb, 'customers', customerId), {
    kycStatus: 'rejected', updatedAt: serverTimestamp(),
  });
  await sendKYCEmail(customerEmail, customerName, 'rejected', reason);
};

```

src/services/loanService.js

```

// src/services/loanService.js
// Loan application, approval, and management.
// All loans stored in private Firestore loans/ collection.

import {
  doc, addDoc, updateDoc, getDocs, getDoc,
  collection, query, where, serverTimestamp,
} from 'firebase/firestore';
import { privateDb } from '../config/firebasePrivate';
import { BANK_ID } from '../config/constants';

const LOAN_TYPES = {
  personal: { label: 'Personal Loan', interestRate: 12 },
  home: { label: 'Home Loan', interestRate: 8.5 },

```

```

    education: { label: 'Education Loan',    interestRate: 9 },
    vehicle:   { label: 'Vehicle Loan',      interestRate: 10 },
    business:  { label: 'Business Loan',     interestRate: 14 },
  };

export { LOAN_TYPES };

// Calculate EMI:  $P \times r \times (1+r)^n / ((1+r)^n - 1)$ 
export const calculateEMI = (principalPaise, annualRate, months) => {
  const principal = principalPaise / 100;
  const r = annualRate / 12 / 100;
  if (r === 0) return principal / months;
  const emi = principal * r * Math.pow(1 + r, months) / (Math.pow(1 + r, months) - 1);
  return Math.round(emi * 100); // return in paise
};

// Customer applies for a loan
export const applyLoan = async (customerId, { loanType, amountPaise, durationMonths, purpose }) => {
  const rate = LOAN_TYPES[loanType]?.interestRate ?? 12;
  const emi = calculateEMI(amountPaise, rate, durationMonths);

  const ref = await addDoc(collection(privateDb, 'loans'), {
    customerId,
    bankId:      BANK_ID,
    loanType,
    amountPaise,
    durationMonths,
    purpose,
    interestRate: rate,
    emiPaise:    emi,
    status:      'pending',
    appliedAt:   serverTimestamp(),
    reviewedAt:  null,
    reviewedBy:  null,
    rejectionReason: null,
    disbursedAt: null,
  });
  return ref.id;
};

// Get loans for a specific customer

```

```

export const getMyLoans = async (customerId) => {
  const q = query(collection(privateDb, 'loans'),
where('customerId', '==', customerId));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }))
    .sort((a, b) => (b.appliedAt?.toMillis?.() ?? 0) -
(a.appliedAt?.toMillis?.() ?? 0));
};

// Get all loans (admin)
export const getAllLoans = async () => {
  const q = query(collection(privateDb, 'loans'), where('bankId',
'==', BANK_ID));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }))
    .sort((a, b) => (b.appliedAt?.toMillis?.() ?? 0) -
(a.appliedAt?.toMillis?.() ?? 0));
};

// Get pending loans (admin)
export const getPendingLoans = async () => {
  const q = query(collection(privateDb, 'loans'), where('status',
'==', 'pending'));
  const snap = await getDocs(q);
  return snap.docs.map(d => ({ id: d.id, ...d.data() }));
};

// Admin approves loan
export const approveLoan = async (loanId, staffUid) => {
  await updateDoc(doc(privateDb, 'loans', loanId), {
    status: 'approved',
    reviewedAt: serverTimestamp(),
    reviewedBy: staffUid,
    disbursedAt: serverTimestamp(),
  });
};

// Admin rejects loan
export const rejectLoan = async (loanId, staffUid, reason) => {
  await updateDoc(doc(privateDb, 'loans', loanId), {
    status: 'rejected',
    reviewedAt: serverTimestamp(),
    reviewedBy: staffUid,
  });
};

```

```

        rejectionReason: reason,
    });
};

// Admin puts loan on hold (pending further review)
export const holdLoan = async (loanId, staffUid, reason) => {
    await updateDoc(doc(privateDb, 'loans', loanId), {
        status: 'hold',
        reviewedAt: serverTimestamp(),
        reviewedBy: staffUid,
        holdReason: reason,
    });
};

// Get loans on hold (admin)
export const getHoldLoans = async () => {
    const q = query(collection(privateDb, 'loans'), where('status',
'==', 'hold'));
    const snap = await getDocs(q);
    return snap.docs.map(d => ({ id: d.id, ...d.data() }));
};

```

src/services/transactionService.js

```

// src/services/transactionService.js
//
// ALL balance mutations use Firestore runTransaction() for atomicity.
// Interbank transfers use a two-phase approach:
//   Phase 1 → atomic deduction in private DB + local pending record
//   Phase 2 → write transfer request to shared hub Firestore
//
// Without Cloud Functions, Phase 1 and Phase 2 are NOT globally
atomic.
// If Phase 2 fails, the local record is marked 'failed' for manual
review.

import {
    doc, collection, runTransaction, setDoc, updateDoc, getDoc,
    serverTimestamp, query, where, getDocs,
} from 'firebase/firestore';
import { privateDb } from '../config/firebasePrivate';
import { hubDb } from '../config/firebaseHub';
import { BANK_ID } from '../config/constants';

```

```
// src/services/transactionService.js - Fixed transfer functions

// — Intrabank transfer



---



// Creates ONLY one record per account view (debit for sender, credit
for receiver)

// But when fetching for a specific account, we only show relevant
direction

export const intrabankTransfer = async ({

  fromAccountId,

  toAccountId,

  amountPaise,

  mode = 'internal',

  remarks = '',

}) => {

  if (fromAccountId === toAccountId) throw new Error('Cannot transfer
to the same account.');
```

```

  const fromRef = doc(privateDb, 'accounts', fromAccountId);

  const toRef = doc(privateDb, 'accounts', toAccountId);

  const transferId =
`TXN_${Date.now()}_${Math.random().toString(36).substring(2, 6)}`;

  await runTransaction(privateDb, async (txn) => {

    const [fromSnap, toSnap] = await Promise.all([txn.get(fromRef),
txn.get(toRef)]);

    if (!fromSnap.exists()) throw new Error('Sender account not
found.');
```



```
    if (!toSnap.exists()) throw new Error('Recipient account not found.');
```

```
    if (fromSnap.data().status !== 'active') throw new Error('Sender account is not active.');
```

```
    if (toSnap.data().status !== 'active') throw new Error('Recipient account is not active.');
```

```
    if (fromSnap.data().balance < amountPaise) throw new Error('Insufficient balance.');
```

```
    const now = serverTimestamp();
```

```
    const newFromBalance = fromSnap.data().balance - amountPaise;
```

```
    const newToBalance = toSnap.data().balance + amountPaise;
```

```
    txn.update(fromRef, { balance: newFromBalance, updatedAt: now });
```

```
    txn.update(toRef, { balance: newToBalance, updatedAt: now });
```

```
    // Create ONE debit record for the sender
```

```
    txn.set(doc(privateDb, 'transactions', `${transferId}_DEBIT`), {
```

```
      transactionId: transferId,
```

```
      type: 'intrabank_transfer',
```

```
      direction: 'debit',
```

```
      fromAccountId,
```

```
      toAccountId,
```

```
      fromBankId: BANK_ID,
```

```
      toBankId: BANK_ID,
```

```
      amount: amountPaise,
```

```
    currency: 'INR',

    status: 'completed',

    mode,

    remarks,

    createdAt: now,

    completedAt: now,

    balanceAfter: newFromBalance,

  });

  // Create ONE credit record for the receiver

  txn.set(doc(privateDb, 'transactions', `${transferId}_CREDIT`), {

    transactionId: transferId,

    type: 'intrabank_transfer',

    direction: 'credit',

    fromAccountId,

    toAccountId,

    fromBankId: BANK_ID,

    toBankId: BANK_ID,

    amount: amountPaise,

    currency: 'INR',

    status: 'completed',

    mode,

    remarks,
```

```

        createdAt: now,

        completedAt: now,

        balanceAfter: newToBalance,

    });

});

return transferId;

};

// — Interbank transfer — Phase 1: deduct + local pending


---



export const initiateInterbankTransfer = async ({

    fromAccountId,

    toAccountId,

    toBankId,

    amountPaise,

    mode,

    remarks = '',

}) => {

    const fromRef = doc(privateDb, 'accounts', fromAccountId);

    const transferId =
`IBT_${Date.now()}_${Math.random().toString(36).substring(2, 8)}`;

    // Phase 1: atomic deduction in private DB

    await runTransaction(privateDb, async (txn) => {

```

```
const fromSnap = await txn.get(fromRef);

if (!fromSnap.exists()) throw new Error('Sender account not found.');
```



```
if (fromSnap.data().status !== 'active') throw new Error('Account is not active.');
```



```
if (fromSnap.data().balance < amountPaise) throw new Error('Insufficient balance.');
```



```
const newBalance = fromSnap.data().balance - amountPaise;
```



```
txn.update(fromRef, {

  balance: newBalance,

  updatedAt: serverTimestamp(),

});
```



```
// Create ONE debit record for the sender (pending status)
```



```
txn.set(doc(privateDb, 'transactions', `${transferId}_DEBIT`), {

  transactionId: transferId,

  type: 'interbank_transfer',

  direction: 'debit',

  fromAccountId,

  toAccountId,

  fromBankId: BANK_ID,

  toBankId,

  amount: amountPaise,

  currency: 'INR',
```

```
        status: 'pending',

        mode,

        remarks,

        createdAt: serverTimestamp(),

        completedAt: null,

        failureReason: null,

        balanceAfter: newBalance,

    });

});

// Phase 2: write to shared hub

try {

    console.log('[TRANSFER] Writing to hub:', transferId, '→',
toBankId);

    await setDoc(doc(hubDb, 'interbank_transfers', transferId), {

        transferId,

        fromBankId: BANK_ID,

        toBankId,

        fromAccountId,

        toAccountId,

        amount: amountPaise,

        currency: 'INR',

        mode,
```

```

        status: 'pending',

        createdAt: serverTimestamp(),

        completedAt: null,

        failureReason: null,

    });

    console.log('[TRANSFER] Hub write success:', transferId);

} catch (hubError) {

    console.error('[TRANSFER] Hub write FAILED:', hubError.code,
hubError.message);

    // Update the transaction to failed

    await updateDoc(doc(privateDb, 'transactions',
`${transferId}_DEBIT`), {

        status: 'failed',

        failureReason: `Hub write failed: ${hubError.message}`,

        completedAt: serverTimestamp(),

    });

    throw new Error(`Hub write failed: ${hubError.message}`);

}

return transferId;

};

// — Process an incoming interbank transfer (called by Bank B's hub
listener) —
//
// CRITICAL FIX: toAccountId in the hub document is the ACCOUNT NUMBER
string

```

```

// (e.g. "123456789012") that the sender typed into the Send Money form
-
// NOT the Firestore document ID. We must resolve it first before
crediting.

export const processIncomingTransfer = async (hubTransfer) => {
  const { transferId, fromBankId, fromAccountId, toAccountId, amount }
= hubTransfer;

  const localRef = doc(privateDb, 'transactions', transferId);
  const hubRef    = doc(hubDb,      'interbank_transfers', transferId);

  // Step 1: Resolve account NUMBER → Firestore document ID
  console.log('[PROCESS] Resolving account number:', toAccountId);
  const acctSnap = await getDocs(
    query(collection(privateDb, 'accounts'), where('accountNumber',
'==', toAccountId))
  );

  if (acctSnap.empty) {
    console.error('[PROCESS] Account number not found:', toAccountId);
    await updateDoc(hubRef, {
      status:      'failed',
      failureReason: `Recipient account number "${toAccountId}" not
found in this bank.`,
      completedAt:  serverTimestamp(),
    });
    return;
  }

  const toDocId = acctSnap.docs[0].id;
  const toRef    = doc(privateDb, 'accounts', toDocId);
  console.log('[PROCESS] Resolved account doc ID:', toDocId, '-
crediting', amount, 'paise');

  // Step 2: Atomic credit via runTransaction
  try {
    await runTransaction(privateDb, async (txn) => {
      const toSnap    = await txn.get(toRef);
      const localSnap = await txn.get(localRef);

      // Duplicate guard - transferId is the idempotency key
      if (localSnap.exists()) throw new Error('DUPLICATE');

```

```

        if (!toSnap.exists()) throw new Error('Recipient account not
found.');
```

```

        if (toSnap.data().status !== 'active') throw new Error('Recipient
account is not active.');
```

```

        const newBalance = toSnap.data().balance + amount;
```

```

        txn.update(toRef, { balance: newBalance, updatedAt:
serverTimestamp() });
```

```

        // Store the resolved doc ID, not the account number
        txn.set(localRef, {
            transactionId: transferId,
            type:          'interbank_transfer',
            direction:     'credit',
            fromAccountId,
            toAccountId:   toDocId,
            fromBankId,
            toBankId:      BANK_ID,
            amount,
            currency:      'INR',
            status:         'completed',
            mode:           hubTransfer.mode,
            createdAt:      serverTimestamp(),
            completedAt:    serverTimestamp(),
            failureReason: null,
        });
    });
});

await updateDoc(hubRef, { status: 'completed', completedAt:
serverTimestamp() });

console.log('[PROCESS] Transfer', transferId, 'completed - balance
updated.');
```

```

    } catch (err) {
        if (err.message === 'DUPLICATE') {
            console.log('[PROCESS] Transfer', transferId, 'already processed
- skipping.');
```

```

            return;
        }
        console.error('[PROCESS] Credit failed:', err.message);
        await updateDoc(hubRef, {

```



```

        status:      'failed',
        failureReason: err.message,
        completedAt:  serverTimestamp(),
    });
    throw err;
}
};

// — Sync local transaction status from hub update


---



export const syncTransactionStatus = async (transferId, status,
failureReason = null) => {
    const ref = doc(privateDb, 'transactions', transferId);
    const snap = await getDoc(ref);
    if (!snap.exists()) return;
    if (snap.data().direction !== 'debit') return; // only sync sender
side
    await updateDoc(ref, {
        status,
        ...(failureReason && { failureReason }),
        completedAt: serverTimestamp(),
    });
};

// — Rollback a stuck/failed interbank debit


---



//
// Eligible:  type === 'interbank_transfer' AND direction === 'debit'
//            AND status is 'pending' OR 'failed'
//
// For 'pending': cancel hub record first so the receiving bank cannot
credit
//                the recipient after we have already refunded the
sender.
// For 'failed':  hub record was never written (or is already failed),
safe to
//                refund without touching the hub.
//
// The operation is NOT globally atomic (two different Firestore
projects), but
// is idempotent — re-running on an already rolled-back transaction is
rejected.

```

```

export const rollbackTransaction = async (transactionId, staffUid) => {
  const txnRef      = doc(privateDb, 'transactions', transactionId);
  const hubRef      = doc(hubDb, 'interbank_transfers', transactionId);
  const refundRef   = doc(collection(privateDb, 'transactions'));

  // — 1. Read the local transaction record
  

---


  const txnSnap = await getDoc(txnRef);
  if (!txnSnap.exists()) throw new Error('Transaction not found.');
```

const txn = txnSnap.data();
 if (txn.type !== 'interbank_transfer')
 throw new Error('Only interbank transfers can be rolled back.');

if (txn.direction !== 'debit')
 throw new Error('Only debit-side transactions can be rolled
back.');

if (!['pending', 'failed'].includes(txn.status))
 throw new Error(`Transaction status is "\${txn.status}" - rollback
not allowed.`);

const accountRef = doc(privateDb, 'accounts', txn.fromAccountId);

// — 2. Cancel the hub record first (for pending transfers)

// This prevents the receiving bank from crediting the recipient
after we

// have returned funds to the sender.

```

if (txn.status === 'pending') {
  try {
    const hubSnap = await getDoc(hubRef);
    if (hubSnap.exists() && hubSnap.data().status === 'pending') {
      await updateDoc(hubRef, {
        status:          'cancelled',
        failureReason: `Rolled back by staff (uid: ${staffUid})`,
        completedAt:    serverTimestamp(),
      });
    }
    // If hub record is already cancelled/completed/failed, proceed
    regardless.
  } catch (hubErr) {
    // Hub is unreachable - refuse to proceed to avoid double-credit
    risk.
  }
}
```

```

        throw new Error(`Cannot reach transfer hub to cancel:
${hubErr.message}. Rollback aborted.`);
    }
}

// — 3. Atomically refund sender + mark transaction rolled_back

await runTransaction(privateDb, async (t) => {
    const [accSnap, freshTxnSnap] = await Promise.all([
        t.get(accountRef),
        t.get(txnRef),
    ]);

    if (!accSnap.exists()) throw new Error('Sender account not
found.');
```

```

    const freshStatus = freshTxnSnap.data()?.status;
    if (freshStatus === 'rolled_back')
        throw new Error('This transaction has already been rolled
back.');
```

```

    if (freshStatus === 'completed')
        throw new Error('Transaction already completed – rollback not
permitted.');
```

```

    const newBalance = (accSnap.data().balance ?? 0) + txn.amount;

    // Refund sender balance
    t.update(accountRef, { balance: newBalance, updatedAt:
serverTimestamp() });

    // Mark original transaction as rolled back
    t.update(txnRef, {
        status: 'rolled_back',
        rolledBackAt: serverTimestamp(),
        rolledBackBy: staffUid,
    });

    // Record the refund credit so it appears in the account history
    t.set(refundRef, {
        transactionId: refundRef.id,
        type: 'rollback',
        direction: 'credit',
        fromAccountId: txn.fromAccountId,
```

```

    toAccountId:    txn.fromAccountId,
    fromBankId:     txn.toBankId,
    toBankId:       txn.fromBankId,
    amount:         txn.amount,
    currency:       'INR',
    status:         'completed',
    mode:           'rollback',
    remarks:        `Refund for rolled-back transfer (ref:
${transactionId})`,
    originalTxnId:  transactionId,
    rolledBackBy:   staffUid,
    createdAt:      serverTimestamp(),
    completedAt:    serverTimestamp(),
  });
});

return refundRef.id;
};

// Get all pending interbank debits (rollback candidates)
export const getPendingInterbankDebits = async () => {
  const snap = await getDocs(
    query(
      collection(privateDb, 'transactions'),
      where('type',      '==', 'interbank_transfer'),
      where('direction', '==', 'debit'),
      where('status',    '==', 'pending'),
    )
  );
  return snap.docs
    .map(d => ({ id: d.id, ...d.data() }))
    .sort((a, b) => (b.createdAt?.toMillis?.() ?? 0) -
(a.createdAt?.toMillis?.() ?? 0));
};

// — Read transactions from private DB

```

```

// src/services/transactionService.js - Fix transaction fetching

export const getTransactionsByAccount = async (accountId) => {

  // Query both debit and credit transactions for this account

```

```

const [debitSnap, creditSnap] = await Promise.all([

  getDocs(query(collection(privateDb, 'transactions'),
    where('fromAccountId', '==', accountId))),

  getDocs(query(collection(privateDb, 'transactions'),
    where('toAccountId', '==', accountId))),

]);

// Use Map to prevent duplicates (by transactionId, not document ID)

const map = new Map();

debitSnap.docs.forEach(d => {

  const data = d.data();

  // Use transactionId as key to group related transactions

  map.set(data.transactionId, {

    id: d.id,

    ...data,

    // For debit, the account is the sender

    direction: 'debit',

  });

});

creditSnap.docs.forEach(d => {

  const data = d.data();

  // Don't override if already exists (would be weird), but if
exists,

  // we want to show the direction from THIS account's perspective

```

```

    if (map.has(data.transactionId)) {

        const existing = map.get(data.transactionId);

        // If both exist, keep the one that shows direction from this
account

        // (already handled by separate queries)

    } else {

        map.set(data.transactionId, {

            id: d.id,

            ...data,

            direction: 'credit',

        });

    }

});

return [...map.values()].sort((a, b) =>

    (b.createdAt?.toMillis?.() ?? 0) - (a.createdAt?.toMillis?.() ?? 0)

);

};

export const getAllTransactions = async () => {
    const snap = await getDocs(collection(privateDb, 'transactions'));
    return snap.docs
        .map(d => ({ id: d.id, ...d.data() }))
        .sort((a, b) => (b.createdAt?.toMillis?.() ?? 0) -
(a.createdAt?.toMillis?.() ?? 0));
};

```

src/utils/formatters.js

```
// src/utils/formatters.js

import { format, formatDistanceToNow } from 'date-fns';

// — Currency



---



/**
 * Convert paise (integer) to a formatted INR string.
 * All balances are stored in paise to avoid floating-point errors.
 * 100 paise = ₹1
 */
export const paiseToRupees = (paise) => {
  if (typeof paise !== 'number') return '₹0.00';
  return new Intl.NumberFormat('en-IN', {
    style: 'currency',
    currency: 'INR',
    minimumFractionDigits: 2,
  }).format(paise / 100);
};

/**
 * Convert rupees input (float string) to paise (integer) for storage.
 */
export const rupeesToPaise = (rupees) => {
  return Math.round(parseFloat(rupees) * 100);
};

// — Dates



---



/**
 * Format a Firestore Timestamp or JS Date for display.
 */
export const formatDate = (timestamp) => {
  if (!timestamp) return '-';
  const date = timestamp?.toDate ? timestamp.toDate() : new Date(timestamp);
  return format(date, 'dd MMM yyyy, HH:mm');
};

/**
```

```

    * Relative time (e.g. "3 minutes ago").
    */
export const timeAgo = (timestamp) => {
    if (!timestamp) return '-';
    const date = timestamp?.toDate ? timestamp.toDate() : new
Date(timestamp);
    return formatDistanceToNow(date, { addSuffix: true });
};

// — Account numbers



---



/**
 * Mask an account number for display — show only last 4 digits.
 */
export const maskAccount = (accountNumber) => {
    if (!accountNumber) return '....';
    const str = String(accountNumber);
    return ' '.repeat(Math.max(0, str.length - 4)) + str.slice(-4);
};

/**
 * Generate a random 12-digit account number.
 */
export const generateAccountNumber = () => {
    return Math.floor(1000000000000 + Math.random() *
9000000000000).toString();
};

// — Transfer mode labels



---



export const modeLabel = (mode) => {
    const labels = {
        internal: 'Internal',
        imps:     'IMPS',
        neft:     'NEFT',
    };
    return labels[mode] ?? mode?.toUpperCase() ?? '-';
};

// — Transaction direction



---



```



```

export const directionLabel = (direction) => {
  return direction === 'credit' ? 'Credit' : 'Debit';
};

export const directionColor = (direction) => {
  return direction === 'credit' ? 'text-green-600' : 'text-red-600';
};

export const directionSign = (direction) => {
  return direction === 'credit' ? '+' : '-';
};

// — Status badge colors

```

```

export const statusColor = (status) => {
  const map = {
    completed: 'bg-green-100 text-green-800',
    pending:    'bg-yellow-100 text-yellow-800',
    failed:     'bg-red-100 text-red-800',
    active:     'bg-green-100 text-green-800',
    inactive:   'bg-gray-100 text-gray-600',
    frozen:     'bg-blue-100 text-blue-800',
  };
  return map[status] ?? 'bg-gray-100 text-gray-600';
};

```

src/utils/transferRouter.js

```

// src/utils/transferRouter.js
// Determines transfer type (intrabank / interbank) and validates mode
choice.

import { BANK_ID, TRANSFER_MODES } from '../config/constants';

/**
 * Given a destination bankId, determine if this is an intrabank or
interbank transfer.
 */
export const getTransferType = (toBankId) => {
  return toBankId === BANK_ID ? 'intrabank' : 'interbank';
};

```

```

/**
 * Validate that the chosen mode is appropriate for the transfer type.
 * Intrabank transfers must use 'internal'.
 * Interbank transfers must use 'imps' or 'neft'.
 */
export const validateMode = (toBankId, mode) => {
  const type = getTransferType(toBankId);

  if (type === 'intrabank' && mode !== TRANSFER_MODES.INTERNAL) {
    return { valid: false, reason: 'Use Internal mode for same-bank transfers.' };
  }
  if (type === 'interbank' && mode === TRANSFER_MODES.INTERNAL) {
    return { valid: false, reason: 'Use IMPS or NEFT for transfers to other banks.' };
  }
  return { valid: true };
};

/**
 * Suggest the best mode based on amount and time.
 * IMPS: instant, 24x7, up to ₹5,00,000
 * NEFT: batch, Mon-Sat 8am-7pm, no upper limit (for large transfers)
 */
export const suggestMode = (toBankId, amountInPaise) => {
  if (toBankId === BANK_ID) return TRANSFER_MODES.INTERNAL;

  const amountInRupees = amountInPaise / 100;
  const hour = new Date().getHours();
  const day = new Date().getDay(); // 0=Sun, 6=Sat

  // NEFT business hours: Mon-Sat 8am-7pm
  const neftAvailable = day >= 1 && day <= 6 && hour >= 8 && hour < 19;

  if (amountInRupees > 500000 && neftAvailable) {
    return TRANSFER_MODES.NEFT;
  }
  return TRANSFER_MODES.IMPS;
};

/**
 * Return a human-readable description of the transfer.

```

```

*/
export const transferSummary = (type, mode, amountInPaise) => {
  const rupees = (amountInPaise / 100).toLocaleString('en-IN', {
    style: 'currency', currency: 'INR',
  });

  if (type === 'intrabank') {
    return `Internal transfer of ${rupees} within ${BANK_ID}.`;
  }
  return `${mode.toUpperCase()} transfer of ${rupees} to another bank
via shared hub.`;
};

```

src/utils/validators.js

```

// src/utils/validators.js
import { z } from 'zod';

// — Auth

export const loginSchema = z.object({
  email: z.string().email('Invalid email address'),
  password: z.string().min(6, 'Password must be at least 6
characters'),
});

export const registerSchema = z.object({
  fullName: z.string().min(2, 'Full name must be at least 2
characters'),
  email: z.string().email('Invalid email address'),
  phone: z.string().regex(/^\\d{10}$/, 'Phone must be exactly 10
digits'),
  password: z.string().min(8, 'Password must be at least 8
characters'),
  confirmPassword: z.string(),
}).refine(data => data.password === data.confirmPassword, {
  message: 'Passwords do not match',
  path: ['confirmPassword'],
});

// — Send Money

```

```
export const sendMoneySchema = z.object({
  fromAccountId: z.string().min(1, 'Select a sender account'),
  toAccountNumber: z.string()
    .min(8, 'Account number must be at least 8 digits')
    .max(18, 'Account number too long')
    .regex(/^\d+$/, 'Account number must contain only digits'),
  toBankId: z.string().min(1, 'Select destination bank'),
  amount: z.string()
    .min(1, 'Amount is required')
    .refine(v => !isNaN(parseFloat(v)) && parseFloat(v) > 0, 'Amount
must be greater than 0')
    .refine(v => parseFloat(v) <= 1000000, 'Amount cannot exceed
₹10,00,000 per transaction'),
  mode: z.enum(['imps', 'neft', 'internal'], { required_error:
'Select transfer mode' }),
  remarks: z.string().max(100, 'Remarks too long').optional(),
});

// — Create Account (staff only)
```

```
export const createAccountSchema = z.object({
  customerId: z.string().min(1, 'Customer ID is required'),
  accountType: z.enum(['savings', 'current', 'salary', 'fd']),
  initialDeposit: z.string()
    .min(1, 'Initial deposit is required')
    .refine(v => !isNaN(parseFloat(v)) && parseFloat(v) >= 0, 'Invalid
amount'),
});

// — Create Customer (staff-side)
```

```
export const createCustomerSchema = z.object({
  fullName: z.string().min(2, 'Full name must be at least 2
characters'),
  email: z.string().email('Invalid email address'),
  phone: z.string().regex(/^\d{10}$/, 'Phone must be 10
digits'),
  pan: z.string().regex(/^[A-Z]{5}[0-9]{4}[A-Z]{1}$/, 'Invalid
PAN (e.g. ABCDE1234F)'),
```

```
    aadhaarLast4: z.string().regex(/^\d{4}$/, 'Enter last 4 digits of Aadhaar'),  
  });  
};
```

```
// — Add Staff
```

```
export const addStaffSchema = z.object({  
  name:      z.string().min(2, 'Name is required'),  
  email:     z.string().email('Invalid email'),  
  password:  z.string().min(8, 'Password must be at least 8 characters'),  
  role:      z.enum(['teller', 'manager', 'admin'], { required_error: 'Select a role' }),  
});
```

app.jsx

```
import { Routes, Route, Navigate } from 'react-router-dom';  
import { Layout } from './components/layout/Layout';  
import { ProtectedRoute, StaffRoute, AdminRoute } from './components/shared/ProtectedRoute';  
import { Login } from './pages/auth/Login';  
import { Register } from './pages/auth/Register';  
import { Dashboard } from './pages/Dashboard';  
import { KYCUpload } from './pages/kyc/KYCUpload';  
import { Portfolio } from './pages/portfolio/Portfolio';  
import { Loans } from './pages/loans/Loans';  
import { SendMoney } from './pages/transfer/SendMoney';  
import { TransactionHistory } from './pages/transactions/TransactionHistory';  
import { Accounts } from './pages/accounts/Accounts';  
import { CreateAccount } from './pages/accounts/CreateAccount';  
import { AccountDetails } from './pages/accounts/AccountDetails';  
import { Notifications } from './pages/notifications/Notifications';  
import { AdminDashboard } from './pages/admin/AdminDashboard';  
  
export default function App() {  
  return (  
    <Routes>  
      <Route path="/login" element={<Login />} />  
      <Route path="/register" element={<Register />} />  
    </Routes>  
  );  
}
```

```

    <Route path="/" element={
      <ProtectedRoute>
        <Layout>
          <Routes>
            <Route index element={<Navigate to="/dashboard" replace
/>} />

            <Route path="dashboard"      element={<Dashboard />} />
            <Route path="kyc"             element={<KYCUpload />} />
            <Route path="portfolio"       element={<Portfolio />} />
            <Route path="loans"           element={<Loans />} />
            <Route path="send-money"      element={<SendMoney />} />
            <Route path="transactions"    element={<TransactionHistory
/>} />

            <Route path="notifications" element={<Notifications />}
/>

            <Route path="accounts"      element={<StaffRoute><Accounts
/></StaffRoute>} />
            <Route path="accounts/new"
element={<StaffRoute><CreateAccount /></StaffRoute>} />
            <Route path="accounts/:id"
element={<StaffRoute><AccountDetails /></StaffRoute>} />
            <Route path="admin"
element={<AdminRoute><AdminDashboard /></AdminRoute>} />
            <Route path="*"             element={<Navigate
to="/dashboard" replace />} />
          </Routes>
        </Layout>
      </ProtectedRoute>
    } />
  </Routes>
);
}

```

Index.css

```

* { box-sizing: border-box; }

body { margin: 0; background-color: #F4F5F8; font-family: Arial,
"Helvetica Neue", Helvetica, sans-serif; }

input:-webkit-autofill,
input:-webkit-autofill:hover,
input:-webkit-autofill:focus {
  -webkit-box-shadow: 0 0 0 1000px #ffffff inset !important;
}

```

```
-webkit-text-fill-color: #1A1A1A !important;
}
```

main.jsx

```
// src/main.jsx

import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { ThemeProvider } from '@mui/material/styles';
import CssBaseline from '@mui/material/CssBaseline';
import { Toaster } from 'react-hot-toast';
import { AuthProvider } from '../context/AuthContext';
import { NotificationProvider } from '../context/NotificationContext';
import { createBankTheme } from '../theme';
import App from './App';
import './index.css';

const BANK_COLOR = import.meta.env.VITE_BANK_COLOR || 'blue';
const theme = createBankTheme(BANK_COLOR);

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <BrowserRouter>
      <ThemeProvider theme={theme}>
        <CssBaseline />
        <AuthProvider>
          <NotificationProvider>
            <App />
            <Toaster
              position="top-right"
              toastOptions={{
                duration: 4000,
                style: { borderRadius: 10, fontSize: 14, maxWidth: 380
},
              }}
            />
          </NotificationProvider>
        </AuthProvider>
      </ThemeProvider>
    </BrowserRouter>
  </StrictMode>,
  </>
);
```

```
);
```

theme.js

```
// src/theme.js - HDFC Bank-style MUI theme
import { createTheme } from '@mui/material/styles';

// HDFC Bank brand palette
const HDFC = {
  red: '#E22026',
  redDark: '#B71C1C',
  redLight: '#FDECEA',
  navy: '#003E8A',
  navyDark: '#002D65',
  white: '#FFFFFF',
  pageBg: '#F4F5F8',
  cardBg: '#FFFFFF',
  border: '#DDE1E7',
  borderLight: '#ECEEF2',
  text: '#1A1A1A',
  subtext: '#5A6472',
  placeholder: '#9AA3AE',
  success: '#1B7E47',
  successBg: '#E8F5EE',
  warning: '#B25E00',
  warningBg: '#FFF3E0',
  error: '#C62828',
  errorBg: '#FDECEA',
  info: '#0055A5',
  infoBg: '#E8F0FB',
};

const COLORS = {
  blue: { main: HDFC.red, dark: HDFC.redDark, light: HDFC.redLight,
  sidebar: HDFC.navy },
  purple: { main: HDFC.red, dark: HDFC.redDark, light: HDFC.redLight,
  sidebar: HDFC.navy },
  emerald: { main: HDFC.red, dark: HDFC.redDark, light: HDFC.redLight,
  sidebar: HDFC.navy },
};

export const createBankTheme = (bankColor = 'blue') => {
  const c = COLORS[bankColor] ?? COLORS.blue;
```



```
return createTheme({
  palette: {
    mode: 'light',
    primary: { main: HDFC.red, dark: HDFC.redDark, light:
HDFC.redLight },
    secondary: { main: HDFC.navy, dark: HDFC.navyDark, light:
'#E8EEF8' },
    background: { default: HDFC.pageBg, paper: HDFC.cardBg },
    text: { primary: HDFC.text, secondary: HDFC.subtext,
disabled: HDFC.placeholder },
    divider: HDFC.border,
    success: { main: HDFC.success, light: HDFC.successBg, dark:
'#145C35' },
    error: { main: HDFC.error, light: HDFC.errorBg, dark:
'#8B0000' },
    warning: { main: HDFC.warning, light: HDFC.warningBg, dark:
'#7A4000' },
    info: { main: HDFC.info, light: HDFC.infoBg, dark:
'#003E8A' },
    grey: {
      50: '#F9FAFB',
      100: '#F4F5F8',
      200: '#ECEE2',
      300: '#DDE1E7',
      400: '#B0B8C4',
      500: '#8895A2',
      600: '#5A6472',
    },
  },
  typography: {
    fontFamily: 'Arial, "Helvetica Neue", Helvetica, sans-serif',
    h4: { fontWeight: 700, color: HDFC.text, fontSize: '1.5rem' },
    h5: { fontWeight: 700, color: HDFC.text, fontSize: '1.25rem' },
    h6: { fontWeight: 700, color: HDFC.text, fontSize: '1.1rem' },
    subtitle1: { fontWeight: 600, color: HDFC.text, fontSize:
'0.95rem' },
    subtitle2: { fontWeight: 600, color: HDFC.text, fontSize:
'0.85rem' },
    body1: { color: HDFC.text, fontSize: '0.875rem' },
    body2: { color: HDFC.subtext, fontSize: '0.8125rem' },
    caption: { color: HDFC.subtext, fontSize: '0.75rem' },
  },
});
```

```

shape: { borderRadius: 4 },
shadows: [
  'none',
  '0 1px 3px rgba(0,0,0,0.08)',
  '0 2px 6px rgba(0,0,0,0.08)',
  '0 2px 8px rgba(0,0,0,0.10)',
  '0 4px 12px rgba(0,0,0,0.10)',
  '0 4px 16px rgba(0,0,0,0.12)',
  ...Array(19).fill('0 4px 16px rgba(0,0,0,0.12)'),
],
components: {
  MuiCssBaseline: {
    styleOverrides: {
      body: { backgroundColor: HDfC.pageBg, color: HDfC.text },
      '*': { boxSizing: 'border-box' },
      '::-webkit-scrollbar': { width: 6, height: 6 },
      '::-webkit-scrollbar-track': { background: HDfC.pageBg },
      '::-webkit-scrollbar-thumb': { background: HDfC.border,
borderRadius: 2 },
      '::-webkit-scrollbar-thumb:hover': { background: HDfC.subtext
    },
    'a': { color: HDfC.navy, textDecoration: 'none' },
    'a:hover': { color: HDfC.red, textDecoration: 'underline' },
  },
},
  MuiButton: {
    defaultProps: { disableElevation: true, disableRipple: false },
    styleOverrides: {
      root: {
        borderRadius: 4,
        textTransform: 'none',
        fontWeight: 600,
        fontSize: 13,
        letterSpacing: 0,
        boxShadow: 'none',
        '&:hover': { boxShadow: 'none' },
      },
      containedPrimary: {
        backgroundColor: HDfC.red,
        color: '#fff',
        '&:hover': { backgroundColor: HDfC.redDark },
      },
      containedSecondary: {

```

```

        backgroundColor: HDFC.navy,
        color: '#fff',
        '&:hover': { backgroundColor: HDFC.navyDark },
    },
    outlinedPrimary: {
        borderColor: HDFC.red,
        color: HDFC.red,
        '&:hover': { backgroundColor: HDFC.redLight, borderColor:
HDFC.red },
    },
    outlinedSecondary: {
        borderColor: HDFC.navy,
        color: HDFC.navy,
        '&:hover': { backgroundColor: '#E8EEF8', borderColor:
HDFC.navy },
    },
    text: {
        color: HDFC.navy,
        '&:hover': { backgroundColor: '#E8EEF8' },
    },
},
},
MuiTextField: {
    defaultProps: { variant: 'outlined', size: 'small', fullWidth:
true },
    styleOverrides: {
        root: {
            '& .MuiOutlinedInput-root': {
                backgroundColor: HDFC.white,
                borderRadius: 4,
                fontSize: 13,
                '& fieldset': { borderColor: HDFC.border },
                '&:hover fieldset': { borderColor: '#9AA3AE' },
                '&.Mui-focused fieldset': { borderColor: HDFC.navy,
borderWidth: 1.5 },
            },
            '& .MuiInputLabel-root': { color: HDFC.subtext, fontSize:
13 },
            '& .MuiInputLabel-root.Mui-focused': { color: HDFC.navy },
            '& .MuiInputBase-input': { color: HDFC.text, fontSize: 13
        },
            '& .MuiFormHelperText-root': { fontSize: 11, marginLeft: 0
        },

```

```

    },
  },
},
MuiCard: {
  defaultProps: { elevation: 0 },
  styleOverrides: {
    root: {
      backgroundColor: HD FC.cardBg,
      borderRadius: 4,
      border: `1px solid ${HD FC.border}`,
      boxShadow: '0 1px 3px rgba(0,0,0,0.06)',
    },
  },
},
MuiCardContent: {
  styleOverrides: {
    root: {
      padding: '16px 20px',
      '&:last-child': { paddingBottom: 16 },
    },
  },
},
MuiCardHeader: {
  styleOverrides: {
    root: {
      padding: '14px 20px',
      borderBottom: `1px solid ${HD FC.border}`,
    },
    title: { fontSize: 14, fontWeight: 600, color: HD FC.text },
  },
},
MuiChip: {
  styleOverrides: {
    root: { borderRadius: 3, fontWeight: 600, fontSize: 11,
height: 22 },
    colorSuccess: { backgroundColor: HD FC.successBg, color:
HD FC.success },
    colorError: { backgroundColor: HD FC.errorBg, color:
HD FC.error },
    colorWarning: { backgroundColor: HD FC.warningBg, color:
HD FC.warning },
    colorInfo: { backgroundColor: HD FC.infoBg, color:
HD FC.info },
  },
},

```

```
        colorDefault: { backgroundColor: HDFC.borderLight, color:
HDFC.subtext },
        colorPrimary: { backgroundColor: HDFC.redLight, color:
HDFC.red },
        colorSecondary: { backgroundColor: '#E8EEF8', color:
HDFC.navy },
    },
},
MuiTableCell: {
  styleOverrides: {
    head: {
      fontWeight: 600,
      color: HDFC.subtext,
      fontSize: 11,
      textTransform: 'uppercase',
      letterSpacing: '0.04em',
      backgroundColor: HDFC.pageBg,
      borderColor: HDFC.border,
      padding: '10px 16px',
    },
    body: {
      borderColor: HDFC.borderLight,
      color: HDFC.text,
      fontSize: 13,
      padding: '10px 16px',
    },
  },
},
MuiTableRow: {
  styleOverrides: {
    root: {
      '&:hover': { backgroundColor: '#FAFBFC' },
    },
  },
},
MuiDrawer: {
  styleOverrides: {
    paper: {
      backgroundColor: HDFC.navy,
      color: '#fff',
      width: 240,
      borderRight: 'none',
    },
  },
},
```

```

    },
  },
  MuiListItemButton: {
    styleOverrides: {
      root: {
        borderRadius: 0,
        margin: 0,
        width: '100%',
        padding: '10px 20px',
        color: 'rgba(255,255,255,0.75)',
        borderLeft: '3px solid transparent',
        '&.Mui-selected': {
          backgroundColor: 'rgba(255,255,255,0.10)',
          color: '#ffffff',
          borderLeft: `3px solid ${HDFC.red}`,
          '& .MuiListItemIcon-root': { color: '#ffffff' },
          '&:hover': { backgroundColor: 'rgba(255,255,255,0.13)' },
        },
        '&:hover': {
          backgroundColor: 'rgba(255,255,255,0.06)',
          color: '#fff',
        },
      },
    },
  },
  MuiListItemIcon: {
    styleOverrides: {
      root: { minWidth: 34, color: 'rgba(255,255,255,0.6)' },
    },
  },
  MuiDivider: {
    styleOverrides: { root: { borderColor: HDFC.border } },
  },
  MuiAppBar: {
    styleOverrides: {
      root: {
        backgroundColor: HDFC.white,
        borderBottom: `1px solid ${HDFC.border}`,
        boxShadow: 'none',
      },
    },
  },
  MuiAlert: {

```

```

    styleOverrides: {
      root: { borderRadius: 4, fontSize: 13 },
      standardInfo: { backgroundColor: HDFC.infoBg, border:
`1px solid #B3CDEF`, color: HDFC.info },
      standardWarning: { backgroundColor: HDFC.warningBg, border:
`1px solid #F0C070`, color: HDFC.warning },
      standardError: { backgroundColor: HDFC.errorBg, border:
`1px solid #F5AAAA`, color: HDFC.error },
      standardSuccess: { backgroundColor: HDFC.successBg, border:
`1px solid #8FCBA8`, color: HDFC.success },
    },
  },
  MuiPaper: {
    styleOverrides: {
      root: { backgroundImage: 'none' },
      outlined: { border: `1px solid ${HDFC.border}` },
      elevation1: { boxShadow: '0 1px 3px rgba(0,0,0,0.08)',
border: `1px solid ${HDFC.border}` },
    },
  },
  MuiMenu: {
    styleOverrides: {
      paper: { border: `1px solid ${HDFC.border}`, boxShadow: '0
4px 12px rgba(0,0,0,0.10)', borderRadius: 4 },
    },
  },
  MuiMenuItem: {
    styleOverrides: {
      root: { fontSize: 13, color: HDFC.text, '&:hover': {
backgroundColor: HDFC.pageBg } },
    },
  },
  MuiTabs: {
    styleOverrides: {
      root: { borderBottom: `1px solid ${HDFC.border}`, minHeight:
40 },
      indicator: { backgroundColor: HDFC.red, height: 2 },
    },
  },
  MuiTab: {
    styleOverrides: {
      root: {
        textTransform: 'none',

```

```
        fontWeight: 500,
        fontSize: 13,
        color: HDFC.subtext,
        minHeight: 40,
        padding: '8px 16px',
        '&.Mui-selected': { color: HDFC.red, fontWeight: 600 },
    },
},
},
MuiStepper: {
    styleOverrides: {
        root: { backgroundColor: 'transparent' },
    },
},
MuiStepLabel: {
    styleOverrides: {
        label: { fontSize: 12, color: HDFC.subtext, '&.Mui-active': {
color: HDFC.navy }, '&.Mui-completed': { color: HDFC.success } },
    },
},
MuiStepIcon: {
    styleOverrides: {
        root: { color: HDFC.border, '&.Mui-active': { color:
HDFC.navy }, '&.Mui-completed': { color: HDFC.success } },
    },
},
MuiLinearProgress: {
    styleOverrides: {
        root: { backgroundColor: HDFC.borderLight, borderRadius: 2,
height: 6 },
        bar: { backgroundColor: HDFC.red, borderRadius: 2 },
    },
},
MuiInputBase: {
    styleOverrides: {
        root: { color: HDFC.text, fontSize: 13 },
    },
},
MuiSelect: {
    styleOverrides: {
        root: { backgroundColor: HDFC.white, fontSize: 13 },
    },
},
},
```



```

    MuiTooltip: {
      styleOverrides: {
        tooltip: { backgroundColor: HDFC.text, color: '#fff',
fontSize: 11, borderRadius: 3 },
      },
    },
    MuiDataGrid: {
      styleOverrides: {
        root: {
          border: `1px solid ${HDFC.border}`,
          borderRadius: 4,
          fontSize: 13,
          '& .MuiDataGrid-cell': { borderColor: HDFC.borderLight },
          '& .MuiDataGrid-columnHeaders': { backgroundColor:
HDFC.pageBg, borderColor: HDFC.border },
          '& .MuiDataGrid-row: hover': { backgroundColor: '#FAFBFC' },
          '& .MuiDataGrid-footerContainer': { borderColor:
HDFC.border, backgroundColor: HDFC.pageBg },
        },
      },
    },
    MuiBadge: {
      styleOverrides: {
        colorError: { backgroundColor: HDFC.red },
      },
    },
    MuiAvatar: {
      styleOverrides: {
        colorDefault: { backgroundColor: HDFC.redLight, color:
HDFC.red, fontWeight: 600 },
      },
    },
  });
});

export const SIDEBAR_COLORS = COLORS;
export const HDFC_COLORS = HDFC;

```

index.html

```

<!DOCTYPE html>
<html lang="en">

```

```
<head>
  <meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>BMS – Banking Management System</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```